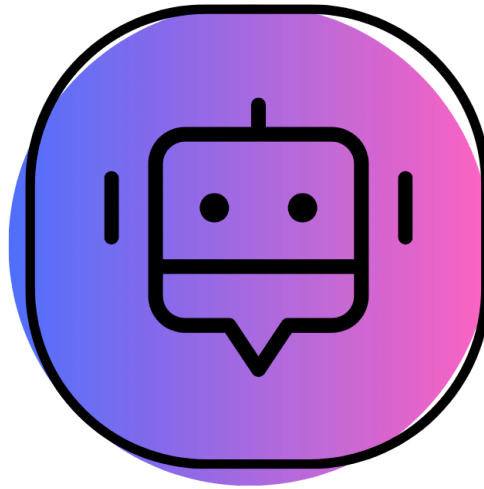




SMART SERVICE

CHAT

סמל מוסד: [REDACTED]

שם המכללה: [REDACTED]

שם הסטודנט: [REDACTED]

ת.ז. סטודנט: [REDACTED]

שם הפרויקט: SmartService Chat.

3.....	1. הצעת פרויקט.....
17.....	2. תקציר.....
21.....	3. מטרות / יעדים.....
21.....	4. אתגרים.....
22.....	5. מדדי הצלחה למערכת.....
22.....	6. רקע תאורטי / ספרות מקצועית (יש לציין מקורות).....
26.....	7. תיאור פתרונות קיימים לבעיה.....
27.....	8. ניתוח חלופות מערכת.....
28.....	9. תיאור החלופה הנבחרת ונימוקים.....
29.....	10. אפיון מערכת.....
32.....	11. תיאור הארכיטקטורה.....
35.....	12. תיאור תהליכי אבטחת מידע במערכת.....
36.....	13. למידת מכונה.....
42.....	14. תיאור התוכנה.....
43.....	15. ניתוח ותרשים Use cases / UML של המערכת המוצעת.....
57.....	16. תיאור מסכים המתאר את היררכיית המסכים והמעברים ביניהם.....
57.....	17. עבור כל מסך באפליקציה.....
63.....	18. הסבר על תפקידי אלמנטים בתצוגת הפרויקט.....
66.....	19. הודעות למשתמש למיניהם.....
66.....	20. ממשק משתמש.....
67.....	21. קוד התוכנית.....
82.....	22. תיאור מסד הנתונים.....
86.....	23. מדריך למשתמש.....
88.....	24. מדריך למשתמש.....
88.....	25. המסקנות.....
89.....	27. ביבליוגרפיה.....

1. הצעת פרויקט

תיאור הפרויקט

בפרויקט זה תפותח מערכת צ'אט חכמה למוקד שירות לקוחות, המשלבת עיבוד שפה טבעית (NLP) וניתוח נתונים לצורך מתן מענה מדויק, מהיר ואישי לקוחות.

נועדה לעזור לחברות ולמוקדי שירות יעילה עם הלקוחות שלהם, לזהות את צרכי הלקוח בזמן אמת, ולשפר את חווית השירות.

בכניסה למערכת המערכת דורש סיסמה לזיהוי המשתמש במידה והמשתמש חדש הוא יצטרך לבצע רישום עם פרטים אישיים ורק לאחר מכן יוכל להיעזר בשירותינו.

המערכת מקליטה/מקבלת-טקסט ומנתחת את צד הלקוח, (בעת הקלטה המערכת ממירה את השמע לטקסט) המערכת אוספת מידע על הלקוח, בעת חוסר מידע על השאלה/לקוח וכולי המערכת שואלת את הלקוח שאלות ממוקדות, המערכת מנתח את השאלה, שואבת את התשובה מבין האתרים, המערכת מטפלת אף בתלונות וכן בכעס/בלבול, (כאשר צד הלקוח היה על ידי תקשור עם הקלטה – המערכת תמיר את הטקסט של התשובה לשמע), בסיום השיחה המערכת מספקת דוחות וסיכום על השיחה עם תובנות מסקנות תמלול ועוד.... ייעוד המערכת היא לשפר את איכות השירות, הדיוק והיעילות של מוקדי השירות על ידי שימוש בבינה מלאכותית לניתוח קולי ולסנטימנט.

מטרת המערכת:

- לספק פתרון מתקדם המשלב בינה מלאכותית, ניתוח טקסטים ונתונים, וזיהוי משתמשים – על מנת לשפר את שירות הלקוחות.
- לחסוך זמן עבודה לנציגים.
- ליצור חווית לקוח חכמה ואישית, זמינות והגעה לשירותי לקוחות רבים בצאט אחד.
- לנהל היסטוריית שיחות לכל לקוח לצורך שיפור השירות בהמשך. מחיקת נתונים ישנים/מיותרים.

הגדרות קלט פלט:

קלט:

- סיסמת משתמש.
- הקלטת הלקוח או טקסט על פי החלטת הלקוח. בזמן אמת.
- התבססות על דוחות סיכום מהשיחות קודמות עם הלקוח. מה הרגיע אותו וכולי...

פלט:

- מענה לשאלת הלקוח או שאלת מיקוד להבנה עמוקה יותר של הנושא. בשמע או בטקסט – על פי צד הלקוח.
- דוחות סיכום שיחה הכוללים תמלול, סנטימנט, הצעות לשיפור, מדדי ביצועים וניקוד למוקדן.
- בזמן פתיחת שיחה חדשה, מזהה האם מדובר בלקוח חדש או לקוח קיים, בודקת אם הנושא קשור לפניות קודמות, וכן עוסק בניקוי רעשי רקע מהשמע.

שלבי הפיתוח:

1. בעת קליטת הנתונים (שמע – ממירה לטקסט/טקסט) המערכת בודקת אם המשתמש הוא לקוח חדש או קיים. בעת הצורך בקשת פרטים נוספים מהלקוח לצורך זיהוי.
2. הבנת משמעות ההודעה: זיהוי הכוונה של הלקוח, סיווג הפנייה לפי נושא, וזיהוי האם השיחה היא המשך לפנייה קודמת או שיחה חדשה לחלוטין.
3. אם חסרים נתונים חשובים לצורך הבנת מכלול הבקשה – המערכת שואלת שאלות ממוקדות כדי להשלים את התמונה ולבנות פרופיל מלא לשאלה.

4. המערכת בודקת אם הלקוח מרוצה, מתוסכל, לחוץ או כועס, לפי טון דיבור ומילים רגשיות. לפי התוצאה היא משנה את אופי התגובה (על פי הסטורית שיחות) או מזהה שמדובר בתלונה ומגיבה בהתאם. כאשר מזהה תלונה — המערכת מסכמת את הבעיה, מחפשת גורם אפשרי לתקלה, ומציעה פתרון, פיצוי ובעת הצורך הפנייה למנהל שירות.
5. שליפה המידע ממסד נתונים, מאגר ידע פנימי או מאתרי מידע. בשלב זה המערכת מנתחת את הנתונים ובוחרת את הפתרון המתאים ביותר.
6. ניסוח תשובה טבעית בהתאם לסגנון השיחה, למצב הרגשי של הלקוח, ולפתרון שנבחר. המערכת דואגת לדיוק, בהירות ונימוס.
7. דוח אוטומטי הכולל: מידע על הלקוח, תמלול שיחה, נושא הפנייה, רמת שביעות הרצון, רגשות שזוהו וצורת טיפול. עם הזמן, המערכת לומדת אילו ביטויים, טונים ותגובות מרגיעים את הלקוחות בצורה היעילה ביותר, בונים אמון ופותרים בעיות - והופכים נתונים אמיתיים להמלצות לשיטות עבודה מומלצות להדרכה ואוטומציה. שמירת הדוח ביומן לקוח.
8. ניהול ההיסטוריה: המערכת שומרת את סיכומי השיחות הקודמות לכל לקוח, מוחקת נתונים לא רלוונטיים או ישנים, וכן מעדכנת המלצות והנחיות בהתבסס על היסטוריית השיחות. במידה והלקוח חדש המערכת יוצרת משתמש חדש ושומרת את הדוח תחת שמו.

סיכום פעילות המערכת:

המערכת מכילה שני משתמשים:

- User-1:** משתמשי הצאט. בכניסה לצאט מקישים קוד משתמש. אם הוא משתמש חדש יהיה עליו להירשם ורק לאחר מכן יוכל להיכנס בכדי להשתמש בשירותינו. על המשתמש לבחור באיזה נושא יהיה מתן השירות. וכעת המשתמש יכול לכתוב/להקליט את בקשתו, הצאט יחזיר אליו את התשובה. משתמש זה יוכל לצפות בהיסטוריית שיחותיו בלבד.
- User-2:** מנהל המערכת. מכניס את סיסמתו. המערכת מזהה שמשתמש זה הינו המנהל ומאפשרת לו להגיע למקומות נוספים. יש למנהל הרשאת גישה לקוד של המערכת החכמה כולה לתקלה או עידכון. ליומן לקוח - הכולל סיכומי שיחה.

הבעיות האלגוריתמיות

SmartService Chat הוא צ'אט חכם המנתח בזמן אמת את ההודעות של המשתמש, מזהה רגשות ותגובות, ומספק מענה מותאם אישית. המערכת משלבת עיבוד שפה טבעית ולמידת מכונה כדי לשפר את האינטראקציה ולהפוך את השיחה ליעילה ומדויקת יותר.

פעמים רבות אנו נפגשים במקרים שבהם לקוח קיבל מידע שגוי/לא מדויק/חסר סעיף מהאיש השרות, וכן מקרים בהם הלקוח התעצבן/דיבר שיח חרשים/התבלבל ממלל המוקדן ונגרם ללקוח ולעיתים אף למוקדן עוגמת נפש מרובה. ועל כן ניצור את המערכת שתפתור בעיות אלו על ידי צ'אט שירות לקוחות המכיל בתוכו תחומים רבים. הלקוח יקבל את מרב השירות, הזמינות, המהירות, הדיוק, האמינות, באופן אובייקטיבי, והכל במקום אחד זמין ונוח. הבעיה העיקרית בפרויקט היא ניתוח טקסט.

שלב א. המרת שמע לטקסט:

- כדי שנוכל לנתח את השיחה להגיע לדיוק מירבי, המערכת ממירה את השיחה לטקסט.

שלב ב. בכניסה למערכת זיהוי משתמש:

- על מנת שהמערכת תוכל לתת שירות מותאם אישית לאדם, על המערכת לזהות את האדם וזאת על ידי הקשת סיסמה.
- במקרה של משתמש חדש: בעת רישום לצאט על הלקוח לתת את פרטיו. במקרה שחסר פרטים המערכת תבקש ממנו באופן ממוקד אלו נתונים חסר לה לצורך שמירתו כמשתמש. בסיום הרישום המערכת תשלח למשתמש סיסמה שעל פיה יוכל להיכנס למערכת.

שלב ג. בדיקה האם השיחה שיחת המשך:

- לעיתים הלקוח רוצה להמשיך את השיחה הקודמת או להתבסס על הנתונים מהשיחה האחרונה, ועל כן על המערכת לבדוק האם שיחה זו שיחת המשך או להתבסס על הנתונים הקודמים.

שלב ד. הבנת משמעות ההודעה:

- על המערכת לבצע על הטקסט ניקוי רעשי רקע כדי שנוכל להתמקד במילים המשמעותיות.
- זיהוי בקשת הלקוח למטרת מענה מתאים.
- הטקסט צריך להיות מובנה ומחולק לפי נושאים כדי שניתן יהיה לנתח את כל חלקי השיחה ולתת מענה מקיף ומדויק ככל האפשר.
- במקרה והלקוח לא נתן מספיק נתונים על בקשתו המערכת תשאל אותו שאלות ממוקדות על הפרטים החסרים להבנת המכלול.

שלב ה. זיהוי תלונה:

- המערכת צריכה לזהות את מצב רוחו של הלקוח: האם הוא כועס, מבולבל, מתעצבן, מרוצה או ניטרלי.
- זיהוי רגשות דורש גם אלגוריתמים לזיהוי דפוסי דיבור, הטונים, והשפה, ולא רק תמלול מילולי.
- כאשר המערכת מזהה אינטונציה תלונתית - על ידי טון דיבור, מילים המשמשות תלונה וכיוצא בזה המערכת תטפל במקרה כתלונה.

שלב ו. טיפול בתלונה:

- שליפת נתונים מהיסטוריית השיחות - מה הרגיע את הלקוח...
- סיכום בעיית התלונה.
- חיפוש גורם אפשרי לתקלה.
- הצעת פתרון/פיצוי.
- בעת הצורך הפנייה למנהל שירות.

שלב ז. שליפת מידע ממסד הנתונים:

- ניתוח הנתונים לצורך מענה נכון ומתאים ביותר.
- בכדי שהמערכת תוכל לענות ללקוח על שאלותיו עליה לשאוב מידע מאתר או ממסד נתונים הקשור לבקשתו.
- בחירת התשובה המתאימה ביותר לבקשה מתוך האתר/מסד נתונים.

שלב ח. ניסוח התשובה:

- בכדי שהלקוח יקבל תשובה מובנת, ברורה, מתאימה וכולי על המערכת לנסח תשובה טבעית בהתאם לסגנון השיחה, למצב הרגשי של הלקוח, ולפתרון שנבחר.
- אם הקלט שהתקבל מהלקוח הוא שמע על המערכת להפוך את התשובה שניסחה ואספה לשמע. וזאת ליצירת שרות הטוב ביותר.

שלב ט. יצירת דוח אוטומטי:

- מטרת המערכת לתת את השירות המקסימלי מותאם לכל אדם על פי אופיו ולחסוך מקרים שבהם השיחה מגיעה למחוזות לא נעימים בשל חוסר הבנה, עלות מחירים... ולכן בכדי שהמערכת תוכל לשפר את השירות במהלך שיחה המערכת יוצרת דף סיכום השיחה הכולל: תמלול השיחה, סנטימנט ורגשות של הלקוח שזוהו. במקרה של כעס - מה הרגיע אותו, הצעות לשיפור. ושומר את הסיכום ביומן לקוחות.
- לאורך זמן, המערכת צריכה ללמוד אלו ביטויים, טונים ותגובות מרגיעים את הלקוחות בצורה המיטבית.

שלב י. שמירת נתונים:

- בסיום השיחה המערכת שומרת את סיכום השיחה ביומן לקוח.

שלב יא. ניהול ההיסטוריה:

- המערכת שומרת לכל לקוח בנפרד את סיכומי השיחות הקודמות. על המערכת לנהל את ההיסטוריה של הסיכומים. מחיקת נתונים לא רלוונטיים עבור כל לקוח, לדוגמא: נתונים ישנים. בעת מחיקת לקוח המערכת מוחקת את כל ההיסטוריה של לקוח זה. בעת זיכרון נתונים מלא, המערכת מרעננת את הזיכרון ומוחקת נתונים שדחופים פחות או מתריעה על מרכז זיכרון חדש.

רקע תאורטי

בעשור האחרון חלה עלייה משמעותית בשימוש במערכות צ'אט ובמוקדי שירות לקוחות דיגיטליים. רוב המערכות שהיו בשימוש בעבר פעלו על בסיס כללים קבועים מראש (Rule-Based Systems) או השתמשו בתבניות תשובה פשוטות, ללא יכולת להבין באמת את כוונת המשתמש או את מצבו הרגשי. תוצאה זו יצרה פעמים רבות שיח לא טבעי, מגושם ולעיתים מתסכל עבור המשתמש.

במקביל, מוקדי שירות הלקוחות סבלו מעומסים כבדים, זמני המתנה ארוכים וחוסר עקביות במתן מענה. נציגי שירות נדרשו להתמודד עם מגוון רחב של פניות בודדות, ללא כלי עזר שיכולים לסייע להם להבין בזמן אמת את רמת שביעות הרצון של הלקוח, את רמת הדחיפות של הפנייה או את רמת הבלבול והמתח שמורגשים בשיחה. כך נוצר פער משמעותי בין הצורך של הלקוחות בחוויית שירות אישית, מהירה ומדויקת — לבין היכולת של מערכות השירות הקיימות לספק מענה זה.

בנוסף, מערכות צ'אט קיימות כמעט ולא כללו יכולות ניתוח רגשי. הן לא היו מסוגלות לזהות האם המשתמש כועס, מתוסכל, רגוע או מבולבל. היעדר הבנה זו מנע מהמערכת להגיב באופן מותאם אישית, לאפשר ניהול שיחה אנושי יותר ולזהות מצבים שבהם הלקוח זקוק להכוונה נוספת. במוקדי שירות, המשמעות הייתה חוסר ביכולת לזהות בזמן אמת שיחה שמתדרדרת, לקוח שמאבד סבלנות או פנייה שמצריכה התערבות מהירה של נציג בכיר.

SmartService Chat פותחת דלת לדור חדש של מערכות תקשורת. היא משלבת טכנולוגיות מתקדמות של עיבוד שפה טבעית (NLP), למידת מכונה וחיזוי רגשות, ומאפשרת ליצור אינטראקציה חכמה ומשמעותית יותר.

היתרונות העיקריים של המערכת הם:

- **הבנת טקסט עמוקה:** המערכת יודעת לזהות כוונות, הקשרים ומשמעויות נסתרים שלא ניתן להגיע אליהם באמצעות חוקים פשוטים.
 - **זיהוי רגשות בזמן אמת:** מאפשר להבין את מצב המשתמש, לזהות מתחים או תסכול, ולהתאים את התגובה בהתאם למצבו.
 - **שיפור מוקדי שירות:** נציגים מקבלים תמונת מצב על רמת שביעות הרצון של הלקוח, יכולים לתעדף פניות בעייתיות ולייעל את השירות.
 - **תגובה דינמית בהתאמה אישית:** המערכת אינה מסתמכת על תבניות קשיחות אלא מתאימה כל תגובה לתוכן ולמצב הרגשי של השיח.
 - **שיפור חוויית השימוש:** השיחה מרגישה טבעית יותר, רציפה וידידותית, מה שמעלה את שביעות רצון המשתמשים ומפחית עומסים על הנציגים.
- באופן זה, SmartService Chat מציעה פתרון כולל שמטפל גם בבעיות של מערכות הצ'אט המסורתיות וגם באתגרים של מוקדי שירות לקוחות — ומייצרת חוויית תקשורת חכמה, יעילה ומותאמת אישית.

פיתוחים הקיימים כיום:

1. הפיכת שמע לטקסט - כשהקלט שמע:
 - **Connectionist Temporal Classification (CTC) Algorithm** : משמש לאימון מערכות זיהוי דיבור להמיר קלט שמע לטקסט, גם אם אורך הקלטת השמע אינו תואם בצורה מושלמת את אורך התמליל הכתוב.
 - **Hidden Markov Models (HMMs)** : מודלים סטטיסטיים של מרקוב בהמרת דיבור לטקסט הם מודלים סטטיסטיים המסייעים לקבוע את רצף המילים והתווים הסביר ביותר על סמך צלילים מדגימת שמע.
 - **Whisper ASR** : היא מערכת זיהוי דיבור בקוד פתוח מבית OpenAI, שאומנה על בסיס מערך נתונים גדול ומגוון של 680,000 שעות של נתונים רב-לשוניים וריבוי משימות שנאספו מהאינטרנט. Whisper יכולה לתמלל דיבור באנגלית ובמספר שפות אחרות, ויכולה גם לתרגם ישירות ממספר שפות שאינן אנגלית לאנגלית.
 - **DeepSpeech** : היא מערכת זיהוי דיבור בקוד פתוח שפותחה על ידי מוזילה בשנת 2017 ומבוססת על אלגוריתם הומונימי של באידו.

- **Wav2vec** : מתוצרת הענקית Meta, היא ערכת כלים לזיהוי דיבור המתמחה באימון עם נתונים לא מותיגים, בניסיון לכסות כמה שיותר ממרחב השפות המכסה שפות המיוצגות בצורה גרועה במערכי הנתונים המבוארים המשמשים בדרך כלל לאימון בפיקוח.
 - **Kaldi** : היא ערכת כלים לזיהוי דיבור שנכתבה ב-C++, שנולדה מתוך הרעיון של קוד מודרני וגמיש שקל לשנות ולהרחיב. חשוב לציין, ערכת הכלים של Kaldi מנסה לספק את האלגוריתמים שלה בצורה הגנרית והמודולרית ביותר האפשרית, כדי למקסם את הגמישות והשימוש החוזר (אפילו לקוד מבוסס בינה מלאכותית אחר מחוץ לתחום פעילותה של kladi).
 - **SpeechBrain** : היא ערכת כלים "הכל באחד" לדיבור. משמעות הדבר היא שהיא לא רק מבצעת ASR, אלא את כל מערך המשימות הקשורות לבינה מלאכותית שיחתית: זיהוי דיבור, סינתזת דיבור, מודלים של שפה גדולה ואלמנטים אחרים הנדרשים לאינטראקציה טבעית מבוססת דיבור עם מחשב או צ'אטבוט.
 - **Sonix** : המרה מדויקת של דיבור לטקסט ביותר מ-53 שפות. עורך הדפדפן של Sonix מאפשר לך לחפש, להשמיע, לערוך, לארגן ולשתף את התמלילים שלך מכל מקום ובכל מכשיר. מושלם לפגישות, הרצאות, ראיונות, סרטים... כל סוג של אודיו או וידאו, למעשה.
2. אלגוריתמים לזיהוי אדם - על ידי סיסמה:
- **Argon2** : הוא האלגוריתם המתקדם ביותר כיום לאחסון סיסמאות. הוא משתמש בגישה שנקראת Memory Hard, כלומר דורש הרבה זיכרון בעת חישוב Hash, מה שמקשה מאוד על תוקפים להשתמש ב-GPU או בחומרה ייעודית לפיצוח סיסמאות.
 - יש לו שלושה גרסאות (Argon2i, Argon2d, Argon2id) המותאמות לסוגי התקפות שונים, ומקובל להשתמש ב-Argon2id.
 - **Bcrypt** : הוא אלגוריתם ותיק ואמין מאוד, שנמצא בשימוש נרחב כבר שנים רבות. הוא מוסיף Salt באופן מובנה ומחשב Hash בצורה 'איטית' בכוונה, כדי להאט תקיפות כוח-גס. ניתן לשלוט על רמת הקושי (cost factor), מה שמאפשר להגביר אבטחה ככל שהחומרה מתחזקת.
 - **PBKDF2** : אלגוריתם תקין ומוכח, בשימוש נרחב במערכות אבטחה רשמיות. פועל בעזרת חזרות רבות של פונקציית HMAC (בדרך כלל עם SHA-256) בשילוב Salt. אמנם פחות מוגן מפני תקיפות מבוססות חומרה לעומת argon2 ו-bcrypt, אך עדיין נחשב בטוח מאוד ומקובל ביישומים ארגוניים.
 - **Scrypt** : אלגוריתם שתוכנן במיוחד כדי להקשות על מתקפות GPU באמצעות צריכת זיכרון גבוהה. בדומה ל-Argon2 גם scrypt הוא Memory Hard, ולכן פיצוח שלו דורש גם זמן וגם משאבי RAM משמעותיים. נחשב חזק ומתאים למערכות הדורשות אבטחה גבוהה מאוד.
3. אלגוריתמים להשוואת טקסט לבדיקת המשכיות שיחה:
- **N-gram Continuation Check** : בודק האם תחילת טקסט B מכילה את אותם רצפים (N מילים) שמסתיימים בטקסט A. שיטה זו מתמקדת ברצף מילים מילולי ולא במשמעות, ולכן טובה במיוחד כשמדובר במשפטים או פסקאות קצרות שבהן המשכיות מילולית ברורה.
 - **המודל (Semantic Similarity (Embeddings)** : ממיר את כל טקסט למרחב וקטורי שבו כל טקסט מיוצג על ידי משמעותו. לאחר מכן מחשבים דמיון קוסינוס בין הווקטורים. דמיון גבוה מצביע על כך ש-B ממשיך את רעיון או נושא A, גם אם המילים עצמן שונות.
 - **Language Model Continuation Probability** : מודל שפה (כמו GPT או BERT) מחשב את ההסתברות שהטקסט B יופיע במשך לטקסט A. אם ההסתברות גבוהה, המשמעות היא שהשפה הטבעית של B מתאימה כתוצאה טבעית מהקשר A.
 - **Pattern Continuation / Structural Similarity** : בודק האם B שומר על אותה תבנית או סגנון של A: אורך משפטים, שימוש במילים מסוימות, נושא, זמני פעלים וכו'. דמיון גבוה בין התכונות המרובות של שני הטקסטים מראה שהמשכיות אפשרית גם אם המילים המדויקות שונות.

- **Levenshtein Distance** : מודד את מספר השינויים (הוספות, מחיקות, החלפות) הדרושים כדי להפוך את סוף A לתחילת B. מרחק קטן מצביע על המשכיות טכנית או מילולית, בעיקר שימושי לטקסטים שבהם המשכיות צריכה להיות כמעט זהה מבחינה מילולית.

שיטת NLP

NLP מחולק ל- 2 מרכיבים עיקריים.

- הבנת שפה טבעית - ייצוג הטקסט בצורה שימושית והבנה מהיבטים שונים.
 - יצירת שפה טבעית - תכנון הטקסט שכולל בחירת מילים ונתינת הטון למשפט.
- SmartService Chat ישתמש בשני חלקים אלו. הבנת השפה טבעית - על מנת להבין את בקשת המשתמש. יצירת שפה טבעית - על מנת ליצור תשובות מובנות למשתמש באופן אישי ככל האפשר.

קשיים:

- עמימות מילונית- מילה עם דו משמעות.
 - עמימות תחבירית- משפט עם דו משמעות.
 - עמימות התייחסותית- דו משמעות כאשר של גוף נסתר.
- לדוגמא: הילדה סיפרה לאימה על החברה שהרביצה לה, היא מאוד כעסה. מי כעסה האמא או הילדה?

מרכיבי NLP

1. ניתוח טקסט ועיבוד מקדים:

- **Tokenization** - כאשר אני מקבלת טקסט אני מנתחת אותו לאסימונים. אסימון יכול להיות חלוקה למשפטים מטקסט ארוך או מילים ממשפט. אני קובעת את החלוקה וכמה מילים יהיו בכל אסימון. טוקניזציה היא תהליך של פירוק טקסט נתון ליחידות הנקראות אסימונים. אסימונים יכולים להיות מילים בודדות, ביטויים או אפילו משפטים שלמים. בתהליך של אסימון, תווים כמו סימני פיסוק עלולים להימחק. האסימונים הופכים בדרך כלל לקלט עבור תהליכים כמו ניתוח וכריית טקסט. פירוק משפט לחלקיו מאפשר למכונה להבין את החלקים כמו את השלם. זה יעזור לתוכנית להבין כל אחת מהמילים בפני עצמה, וגם כיצד הן פועלות, מאחרים יותר בעיבוד שפה טבעית. הדיוק של האסימון מבוסס על האוצר מילים שהוא מאומן עליו.
- **Stemming** - מציאת מקור המילה - השורש - מציאת השורש ע"י הסרת תחילת וסוף המילה. צריך לשים לב שלא להרוס את המשמעות.
- **Lemmatizat** - הוצאת מילים לא משמעותיות למשפט. מילים לא משמעותיות הן דבר משתנה. בכל ניתוח טקסט יש מילים אחרות שהן לא משמעותיות. תלוי מה מנתחים. יש ניתוחים בהם התארים של הפעלים או שמות העצם יהיו חשובים ואפילו קריטיים לניתוח ולעומת זאת לעיתים המילים החשובות יהיו הפעלים, שמות העצם, או תאורי הזמן - עבר, הווה, עתיד וכו'.
- **Text Normalization**: סטנדרטיזציה של טקסט, כולל נרמול אותיות גדולות וקטנות, הסרת פיסוק ותיקון שגיאות כתיב.
- **tags POS** - מתייג את האסימונים לפעלים שמות עצם וכו'.

2. תחביר וניתוח:

- **Part-of-Speech (POS) Tagging**: הקצאת חלקי דיבר לכל מילה במשפט (למשל, שם עצם, פועל, שם תואר).
- **Dependency Parsing**: ניתוח המבנה הדקדוקי של משפט כדי לזהות קשרים בין מילים.
- **Constituency Parsing**: פירוק משפט לחלקים או לביטויים המרכיבים אותו (למשל, צירופי עצם, צירופי פעלים).

3. ניתוח סמנטי:

- **Named Entity Recognition (NER)**: זיהוי וסיווג ישויות בטקסט, כגון שמות של אנשים וארגונים, מיקומים, תאריכים וכו'.

- המודל (WSD) Word Sense Disambiguation: קביעת המשמעות של מילה בה נעשה שימוש בהקשר נתון.
 - Coreference Resolution: זיהוי מתי מילים שונות מתייחסות לאותה ישות בטקסט (למשל, "הוא" מתייחס ל"יוחנן").
 - 4. חילוף מידע:
 - Entity Extraction: זיהוי ישויות ספציפיות והקשרים ביניהן בתוך הטקסט.
 - Relation Extraction: זיהוי וסיווג הקשרים בין ישויות בטקסט.
 - 5. סיווג טקסטים ב-NLP:
 - Sentiment Analysis: קביעת הרגש או הטון הרגשי המובעים בטקסט (למשל, חיובי, שלילי, ניטרלי).
 - Topic Modeling: זיהוי נושאים או תמות בתוך אוסף גדול של מסמכים.
 - Spam Detection: סיווג טקסט כספאם או לא ספאם.
 - 6. עיבוד דיבור:
 - Speech Recognition: המרת שפה מדוברת לטקסט.
 - Text-to-Speech (TTS) Synthesis: המרת טקסט כתוב לשפה מדוברת.
 - 7. מענה לשאלות:
 - Retrieval-Based QA: מציאת והחזרת קטע הטקסט הרלוונטי ביותר בתגובה לשאלתה.
 - Generative QA: יצירת תשובה המבוססת על המידע הזמין בקורפוס טקסט.
 - 8. מערכות דיאלוג:
 - Chatbots and Virtual Assistants: מתן אפשרות למערכות לנהל שיחות עם משתמשים, לספק תגובות ולבצע משימות על סמך קלט המשתמש.
 - 9. ניתוח סנטימנטים ורגשות ב-NLP:
 - Emotion Detection: זיהוי וסיווג של רגשות המובעים בטקסט.
 - Opinion Mining: ניתוח דעות או ביקורות כדי להבין את הסנטימנט הציבורי כלפי מוצרים, שירותים או נושאים.
-
- 4. אלגוריתמים לניקוי מילות עצירה, סימני פיסוק, רווחים מיותרים, מספרים חסרי משמעות, הטיות מילים:
 - **Gensim** : ג'נסים (Generate Similar) היא ספריית תוכנה בקוד פתוח המשתמשת בלמידת מכונה סטטיסטית מודרנית. על פי ויקיפדיה, ג'נסים נועדה לטפל באוספי טקסט גדולים באמצעות הזרמת נתונים ואלגוריתמים מקוונים מצטברים, מה שמבדיל אותה מרוב חבילות התוכנה האחרות בלמידת מכונה המכוונות רק לעיבוד בזיכרון.
 - **Scikit-Learn** : בספריית Scikit-Learn, קיימים כלים מובנים לניקוי טקסטים, ובהם תמיכה בסינון מילות עצירה — כלומר מילים שכיחות מאוד שלא מוסיפות משמעות מהותית למשפט (כמו: "של", "על", "עם", "the", "and"). הספרייה כוללת רשימת מילות עצירה מוכנה מראש עבור אנגלית ('stop_words='english'), אך מאפשרת גם להזין רשימת מילות עצירה מותאמת אישית.
 - **SpaCy** : ספריית spaCy היא אחת מספריות ה-NLP המתקדמות ביותר, והיא כוללת מנגנון מובנה וחזק לטיפול במילות עצירה. בניגוד לספריות פשוטות יותר, spaCy לא רק מוחקת מילות עצירה — היא יודעת לזהות אותן כחלק מהעיבוד הלשוני העמוק שהיא מבצעת.
 - **NLTK** : היא אחת הספריות הוותיקות והנפוצות ביותר בתחום עיבוד שפה טבעית. היא כוללת מאגר גדול של מילות עצירה עבור שפות רבות, ומספקת כלים פשוטים וגמישים לניקוי טקסטים.
 - **TextBlob** : היא ספריית NLP ידידותית וקלה לשימוש, המבוססת על NLTK אך מספקת ממשק פשוט ונוח הרבה יותר. היא מיועדת במיוחד למי שרוצה לבצע משימות עיבוד טקסט בסיסיות בצורה מהירה, בלי להעמיק באלגוריתמים מסובכים.

- **OpenRefine** : הוא כלי קוד פתוח המיועד לניקוי נתונים, סידור מידע מבולגן, איחוד ערכים, ו-המרת מבנה נתונים. הוא משמש חוקרי נתונים, אנליסטים ומפתחים לעבודות שבהן הנתונים מגיעים מפוזרים, לא אחידים, מלאי שגיאות — וצריך להפוך אותם לנתונים נקיים ומסודרים.

5. חלוקת הטקסט לנושאים:

- **NLTK Sentence Tokenizer** : ערכת הכלים לשפה טבעית (NLTK) מספקת פונקציה שימושית לפיצול טקסט למשפטים. טוקניזטור משפטים זה מחלק גוש טקסט נתון למשפטים המרכיבים אותו, אשר לאחר מכן ניתן להשתמש בהם לעיבוד נוסף.
- **Evaluating Spacy Sentence Splitter** : מפצל המשפטים של Spacy נוטה ליצור קטעים קטנים יותר בהשוואה למפצל טקסט התווים של Langchain, מכיוון שהוא דבק בקפדנות בגבולות המשפט. זה יכול להיות יתרון כאשר יחידות טקסט קטנות יותר נחוצות לניתוח.
- **Adjacent Sequence Clustering** : שיטה זו מייצרת התפלגות מגוונת יותר, דבר המעיד על גישתה התלויה בהקשר. על ידי קיבוץ באשכולות המבוססים על דמיון סמנטי, היא מבטיחה שהתוכן בתוך כל מקטע יהיה קוהרנטי תוך מתן אפשרות לגמישות בגודל המקטע. שיטה זו עשויה להיות יתרון כאשר חשוב לשמר את הרציפות הסמנטית של נתוני טקסט.

6. אלגוריתמים לזיהוי רגשות לצורך זיהוי תלונת לקוח:

- **Naive Bayes** : אלגוריתם זה מסתמך על הסתברות סטטיסטית כדי לקבוע האם טקסט מסוים הוא תלונה. הוא בוחן את תדירות המילים ומחשב את הסיכוי שמשפט שייך לקטגוריה מסוימת על בסיס דוגמאות קודמות. למרות הפשטות שלו, הוא מדויק מאוד במקרים שבהם יש מילים אופייניות שמופיעות לעיתים קרובות בתלונות, ולכן הוא נפוץ מאוד בעיבוד שפה טבעית.
- **Support Vector Machines (SVM)** : המודל : אלגוריתם SVM מחפש את הגבול הברור ביותר בין טקסטים שהם "תלונה" לבין טקסטים שהם "לא תלונה". הוא מייצג כל טקסט כנקודה מרובת-מאפיינים ובונה קו שמפריד בין הקבוצות בצורה מיטבית. היכולת שלו להתמודד עם נתונים מורכבים וגבולות סיווג לא ליניאריים הופכת אותו לאחד האלגוריתמים המדויקים ביותר בניתוח טקסטים.
- **Logistic Regression** : רגסיה לוגיסטית ממירה טקסט לוקטור מאפיינים, ולאחר מכן מעריכה את ההסתברות שהוא מייצג תלונה. המודל פשוט להבנה, ביצועי, ומספק תוצאות ברורות — האם הטקסט שייך לקבוצת התלונות או לא. הוא יעיל במיוחד במערכות שירות לקוחות שבהן יש צורך בחיזוי דו-ערכי מהיר.
- **Random Forest** : אלגוריתם זה מורכב ממספר רב של "עצים" שכל אחד מהם מנתח את הטקסט מזווית אחרת. כל עץ מבצע סיווג משלו, והמודל משלב את התוצאות של כולם כדי להגיע להחלטה סופית. הגישה הזו עמידה לרעש בטקסט ולסגנונות כתיבה שונים, ולכן היא טובה במיוחד לזיהוי תלונות לא אחידות.
- **CNN To Text** : למרות שפותח במקור עבור תמונות, CNN מצטיין גם בניתוח טקסט באמצעות זיהוי "תבניות" של מילים או צירופי מילים שמאפיינות תלונות. הוא מחליק על הטקסט כמו פילטר ומאתר אזורים חשובים. זהו מודל מהיר ויעיל במיוחד עבור הודעות קצרות או משפטים ברצף.
- **Speech Emotion Recognition (Audio)** : המודל : בניתוח תלונות מתוך שמע, אלגוריתם זיהוי רגשות מנתח את טון הדיבור, העוצמה, המהירות והקצב כדי לזהות רגשות כמו תסכול, כעס או לחץ. גם אם הלקוח לא אומר מילים ברורות של תלונה, שינוי בקולו יכול להצביע על בעיה — והאלגוריתם מזהה זאת.

7. אלגוריתמים לשליפת נתונים ממסד נתונים/מאתרים:

- **Web Scraping + XPath/CSS Selectors** : זוהי השיטה הקלאסית לשליפת מידע מאתרי אינטרנט: המערכת מורידה את דף ה-HTML ואז משתמשת ב-XPath או CSS Selectors כדי לאתר ולחלץ תגים, טקסטים או מבנים ספציפיים. היתרון הוא דיוק גבוה וקוד יחסית פשוט, אך היא רגישה לשינויים בעיצוב האתר.

- **API-Based Retrieval** : כאשר אתר מספק API (לרוב JSON/REST), האלגוריתם מבצע שאילתות ישירות לממשק הרשמי ללא צורך בניתוח HTML. זהו המנגנון המהיר והמדויק ביותר לשליפת מידע ממערכות, ומפחית תקלות בעת שינוי מבנה הדף.
 - **Database extraction or querying** : חילוץ מסד נתונים הוא תהליך של אחזור נתונים ספציפיים ממסד נתונים למטרות כמו ניתוח, דיווח, אינטגרציה או הגירה. זה כרוך בשאילתה על מסד הנתונים כדי לחלץ את המידע הדרוש והמרתו לפורמט מובנה שניתן להשתמש בו על ידי יישומים או מערכות אחרות.
 - **ETL (Extract-Transform-Load) Pipelines** : אלגוריתמי ETL נועדו לשלוף נתונים ממקורות שונים (אתרים, APIs, DBs), לבצע ניקוי/עיבוד ואז לטעון אותם למערכת מרכזית. השיטה מבוססת לרוב על מנועי תזמון (כמו Apache Airflow) ומספקת שליפה רציפה ומבוקרת.
8. אלגוריתמים לניסוח התשובה בצורה אנושית:
- **Abstractive Summarization – Seq2Seq Models** : אלגוריתמים מבוססי Sequence-to-Sequence עם Attention (כמו LSTM או Transformer) אינם רק מצטטים משפטים קיימים, אלא מנסחים מחדש בצורה ברורה וקצרה. המערכת "מבינה" את ההקשר ובונה ניסוח חדש, מה שהופך אותם לבסיס נפוץ למערכות מענה אוטומטיות.
9. אלגוריתמים הפיכת טקסט לשמע: במקרה שהלקוח על פונקצית שמע.
- **VITS (Variational Inference TTS)** : מהמודלים המתקדמים ביותר כיום. VITS מייצר את השמע ישירות ללא צורך בשני שלבים (ללא ווקודר נפרד). השיטה מחברת בין Variational Autoencoders, Normalizing Flows ו-GANs ומייצרת קול טבעי מאוד, עם דיקציה מצוינת ואינטונציה אנושית. נחשב ל- "State of the Art".
 - **Concatenative Synthesis Models** : מודלים של Synthesis משולשת מייצגים גישה מסורתית בתחום טכנולוגיית המרת טקסט לדיבור (TTS). מודלים אלה פועלים באמצעות קטעי דיבור מוקלטים מראש שנאספו בקפידה מקולות אנושיים אמיתיים. כל קטע, לרוב פונמה או הברה, מאוחסן במסד נתונים עצום. כאשר מתקבל קלט טקסט, המודל בוחר קטעים מתאימים ומשרשר אותם ליצירת פלט דיבור קוהרנטי.
 - **Parametric Synthesis Models** : מודלים של סינתזה פרמטרית מציעים גישה רב-תכליתית לטכנולוגיית טקסט לדיבור. הם יוצרים דיבור על סמך פרמטרים פונטיים במקום להסתמך על קטעי קול מוקלטים מראש. מודלים אלה ממירים טקסט לייצוגים פונטיים ולאחר מכן מסנתזים דיבור באמצעות פרמטרים כמו גובה צליל, משך זמן ואינטונציה.
 - **Neural Network-Based Models** : מודלים מבוססי רשתות עצביות משתמשים בשיטות סטטיסטיות ולמידת מכונה כדי לייצר דיבור. הם ממנפים אוספים נרחבים של דגימות דיבור כדי ללמד רשתות עצביות כיצד לייצר דיבור המחקר אינטונציות ומקצבים אנושיים.
 - **Murf Falcon** : מתוכנן לספק דיבור דמוי אדם עם זמן השהייה מוביל בתעשייה של 55 מילישניות ברחבי העולם. השתמשו ב-Falcon כדי לפרוס סוכני קול מבוססי בינה מלאכותית שלא רק מדברים כמו בני אדם רגילים, אלא גם מספקים את הדיבור במהירות מסחררת ובדיוק אולטרה-מדויק. Falcon הוא ממשק ה-TTS היחיד ששומר באופן עקבי על זמן השמע הראשון מתחת ל-130 מילישניות על פני +10 אזורים גלובליים, אפילו בעת עיבוד של עד 10,000 שיחות בו זמנית. Falcon מספק דיבור טבעי ללא הפרעות. ללא השהייה, ללא ביטויים מקוטעים, ללא צליל רובוטי.

מודלים שנבחרו

- **BERTopic** - עבור מציאת נושא מתוך טקסט. BERTopic חזק בזיהוי נושאים סמנטיים מתוך טקסט חופשי. הוא מאפשר למערכת להבין במהירות את ההקשר של השיחה, גם כשמדובר בטקסטים מורכבים או מגוונים, ולספק ניתוח משמעותי של תוכן המשתמש.

- **S-BERT - להשוואה בין טקסטים.**
S-BERT מצטיין במדידת דמיון סמנטי בין טקסטים. היתרון שלו הוא הדיוק בהשוואה בין נושאים שונים, מה שמאפשר למנוע כפילויות ולארגן את המידע בצורה עקבית וממוקדת.
- **DistilBERT fine-tuned - להבנת משמעות הטקסט.**
DistilBERT מהיר וקל, ובזכות ה-fine-tuning הוא מבין היטב את הכוונה של המשתמש. היתרון שלו הוא יכולת זיהוי כוונה גם בטקסטים קצרים או לא פורמליים, מה שמגביר את הדיוק במענה לצרכי המשתמש.
- **T5/BART - ליצירת שאלות ממוקדות על פרטים חסרים.**
T5 ו-BART מצטיינים ביכולת לייצר טקסטים בצורה טבעית וחכמה. היתרון הוא שהם מאפשרים למערכת לשאול את השאלות הנכונות בדיוק ובאופן שהלקוח ירגיש טבעי ומעורב בשיחה, בלי להיראות מכני.
- **DistilBERT fine-tuned - זיהוי תלונה מתוך טקסט.**
מודל זה חזק בסיווג טקסטים ומסוגל לזהות מתי מדובר בתלונה. היתרון הוא שהמערכת יכולה להתמקד רק במידע הרלוונטי, ולא להתבלבל בין הערות, בקשות או תלונות, מה שמיעיל את התגובה.
- **MFCC → BiLSTM - זיהוי רגש תלונה מתוך שמע.**
שילוב זה מאפשר לזהות רגשות מתוך הקול של המשתמש. היתרון הוא שהוא מתרגם אינטונציה, טון ועוצמת דיבור למידע אמיתי על רגשות, וכך מאפשר הבנה עמוקה יותר של חומרת התלונה.
- **BART - סיכום התלונה.**
המודל BART מצטיין ביצירת סיכומים ברורים וטבעיים. היתרון הוא שהמערכת יכולה להבין במהירות את עיקרי התלונה בלי לפספס פרטים חשובים, תוך שמירה על ניסוח קולח ואנושי.
- **DistilBERT fine-tuned - מציאת סיבת התלונה.**
DistilBERT כאן מספק דיוק בזיהוי הגורם העיקרי לתלונה. היתרון הוא שהוא מסנן מידע לא רלוונטי ומספק ניתוח ממוקד שמוביל לפתרון יעיל.
- **T5/BART fine-tuned - יצירת פיצוי או פתרון לבעיה.**
T5 ו-BART מאפשרים ניסוח פתרונות מותאמים אישית. היתרון הוא שהמערכת יכולה להציע מענה מקצועי, ברור ונעים ללקוח, שכולל פיצוי או פתרון שמרגיש טבעי ואנושי.
- **מודל GPT (LLM) - שליפת הנתונים**
GPT מצטיין בגמישות וביכולת להתמודד עם שאלות חופשיות. היתרון הוא שהמערכת יכולה לשלוף מידע רלוונטי גם כשבקשת המשתמש מורכבת או לא מדויקת, מבלי לאבד דיוק.
- **המודל S-BERT - לבחירת התשובה המתאימה ביותר.**
מודל זה מאפשר מדידה מדויקת של דמיון בין בקשת המשתמש לאפשרויות התשובה. היתרון הוא הבחירה המדויקת ביותר עבור כל מקרה, מה שמעלה את איכות התשובה הסופית ומפחית טעויות.
- **המודל BART - ניסוח התשובה למשתמש בצורה טבעית.**
BART מצטיין ביצירת ניסוח קולח, מקצועי וטבעי. היתרון הוא שהתשובות שמתקבלות מרגישות אנושיות, נוחות לקריאה ומקצועיות, מה שמגביר את חוויית המשתמש.

הליכים עיקריים בפתרון בעיה בטכנולוגיות הנדסה מתקדמות

- זיהוי משתמש על ידי הזנת קוד.
- בדיקה על פי השאלה האם שיחה זו שיחת המשך.
- קבלת שמע או טקסט מהמשתמש המכיל את הבקשה.
- ניתוח הבקשה.
- חיפוש ובחירת תשובה ממסדי נתונים או אתרי אינטרנט.
- ניסוח התשובה בצורה אנושית.
- החזרת התשובה למשתמש. שמע או טקסט על פי צד המשתמש.
- בסיום השיחה שמירת סיכום השיחה.

הליכים עיקריים בלמידת מכונה

1. למידת מכונה עבור המרת אודיו לטקסט :

נתונים ל-Dataset:

- הקלטות קול של אנשים שונים.
- תמלול מדויק לכל הקלטה.
- מגוון סגנונות דיבור, מבטאים ומהירויות.
- הקלטות עם וריאציות של רעשי רקע.

מודלים מתאימים:

- מודלים אקוסטיים + מודלי שפה, לדוגמא: CNN + RNN* .
- *מודלי Sequence-to-Sequence .
- *מודלים מודרניים כמו Wav2Vec2, Whisper .
- המודל לומד לקשר בין מאפייני קול, תווים/ מילים.

אימון:

- המודל לומד לקשר בין מאפייני קול, תווים/ מילים.
- שילוב של מודל אקוסטי ומודל שפה לדיוק גבוה.

מדדי הצלחה:

- WER – Word Error Rate (כמה מילים שגויות).
- CER – Character Error Rate .
- השוואת רמת שגיאות התמלול בין סוגים שונים של קטעי דיבור.

2. למידת מכונה עבור ניתוח טקסט:

נתונים ל-Dataset:

- משפטים וטקסטים עם תיוגים, כגון: רגש: חיובי/ שלילי/ ניטרלי. סיווג כוונה: שאלה, טענה, בקשה. ונושאים (Topic Classification).

מודלים מתאימים:

- Bag-of-Words/TF-IDF + SVM/Logistic Regression .
- המודל (Word2Vec / GloVe + LSTM) Word Embeddings .
- מודלים מתקדמים: BERT או RoBERTa .

אימון:

- הכנסת טקסט כווקטורים.
- המודל לומד לזהות רגש, כוונה וסגנון.

מדדי הצלחה:

- דיוק.
- Precision / Recall / F1-score .
- בלבול בין רגשות.

3. למידת מכונה עבור זיהוי רגשות:

נתונים ל-Dataset:

- משפטים משיחות אמיתיות.
- תיוג רגשות: כעס, בלבול, שביעות רצון, תלונה, נייטרלי וכו'.
- אם נדרש: גם מאפיינים קוליים (pitch, energy).

מודלים מתאימים:

- Text Emotion Classifiers: BERT Emotion, RoBERTa, DistilBERT
- Audio Emotion Recognition (SER): CNN + LSTM, wav2vec2 finetuned

אימון:

- שילוב טקסט + פרמטרים קוליים (מודל מולטי-מודלי).
- Fine-tuning על דוגמאות של לקוחות אמיתיים.
- לימוד דפוסים המעידים על תלונה/כעס/תסכול.

מדדי הצלחה:

- F1 Score לכל רגש.
- Emotion Confusion Matrix

4. למידת מכונה עבור הפיכת טקסט לשמע:

נתונים ל-Dataset:

- זוגות טקסט-שמע.
- מאגרי דיבור נקיים: משפט → קובץ קול.
- אפשר לגוון דוברים, טונים וקצבים (אם צריך).

מודלים מתאימים:

- Tacotron 2 – ליצירת ספקטרוגרמה.
- FastSpeech / FastSpeech2 – דגמים מהירים יותר.
- WaveGlow / HiFi-GAN – ווקודרים להמרת ספקטרוגרמה לשמע.

אימון:

- שלב 1: אימון Tacotron/FastSpeech על זוגות טקסט-מל-ספקטרוגרמה.
- שלב 2: אימון ווקודר להמרת ספקטרוגרמה לשמע אמיתי.
- Fine-tuning על דגימות שלך לשמירה על סגנון דיבור עקבי.

מדדי הצלחה:

- MOS (Mean Opinion Score): דירוג אנושי על איכות הקול.
- טבעיות (Naturalness).
- הבהירות (Intelligibility).
- זמן ריצה ליצירת שמע.

הליכים עיקריים בתחום מערכות הפעלה/רשתות מחשבים/תקשורת נתונים/אבטחת

מידע

ארכיטקטורת המערכת (לקוח- שרת):

- צד לקוח: דפדפן בו המועמד והחברה משתמשים בממשק אינטרנטי (טופס הגדרות לחברה, מסך ריאיון למועמד).
- צד שרת: שרת יישומים שמקבל בקשות מהלקוח, מעבד את האודיו והטקסט, ומחזיר תוצאות ודו"חות.
- מסד נתונים: שרת בסיס נתונים שבו נשמרים נתוני משתמשים, שאלות, תמלולים ותוצאות ניתוח.

התקני קלט/פלט:

- מחשב.
- עכבר.
- מקלדת.
- מיקרופון.
- רמקול.

תהליכי תקשורת בין לקוח לשרת:

- שליחת בקשת התחברות מהלקוח לשרת לצורך זיהוי ואימות.
- קבלת שאלה מהשרת עבור הבוט.
- שליחת קובץ שמע/טקסט של שאלת המשתמש לשרת.
- העברת תוצאות העיבוד מהשרת ללקוח.
- שליחת בקשה לקבלת דו"ח מסכם.
- החזרת דו"ח מסכם מהשרת ללקוח.

אבטחת מידע:

- הצפנת תקשורת בין הלקוח לשרת.
- שמירת נתונים רגישים (שאלות, תמלולים) במסד הנתונים בצורה מאובטחת ומוגנת.
- הגבלת גישה לפי תפקיד: משתמש רגיל רואה רק את הנתונים והיסטורית שיחות שלו, מנהל רואה את כל הסטוריות השיחות וכן את סיכומי השיחות.
- אימות משתמשים בעת כניסה למערכת כדי למנוע גישה של גורם לא מורשה.

תיאור פרוטוקולי תקשורת

פרוטוקול HTTPS.

פיתוחים עתידיים

- **הרחבת יכולות הניתוח:** שילוב רכיבים מתקדמים לניתוח עומק של תשובות, כמו זיהוי דקויות ניסוח ומורכבות חשיבה. המערכת תוכל להציע קטגוריות עומק נוספות ולהבין סוגי הבעה שונים בצורה מדויקת יותר.
- **הרחבה יכולות שפתיות:** הרחבת תמיכה לשפות נוספות כמו אנגלית וערבית על מנת להנגיש את הכלי ליותר אנשים.
- **שרות פרואקטיבי:** הצאת מנתח את אופן פעולות המשתמש וחזרה פעולות נוספות ומקפיץ הודעה למשתמש על כך.
- **אינטגרציה רב-ערוצית:** הצאת יכול לסנכרן נתונים בין אפשרויות שימוש שונות. כלומר המשתמש יכול להשתמש בשירותי הצאת באותה עת גם מהמייל וגם מהשיחה קולית.

תיאור טכנולוגיה הנדסה

צד שרת - Python , C#.

צד לקוח - react.

בצד שרת נבדקו החלופה הבאה: C++
C++ היא שפה "קרובה לחומרה", שנותנת למתכנת שליטה ידנית מלאה על הזיכרון ומציעה ביצועים מקסימליים. היא משמשת למערכות קריטיות כגון מנועי משחקים ומערכות הפעלה.
#C היא שפה "ברמה גבוהה", מודרנית, ומופעלת על ידי סביבת .NET. היא מנהלת את הזיכרון באופן אוטומטי, מה שהופך אותה לקלה יותר לתכנות. היא אידיאלית לפיתוח יישומי ווב ואפליקציות.
Python שפת תכנות ברמה גבוהה, קלה ונוחה לשימוש, שפה דינמית, שפה מפורשת מה שמקל על איתור באגים היתרון הבולט שלה זה התמיכה הרחבה בספריות שיכולות לעזור בעת כתיבת הפרויקט-מתאימה ללמידת מכונה.
בצד לקוח נבדקו החלופה הבאה: Angular.
React - מספקת פתרון חלקי הכולל רק ספרייה, יש לעדכן את הקוד לעיתים קרובות לפי העדכונים של הספרייה.
Angular - מספקת פתרון מלא, יש לה מלא ספריות דבר שיכול לחסוך את תהליך התכנון.

פרטים פורמליים

מסד נתונים:

הפרויקט נעזר במסד נתונים Sql לשמירת משתמשים ודוחות שיחה.

לוחות זמנים:

נובמבר: חקירה מעמיקה על הנושאים.
 דצמבר: המשך החקירה והתחלת הכתיבה.
 ינואר: כתיבת צד שרת ואימון המודלים.
 פברואר-מרץ: סיום צד שרת ומעבר לצד לקוח.
 אפריל: חיבור בין צד שרת ללקוח.
 מאי: שיפור ושיוף של המערכת.

מנחות הפרויקט:



חתימת הסטודנט:



חתימת רכזת המגמה:



2. תקציר

הפרויקט "SmartService Chat" עוסק בפיתוח מערכת צ'אט חכמה למתן שירות לקוחות בצורה מהירה, נוחה ויעילה.

המערכת תומכת בקלט טקסטואלי וקולי. במקרה של שאלה קולית, המערכת ממירה את הדיבור לטקסט, מנתחת את השאלה, מחפשת מידע מתאים במאגרי מידע ובמאמרים, ובונה תשובה רלוונטית למשתמש. במקרה הצורך, התשובה יכולה לחזור גם בצורה קולית.

בשלב הראשון המערכת מסווגת את שאלת המשתמש לאחד מתוך 12 סוגי שאלות מוגדרים, יוצרת עבורה מבנה JSON מתאים, ומתוך הנתונים שנשלפו בונה שאילתת חיפוש מדויקת וקצרה יותר.

בנוסף, חברות המעוניינות בכך יכולות להצטרף למערכת בתשלום. כאשר נשאלת שאלה הקשורה למוצר מסוים הקיים במספר חברות, המערכת נותנת עדיפות למידע של חברות שנמצאות במאגר, ורק לאחר מכן מבצעת חיפוש כללי.

בסיום השיחה נשמר במסד הנתונים דו"ח שיחה הכולל את רצף השאלות והתשובות, ניתוח סנטימנט, רגשות המשתמש ונושאי השיחה.

מטרת הפרויקט היא לשפר את שירות הלקוחות, לקצר זמני מענה, ולאפשר זמינות גבוהה וחווית שימוש נוחה ואמינה.

2.1. הרקע לפרויקט:

כיום שירות לקוחות הוא חלק מרכזי כמעט בכל חברה ועסק. לקוחות מצפים לקבל מענה מהיר, זמין וברור, אך בפועל פעמים רבות קיימים זמני המתנה ארוכים, עומס על נציגים וקושי למצוא מידע מדויק במהירות.

לכל חברה יש צ'אט שירות נפרד משלה, ולכן הלקוח צריך לפנות לכל עסק בנפרד כדי לקבל מידע או תמיכה.

הפרויקט SmartService Chat מציע פתרון שונה: מערכת צ'אט אחת המרכזת שירות עבור מגוון חברות ועסקים. במקום שהמשתמש יעבור בין אתרים וצ'אטים שונים, הוא יכול לשאול שאלה במקום אחד ולקבל מענה בהתאם לחברה או לנושא הרלוונטי.

הצורך בפרויקט נובע מהרצון לייעל את תהליך שירות הלקוחות, לקצר זמני מענה, להפחית עומס מנציגי שירות, ולספק ללקוח תשובה מהירה, זמינה ומדויקת. מערכת כזו שימושית במיוחד עבור חנויות אונליין, חברות שירות, עסקים קטנים, מוקדי תמיכה וכל מקום שבו לקוחות שואלים שאלות על מוצרים, מחירים, הזמנות, תשלומים ותקלות.

2.2. תהליך המחקר:

במסגרת המחקר המקדים לפרויקט, בוצעה למידה של תחום שירות הלקוחות ושל אופן מתן מענה ללקוחות בעסקים וחברות.

המחקר כלל הבנה של סוגי פניות נפוצות של לקוחות, כגון שאלות על מחירים, הזמנות, תשלומים, החזרות, תקלות ומידע על מוצרים. בנוסף, נבדקו הקשיים המרכזיים בתחום, כמו זמני המתנה ארוכים, עומס על נציגי שירות, וחוסר נוחות במעבר בין צ'אטים שונים של חברות שונות.

כמו כן, נלמדו עקרונות בתחום עיבוד שפה טבעית, במטרה להבין כיצד ניתן לנתח שאלות של משתמשים, לזהות את נושא הפנייה, ולבנות תשובה מתאימה באופן אוטומטי. נלמד צורת עבודה של צ'אטים, חיפוש תשובה, מבנה תשובה, והחזרתה למשתמש.

המחקר אפשר הבנה רחבה של תחום שירות הלקוחות, והיווה בסיס לפיתוח מערכת צ'אט אחת המרכזת שירות עבור מגוון חברות ועסקים.

2.3. סקירת ספרות:

תחום ניתוח השפה הטבעית (Natural Language Processing) מהווה אחד התחומים המרכזיים במחקר הבינה המלאכותית, ומתמקד ביכולת של מערכות מחשב להבין, לנתח ולהפיק משמעות מטקסטים בשפה אנושית.

מחקרים בתחום מצביעים על מעבר משיטות סטטיסטיות פשוטות למודלים מתקדמים מבוססי למידת מכונה ורשתות נוירונים עמוקות. בפרט, מודלים מסוג Transformers מאפשרים הבנה הקשרית של טקסט, תוך התחשבות ביחסים בין מילים בתוך המשפט ומחוצה לו.

לצד זאת, תחום זיהוי הדיבור (Speech Recognition) עבר התפתחות משמעותית, כאשר מודלים מתקדמים מצליחים להמיר דיבור לטקסט ברמת דיוק גבוהה, גם בתנאים מורכבים הכוללים רעשי רקע ושונות בין דוברים. עם זאת, יכולת זו מתמקדת בהמרת מידע ואינה כוללת הבנה של משמעות התוכן.

בנוסף, במערכות חכמות קיימת חשיבות לייצוג מובנה של שאלת המשתמש. במקום להסתמך רק על הטקסט המקורי, ניתן לחלץ ממנו שדות מרכזיים לפי סוג השאלה, וכך לבצע חיפוש ממוקד יותר ללא איבוד נתונים חשובים.

בנוסף, קיימת חשיבות לתהליך חיפוש מידע מתוך מאגרי ידע ומסמכים. מערכות מתקדמות נדרשות לא רק למצוא טקסטים הקשורים לשאלה, אלא גם לבחור מתוכם את המידע המדויק והרלוונטי ביותר עבור המשתמש.

בתחום שירות הלקוחות קיימים מחקרים העוסקים בצמצום זמני מענה, שיפור חוויית המשתמש, והפחתת עומס מנציגי שירות באמצעות מערכות אוטומטיות. שירות לקוחות כולל מגוון רחב של פניות, כגון בירור מחירים, מעקב אחר הזמנות, תשלומים, החזרות, תקלות טכניות, מידע על מוצרים ותלונות. בשל ריבוי סוגי הפניות, קיימת חשיבות לזיהוי נכון של נושא השאלה ולהפנייתה למידע המתאים.

כמו כן, בתחום זה קיימת חשיבות לזמינות גבוהה ולמתן מענה מהיר וברור. לקוחות מצפים לקבל תשובה ללא המתנה ממושכת וללא צורך לעבור בין מספר גורמי שירות. לכן, מערכות חכמות לשירות לקוחות נדרשות לשלב בין הבנת שפה טבעית, חיפוש מידע מדויק, שמירת הקשר השיחה והתאמה בסיסית לצורכי המשתמש.

בנוסף, קיימת חשיבות להתאמה אישית בסיסית של השירות, כגון שמירת היסטוריית שיחות, זיהוי נושאי פנייה חוזרים, והתאמת המענה לצורכי המשתמש.

הפרויקט הנוכחי משתלב בתחומים אלו באמצעות פיתוח מערכת צ'אט אחת, המרכזת שירות עבור מגוון חברות ועסקים, מנתחת שאלות משתמשים, מחפשת מידע מתאים, מתחשבת במידע בסיסי על המשתמש, ומפיקה תשובה רלוונטית בצורה מהירה ונוחה.

2.4. אתגרים מרכזיים:

הפרויקט מציב מספר אתגרים אלגוריתמיים מרכזיים, כאשר עיקרם מתמקד בהבנת שפה טבעית, ניתוח שאלות משתמשים ויצירת תשובה מתאימה.

האתגר המרכזי הוא פיתוח מנגנון המסוגל להבין את משמעות השאלה של המשתמש, ולא רק לזהות מילות מפתח. המערכת נדרשת לזהות את נושא הפנייה, לסווג לפי סוג הפנייה, להבין את כוונת המשתמש, ולאתר את המידע הרלוונטי ביותר מתוך מאגרי המידע הקיימים.

אתגר נוסף הוא סיווג השאלה לאחד מתוך סוגי שאלות מוגדרים, וחילוף הנתונים המתאימים לכל סוג למבנה JSON ייעודי. תהליך זה דורש הבנה של אילו פרטים חשובים לכל סוג פנייה, כדי שהחיפוש יתבצע בצורה מדויקת.

בנוסף, קיים אתגר בהישענות על שיחות קודמות של המשתמש. המערכת צריכה לשמור רצף שיחה, להבין הקשר משאלות קודמות, ולהשתמש במידע זה כדי לתת מענה מדויק ואישי יותר.

אתגר נוסף הוא יצירת תשובה טבעית וברורה למשתמש. לאחר איתור המידע המתאים, המערכת צריכה לנסח תשובה בשפה פשוטה, רציפה ומובנת, כך שהמשתמש יקבל מענה שנראה טבעי ולא רק אוסף של משפטים שנלקחו ממקורות שונים, מבנה השאלה משתנה לפי סוג הפנייה.

2.4.1. הבעיה איתה התמודד התלמיד:

הבעיה המרכזית בפרויקט היא פיתוח מערכת המסוגלת להבין שאלות של משתמשים, להסתמך על שיחות קודמות, לאתר עבורן מידע רלוונטי, ולהפיק תשובה ברורה וטבעית בתחום שירות הלקוחות, מבנה השאלה יהיה לפי סוג השאלה.

המערכת נדרשת:

- לזהות את בקשת המשתמש מתוך ניסוח חופשי של שאלה.
- לסווג כל שאלה לאחד מתוך 12 סוגי שאלות.
- לבנות מבנה JSON מתאים לפי סוג השאלה.
- לחלץ מתוך השאלה את הנתונים החשובים למבנה זה.
- לבנות מתוך ה-JSON שאילתת חיפוש קצרה ומדויקת.
- לחפש מידע מתאים מתוך מאגרי ידע ומאמרים, ממסדי נתונים - חברות ששילמו ומהאינטרנט.
- לבחור את המשפטים או הפסקאות המתאימים ביותר לשאלה.
- להיעזר בהיסטוריית השיחה כדי להבין הקשר ולהחזיר תשובה מדויקת יותר.
- לנסח תשובה סופית בצורה טבעית וברורה למשתמש לפי סוג הפנייה.

תהליך המרת קול לטקסט אינו חלק מהבעיה האלגוריתמית המרכזית, אלא מהווה רכיב עזר המספק קלט טקסטואלי למערכת.

2.4.2. הסיבות לבחירת הנושא:

הנושא נבחר מתוך עניין בתחום הבינה המלאכותית, צורת פעולת צ'אטים ובוטות חכמים, ובפרט בתחום עיבוד השפה הטבעית והבנת טקסט.

תחום זה מציב אתגרים מעניינים, משום שהוא דורש מהמחשב להתמודד עם שפה אנושית חופשית, הכוללת הקשרים, משמעויות מרומזות ודקויות לשוניות. להבין כוונה, לזהות הקשר, ולבחור תשובה מתאימה מתוך מידע קיים.

בנוסף, נבחר נושא בעל רלוונטיות מעשית גבוהה לעולם האמיתי. כולנו נתקלים לעיתים בהמתנה ארוכה לקבלת מענה גם עבור שאלה קטנה ופשוטה. מערכת חכמה לשירות לקוחות יכולה לקצר זמני המתנה, לשפר את זמינות השירות, ולאפשר למשתמש לקבל תשובה מהירה ונוחה במקום אחד.

2.4.3. מוטיבציה לעבודה:

המוטיבציה לפרויקט נובעת מהרצון לפתח מערכת חכמה המסוגלת להבין שפה אנושית בצורה טובה, ולא רק לבצע חיפוש טכני של מילים או נתונים.

הפרויקט מאפשר לשלב בין ידע תיאורטי לבין יישום מעשי, תוך התמודדות עם בעיות אמיתיות בתחום הבנת השפה, זיהוי כוונת המשתמש, חיפוש מידע רלוונטי ויצירת תשובה מתאימה מתוך טקסט.

בנוסף, קיימת מוטיבציה ליצור מערכת בעלת ערך ממשי, היכולה לשפר את תהליך שירות הלקוחות, לקצר זמני המתנה, ולרכז מענה עבור כמה חברות ועסקים במקום אחד, ולהקל על משתמשים בעולם האמיתי.

2.4.4. על איזה צורך הפרויקט עונה? איזה פתרון פרויקט זה באה לתת:

הפרויקט נותן מענה לצורך ממשי בתחום שירות הלקוחות, שבו משתמשים רבים נדרשים להמתין זמן רב למענה או לעבור בין צ'אטים ואתרים שונים של חברות שונות.

קהל היעד של המערכת כולל:

- לקוחות המחפשים תשובה מהירה לשאלה.
- משתמשים המעוניינים לקבל שירות מכמה חברות במקום אחד.
- עסקים וחברות המעוניינים לשפר את השירות ללקוחותיהם.
- מוקדי שירות המעוניינים להפחית עומס מנציגים.

המערכת עונה על מספר צרכים מרכזיים:

- מתן מענה מהיר וזמין לשאלות לקוחות.
- ריכוז שירות עבור מגוון חברות ועסקים במערכת אחת.
- הבנת שאלות בניסוח חופשי ולא רק לפי מילות מפתח.
- הפיכת שאלה חופשית של משתמש למבנה נתונים מסודר, המאפשר חיפוש מדויק יותר.
- חיפוש מידע רלוונטי מתוך מאגרי ידע ומאמרים.
- יצירת תשובה ברורה וטבעית למשתמש.
- שמירת היסטוריית שיחות לצורך התאמה טובה יותר בהמשך.

בניגוד לצ'אט רגיל של חברה אחת, המערכת שמה דגש על שירות מרוכז למגוון חברות, הבנת כוונת המשתמש, והתאמת התשובה למידע הרלוונטי ביותר.

2.4.5. הצגת פתרונות לבעיה:

במסגרת המחקר המקדים נבחנו מספר גישות לפתרון בעיית מתן שירות לקוחות מהיר וזמין.

הגישה הראשונה היא שירות לקוחות אנושי, שבו נציג שירות עונה לפניות הלקוחות. גישה זו מאפשרת מענה אישי ומדויק, אך דורשת זמן, כוח אדם וזמינות של נציגים, ולעיתים גורמת להמתנה ארוכה. עומס רב על המוקדן יכול לגרום למתן תשובה שגויה, קוצר רוח ואף זלזול.

גישה נוספת היא שימוש בצ'אט שירות נפרד לכל חברה או עסק. פתרון זה קיים כיום במקומות רבים, אך הוא מחייב את המשתמש לפנות לכל חברה בנפרד, ואינו מספק מקום מרכזי אחד לקבלת שירות.

גישה נוספת היא ייצוג מובנה של שאלות לפי סוג פנייה. בגישה זו השאלה מסווגת תחילה לנושא מוגדר, ולאחר מכן נשלפים ממנה נתונים חשובים למבנה JSON מתאים. שיטה זו מאפשרת להפוך שאלה חופשית לשאלת חיפוש קצרה ומדויקת יותר.

גישה נוספת כוללת מערכות צ'אט אוטומטיות בסיסיות, המבוססות בעיקר על שאלות קבועות או מילות מפתח. מערכות אלו יכולות לתת מענה מהיר, אך הן מוגבלות כאשר המשתמש שואל שאלה בניסוח חופשי או כאשר נדרש להבין את ההקשר של השיחה.

מסקנת המחקר הייתה כי קיים צורך במערכת מרכזית וחכמה, המסוגלת לתת שירות למגוון חברות ועסקים, להבין שאלות משתמשים, לחפש מידע רלוונטי ולהפיק תשובה ברורה וטבעית.

3. מטרות / יעדים

מטרת הפרויקט מבחינתי כמפתחת היא לפתח מערכת צ'אט חכמה לשירות לקוחות, המסוגלת להבין שאלות משתמשים, לאתר מידע רלוונטי, ולהחזיר תשובה ברורה וטבעית.

היעדים המרכזיים:

- פיתוח מנגנון להבנת שאלות בשפה חופשית.
- פיתוח מנגנון לסיווג שאלות ל-12 סוגי פניות.
- בניית מבני JSON ייעודיים לכל סוג שאלה.
- חילוץ נתונים חשובים מתוך שאלת המשתמש לצורך חיפוש מדויק.
- חיפוש מידע מתאים מתוך מאגרי ידע ומאמרים.
- בחירת המשפטים או הפסקאות הרלוונטיים ביותר לשאלה.
- יצירת תשובה סופית בצורה טבעית וברורה.
- שילוב אפשרות לקלט קולי והמרתו לטקסט.
- שמירת היסטוריית שיחה וניתוח בסיסי של נושאי השיחה ורגשות המשתמש.
- בניית מערכת מרכזית המספקת שירות עבור מגוון חברות ועסקים במקום אחד.

4. אתגרים

במהלך העבודה על הפרויקט התמודדתי עם מספר קשיים:

- ניהול זמנים תחת עומס-השילוב בין פיתוח הפרויקט לבין עמידה בלוח"ז הצפוף של המבחנים החיצוניים. זה דרש ממני לתכנן את הזמנים בצורה קפדנית כדי לוודא שכל שלב בפרויקט מתקדם בזמן, גם בתקופות הכי לחוצות.
- בעיות טכניות סביב התקנת ספריות וסביבות עבודה מורכבות.
- קושי בהבנת והטמעת מודלים מתקדמים של עיבוד שפה טבעית.
- בחירת המודל המתאים לכל שלב במערכת, כגון סיווג נושא, חיפוש מידע, חילוץ תשובה ובניית תשובה סופית.

- אימון מודל על שאלות שירות לקוחות וחלוקתן לנושאים מוגדרים כמו מחירים, הזמנות, תשלומים, החזרות ותקלות.
 - קושי ביצירת Dataset מתאים, הכולל דוגמאות מגוונות מספיק כדי שהמודל ילמד לזהות נכון את סוג הפנייה.
 - הבנה איזה מבני נתונים מתאימים לניהול המידע במערכת, כדי לאפשר חיפוש יעיל, שמירת שיחות, דירוג התאמות, ניהול חברות הרשומות במערכת ושליפת תשובות.
 - התמודדות עם שאלות חופשיות של משתמשים, שלא תמיד מנוסחות בצורה ברורה או ישירה.
 - שילוב בין כמה רכיבים אלגוריתמיים, כגון ניקוי טקסט, Embedding, חישוב דמיון סמנטי, DistilBERT ו-FLAN-T5.
- קשיים אלו דרשו למידה עצמאית, ניסוי וטעייה, בדיקת פתרונות שונים ושיפור הדרגתי של המערכת.

5. מדדי הצלחה למערכת

הצלחת המערכת תימדד על פי מספר קריטריונים מרכזיים:

- עד כמה התשובה שהמערכת מחזירה עונה על שאלת המשתמש.
 - יכולת המערכת להבחין בין נושאים שונים, כגון מחירים, הזמנות, תשלומים, החזרות ותקלות.
 - עד כמה המערכת מצליחה למלא נכון את שדות ה-JSON מתוך שאלת המשתמש.
 - עד כמה השאלות שנבנית מתוך ה-JSON קצרה, מדויקת ושומרת על הנתונים החשובים.
 - יצירת תשובה במבנה מתאים בהתאם לסוג השאלה שזוהה.
 - יכולת להסתמך על שאלות ותשובות קודמות כדי להבין הקשר ולספק מענה מדויק יותר.
 - עבודה תקינה עם סוגים שונים של טקסטים, ניסוחים חופשיים ושאלות שאינן מנוסחות באופן ברור.
 - בחירת מקורות מידע, משפטים או פסקאות הרלוונטיים ביותר לשאלה.
 - הערכת המשתמשים לגבי מהירות, בהירות ורלוונטיות התשובה.
- בנוסף, הצלחה תוגדר גם ביכולת המערכת להפיק תובנות בעלות ערך מתוך השיחה, כגון נושאי הפנייה המרכזיים, סנטימנט ורגשות המשתמש, ולא להסתפק רק בהחזרת נתונים גולמיים.

6. רקע תאורטי / ספרות מקצועית

הפרויקט עוסק בפיתוח מערכת צ'אט חכמה לשירות לקוחות, ולכן הוא משלב כמה תחומי ידע: שירות לקוחות, חוויית משתמש, עיבוד שפה טבעית, חיפוש מידע, זיהוי דיבור ובניית תשובה אוטומטית.

שירות לקוחות הוא תחום מרכזי בעולם העסקי, משום שהוא משפיע על שביעות רצון הלקוח, על אמון המשתמש בחברה ועל יעילות תהליך קבלת המענה. כיום קיימות מערכות אוטומטיות המסייעות בקיצור זמני תגובה ובמתן מענה ראשוני ללקוח, אך עדיין קיים צורך במערכת שמרכזת שירות עבור כמה חברות במקום אחד, ולא מחייבת את המשתמש לעבור בין צ'אטים שונים. מתוך IBM.

תחום עיבוד השפה הטבעית — NLP — עוסק ביכולת של מחשבים להבין, לנתח ולהפיק משמעות מטקסט או מדיבור בשפה אנושית. בפרויקט זה התחום בא לידי ביטוי בניתוח שאלת המשתמש, זיהוי נושא הפנייה, הבנת כוונת השאלה, וחיפוש מידע מתאים מתוך מקורות קיימים. מתוך IBM.

אחד הפיתוחים המרכזיים בתחום הוא מודל ה-Transformer, המבוסס על מנגנון Attention. מנגנון זה מאפשר למודל להתחשב בקשרים בין מילים שונות בטקסט, גם כאשר הן אינן סמוכות זו לזו. על בסיס רעיון זה פותחו מודלים מתקדמים רבים להבנת שפה, ובהם BERT, המסוגל להבין טקסט באופן הקשרי ולשמש למשימות כמו מענה על שאלות והבנת משפטים. מתוך arXiv.

בפרויקט נעשה שימוש ברעיונות של ייצוג טקסט באמצעות Embedding, כלומר המרת משפטים או קטעי טקסט לייצוג מספרי. ייצוג זה מאפשר להשוות בין שאלת המשתמש לבין קטעי מידע קיימים, ולבחור את הקטעים הקרובים ביותר מבחינת משמעות ולא רק מבחינת מילים זהות. מחקרים בתחום Sentence-BERT מציגים שימוש ב-Sentence Embeddings לצורך השוואת דמיון סמנטי בין משפטים בצורה יעילה. מתוך arXiv.

לצורך חילוץ תשובה מתוך טקסט רלוונטי, ניתן להשתמש במודלים ממשפחת BERT, ובפרט ב-DistilBERT, שהוא מודל קטן ומהיר יותר שנבנה על בסיס BERT. יתרונו הוא בכך שהוא שומר על יכולות הבנת שפה טובות, תוך שימוש יעיל יותר במשאבי מחשב. מתוך arXiv.

בנוסף, לצורך יצירת תשובה טבעית וברורה, קיימים מודלים המבוססים על הוראות, כמו FLAN-T5. מודלים אלו מאומנים להגיב להנחיות שונות, ולכן מתאימים למשימות שבהן יש צורך להפוך מידע שנמצא במקורות שונים לתשובה מנוסחת וברורה למשתמש. מתוך arXiv.

המערכת כוללת גם אפשרות לקלט קולי. תחום זיהוי הדיבור עוסק בהמרת דיבור אנושי לטקסט כתוב, ובכך מאפשר למשתמשים לפנות למערכת גם באמצעות קול. עם זאת, זיהוי הדיבור הוא רק שלב קלט; לאחר ההמרה נדרשת הבנת תוכן השאלה והפקת תשובה מתאימה. מתוך IBM.

לסיכום, הרקע התיאורטי של הפרויקט מבוסס על שילוב בין עולם שירות הלקוחות לבין תחום עיבוד השפה הטבעית. המערכת נדרשת לא רק לקבל שאלה ולהחזיר מידע, אלא להבין את משמעות הפנייה, למצוא מידע רלוונטי, להסתמך על הקשר בסיסי של המשתמש, ולנסח תשובה ברורה, מהירה ונוחה.

6.1. מושגים עיקריים בתחום הדעת של הבעיה:

שירות לקוחות הוא תחום שמטרתו לתת מענה לפניות של לקוחות לפני קנייה, במהלך שימוש במוצר או בשירות, ולאחר ביצוע רכישה. התחום כולל שאלות, בקשות, תלונות, בירורים ותמיכה בבעיות שונות.

פניית לקוח היא הודעה שבה המשתמש מבקש מידע או עזרה.

הפנייה יכולה להיות בנושאים:

- מחיר.
- הזמנה.
- מעקב משלוח.
- ביטול / שינוי.
- החזרות והחזרים.
- גישה לחשבון.
- תשלום.
- תקלה טכנית.
- מידע על מוצר.
- מדיניות.
- תלונה.

שירות לקוחות אוטומטי היא מערכת המנסה לתת מענה ללקוח ללא נציג אנושי. המטרה היא לקצר זמני תגובה, לספק זמינות גבוהה, ולהפחית עומס ממוקדי השירות.

חווית משתמש מתייחסת לאופן שבו המשתמש מרגיש בזמן השימוש במערכת: האם השירות ברור, מהיר, נוח ומובן. בפרויקט זה יש חשיבות לכך שהמשתמש יקבל תשובה פשוטה וברורה במקום אחד.

התאמה אישית היא שימוש במידע קודם על המשתמש או על השיחה כדי לתת מענה מדויק יותר. לדוגמה, אם המשתמש כבר שאל על הזמנה מסוימת, המערכת יכולה להתחשב בכך בהמשך השיחה.

ניתן לראות כי תחום שירות הלקוחות בפרויקט מתבסס על שלושה צירים מרכזיים:

- **מה הלקוח שואל** - זיהוי נושא הפנייה.
- **איזה מידע מתאים לו** - איתור מידע רלוונטי.
- **איזה מידע כבר יש לנו** - התבססות על נתונים משיחות קודמות.
- **כיצד להחזיר לו תשובה** - ניסוח מענה ברור, מהיר ונוח. המערכת משתמשת בעקרונות של צ'אטים, בוטים וצ'אטבוטים, המאפשרים אוטומציה של שירות לקוחות. כך ניתן לעבד את שאלת המשתמש, למצוא מידע מתאים ולהחזיר תשובה מהירה וברורה ללא תלות מיידית בנציג אנושי.

6.2. נושאים רלוונטיים בתחום הדעת של הבעיה:

מחקרים ופרסומים בתחום שירות הלקוחות מצביעים על כך שמערכות אוטומטיות וצ'אטבוטים יכולים לקצר זמני מענה, לתת תגובה ראשונית מיידית, ולהפחית עומס מנציגי שירות. מערכות אלו שימושיות במיוחד עבור שאלות חוזרות, כגון בירור מחיר, סטטוס הזמנה, תשלומים, החזרות ותקלות נפוצות. לפי IBM, צ'אטבוטים הם אחד מכלי הבינה המלאכותית הנפוצים לקיצור זמני תגובה בשירות לקוחות. מתוך IBM.

מחקרים בתחום **Question Answering** עוסקים ביכולת של מערכת להבין שאלה ולהחזיר תשובה מדויקת מתוך מידע קיים או באמצעות יצירת תשובה חדשה. בתחום זה קיימות שתי גישות מרכזיות: גישה מחלצת, שבה המערכת בוחרת תשובה מתוך טקסט קיים, וגישה יוצרת, שבה המערכת מנסחת תשובה חדשה. בפרויקט נעשה שילוב רעיוני בין שתי הגישות: חילוץ מידע רלוונטי מתוך מקורות קיימים, ולאחר מכן ניסוח תשובה ברורה למשתמש. מתוך arVix.

נושא נוסף הרלוונטי לפרויקט הוא **דמיון סמנטי בין משפטים**. מחקרים בתחום מראים כי התאמה בין שאלה למידע אינה יכולה להתבסס רק על מילים זהות, משום שמשתמשים עשויים לנסח את אותה כוונה בדרכים שונות. מודל Sentence-BERT מציג שימוש ב-Embeddings לצורך ייצוג משפטים והשוואת משמעות ביניהם בצורה יעילה באמצעות מדדי דמיון כגון Cosine Similarity. מתוך arVix.

בתחום **זיהוי דיבור**, מחקרים מראים כי מערכות ASR עברו התפתחות משמעותית בזכות למידה עמוקה ומודלים מתקדמים. תחום זה רלוונטי לפרויקט משום שהוא מאפשר למשתמש לשאול שאלה בקול, כאשר המערכת ממירה את הדיבור לטקסט וממשיכה את תהליך הניתוח על בסיס הטקסט שהתקבל. עם זאת, זיהוי הדיבור אינו מספיק בפני עצמו, משום שלאחר ההמרה עדיין נדרשת הבנת משמעות השאלה. מתוך arVix.

לסיכום, המחקרים והפרסומים בתחום מצביעים על צורך במערכות המסוגלות לשלב בין זמינות גבוהה, הבנת שפה טבעית, חיפוש מידע רלוונטי, התאמה לשאלת המשתמש וניסוח תשובה ברורה. הפרויקט משתלב בצורך זה באמצעות מערכת צ'אט אחת המרכזת שירות עבור מגוון חברות ועסקים.

6.3. סקירה מקצועית של הבעיה בעולם:

בעולם שירות הלקוחות, מתן מענה ללקוחות מתבצע בעיקר באמצעות נציגי שירות, מוקדים טלפוניים, מיילים, טפסי פנייה וצ'אטים באתרי חברות. תהליכים אלו מאפשרים מענה אנושי ואישי, אך הם כוללים מספר מגבלות:

- זמני המתנה ארוכים.
- עומס על נציגי השירות.
- חוסר זמינות בשעות מסוימות.
- צורך לפנות לכל חברה בנפרד.
- קושי לקבל תשובה מהירה לשאלה פשוטה.

כדי להתמודד עם מגבלות אלו, חברות רבות משתמשות בכלים שונים:

- מוקדי שירות גדולים.
- מערכות ניתוב שיחות.
- עמודי שאלות ותשובות נפוצות.
- צ'אטבוטים בסיסיים באתרי חברות.
- מערכות פנייה דרך טפסים ומיילים.
- שירות עצמי דרך אזור אישי באתר.

עם זאת, פתרונות אלו עדיין אינם פותרים את הבעיה באופן מלא. לרוב, כל חברה מפעילה מערכת שירות נפרדת, ולכן המשתמש נדרש לעבור בין אתרים שונים כדי לקבל מידע. בנוסף, מערכות אוטומטיות רבות מתקשות להבין שאלות חופשיות, להסתמך על הקשר הקודם, ולנסח תשובה טבעית ומדויקת.

הבעיה המרכזית שנותרה היא היכולת לספק למשתמש מקום אחד שבו ניתן לקבל שירות עבור מגוון חברות ועסקים, תוך הבנת משמעות השאלה, איתור מידע רלוונטי, ומתן תשובה ברורה ומהירה.

6.4. נושאים רלוונטיים במדעי המחשב בהקשר לבעיה:

בתחום מדעי המחשב קיימות מספר "משפחות" של שיטות המתמודדות עם הבנת טקסט, חיפוש מידע ויצירת תשובה למשתמש.

עיבוד שפה טבעית — NLP

תחום זה עוסק בהבנת שפה אנושית על ידי מחשב. הוא כולל:

- פירוק טקסט למרכיבים.
- ניקוי ועיבוד טקסט.
- זיהוי נושא וכוונת המשתמש.
- ניתוח משמעות והקשר.

מודלים מבוססי Embedding

מודלים אלו ממירים טקסט לווקטורים מספריים, כך שמשפטים בעלי משמעות דומה יהיו קרובים זה לזה במרחב המתמטי. בפרויקט, שיטה זו מסייעת להשוות בין שאלת המשתמש לבין מקורות מידע קיימים.

מודלי דמיון סמנטי

מודלים אלו משווים בין טקסטים ומחשבים עד כמה הם דומים מבחינת משמעות, ולא רק מבחינת מילים זהות. הם מתאימים במיוחד למצבים שבהם המשתמש שואל שאלה בניסוח שונה מהמידע הקיים במאגר.

מודלי Transformer

מודלים מתקדמים המבוססים על מנגנון Self-Attention. מנגנון זה מאפשר לכל מילה "להתייחס" לשאר המילים במשפט, וכך להבין הקשר רחב יותר ולא רק משמעות מקומית של מילה בודדת.

ייצוג מובנה של טקסט

בתהליך זה שאלה חופשית מומרת למבנה נתונים מסודר, כגון JSON. המבנה כולל שדות קבועים לפי סוג השאלה, ומאפשר למערכת לעבוד עם נתונים ברורים במקום עם טקסט חופשי בלבד.

מודלי הבנת טקסט וחילוץ תשובה

מודלים כגון לדוגמא: DistilBERT מתמקדים בהבנת משמעות של טקסט. הם מתאימים למשימות כמו:

- הבנת שאלת המשתמש.
- איתור קטע רלוונטי בתוך טקסט.
- חילוץ משפט או תשובה מתוך מקור מידע.
- בדיקת התאמה בין שאלה לבין טקסט קיים.

מודלים ליצירת תשובה טבעית

מודלים כגון FLAN-T5 משמשים ליצירת תשובה מנוסחת וברורה על בסיס מידע שנמצא קודם לכן. בפרויקט, הם מסייעים להפוך מידע גולמי או משפטים שנשלפו לתשובה טבעית יותר למשתמש.

שילוב בין מודלים חישוביים וכללים לוגיים

בנוסף למודלים, ניתן להשתמש גם בכללים לוגיים כדי לשפר יציבות. לדוגמה: בחירת מבנה תשובה לפי סוג השאלה, טיפול בשאלות שאינן מתאימות לנושאים הקיימים, או החזרת הודעה מתאימה כאשר לא נמצא מידע מספק.

7. תיאור פתרונות קיימים לבעיה

בעיית מתן מענה אוטומטי בשירות לקוחות נבחנה במספר גישות אלגוריתמיות מרכזיות, הנבדלות זו מזו ברמת ההבנה של שאלת המשתמש, אופן חיפוש המידע ויכולת יצירת התשובה.

גישה 1: התאמת מילות מפתח — Keyword Matching

גישה זו מבוססת על זיהוי מילים מרכזיות בשאלת המשתמש והשוואתן למילים הקיימות במאגרי המידע. האלגוריתם מפרק את השאלה למילים, מסיר מילים לא חשובות, ומחפש התאמה לפי מילים דומות או זהות. שיטה זו פשוטה ומהירה, אך היא מוגבלת משום שאינה מבינה משמעות. לדוגמה, שתי שאלות בעלות אותה כוונה אך בניסוח שונה עלולות שלא להיחשב דומות.

גישה 2: סיווג שאלות לפי נושאים — Classification

בגישה זו מאמנים מודל לזהות את סוג הפנייה של המשתמש, כגון מחיר, הזמנה, מעקב משלוח, תשלום, החזרות או תקלה טכנית. האלגוריתם מקבל שאלה כקלט ומחזיר תווית המתארת את הנושא המתאים ביותר. גישה זו מסייעת להפנות את השאלה למסלול טיפול מתאים, אך אינה מספיקה לבדה כדי להחזיר תשובה מלאה.

גישה 3: חילוץ מידע מובנה — Information Extraction

בגישה זו המערכת מזהה מתוך טקסט חופשי פרטים חשובים, כגון מוצר, חברה, כמות, זמן או סוג פעולה. הנתונים נשמרים במבנה מסודר, למשל JSON, ומשמשים בהמשך לחיפוש מידע או לבניית תשובה.

גישה 4: דמיון סמנטי מבוטא — Embedding

בגישה זו ממירים את שאלת המשתמש ואת קטעי המידע לווקטורים מספריים. לאחר מכן מחשבים את מידת הדמיון ביניהם, בדרך כלל באמצעות Cosine Similarity.

גישה זו מאפשרת לזהות קירבה רעיונית גם כאשר אין התאמה מילולית מדויקת, ולכן היא מתאימה לחיפוש מידע רלוונטי מתוך מאגרי ידע. עם זאת, היא אינה תמיד יודעת להסביר מדוע קטע מסוים נבחר.

גישה 5: מערכות Question Answering לחילוף תשובה

בגישה זו המערכת מקבלת שאלה וטקסט מקור, ומנסה לחלץ מתוך הטקסט את המשפט או הקטע שעונה על השאלה. מודל DistilBERT מתאים לגישה זו, משום שהוא מסוגל להבין הקשר בתוך טקסט ולבחור את מיקום התשובה.

היתרון הוא קבלת תשובה ממוקדת מתוך מקור מידע קיים. החיסרון הוא שהמודל תלוי באיכות הטקסט שנבחר, ואם המידע אינו מופיע בטקסט — לא תתקבל תשובה טובה.

גישה 6: יצירת תשובה טבעית — Generative Models

בגישה זו המערכת אינה מסתפקת בשליפת משפט קיים, אלא מנסחת תשובה חדשה וברורה על בסיס המידע שנמצא. מודלים כגון T5 או FLAN-T5 מסוגלים לקבל הוראה, שאלה ומידע רלוונטי, ולהפיק תשובה מנוסחת בשפה טבעית.

גישה זו משפרת את חוויית המשתמש, אך דורשת בקרה כדי לוודא שהתשובה נאמנה למידע המקורי ואינה מוסיפה מידע לא מבוסס.

גישה 7: מערכות מבוססות כללים — Rule-Based Systems

מערכות אלו משתמשות בחוקים מוגדרים מראש. לדוגמה: אם השאלה עוסקת במחיר, יש להחזיר תשובה במבנה מסוים; אם לא נמצא מידע מתאים, יש להחזיר הודעה מתאימה; ואם השאלה אינה שייכת לנושאי שירות הלקוחות, יש לסווג אותה כ-*other*.

הגישה מאפשרת שליטה ויציבות, אך היא פחות גמישה כאשר המשתמש מנסח שאלה מורכבת או לא צפויה.

גישה 8: שימוש בהיסטוריית שיחה — Context-Aware Systems

בגישה זו המערכת אינה מנתחת כל שאלה בנפרד בלבד, אלא מתחשבת גם בשאלות ותשובות קודמות של המשתמש. כך ניתן להבין המשך שיחה, אזכורים כמו "זה", "ההזמנה הזאת" או "כמה זה עולה?", ולספק מענה מדויק יותר.

גישה זו חשובה לשירות לקוחות טבעי, אך דורשת שמירה מסודרת של היסטוריית השיחה והבנה מתי מידע קודם עדיין רלוונטי.

8. ניתוח חלופות מערכת

הפתרונות שנבחנו מציגים יתרונות וחסרונות שונים.

התאמת מילות מפתח היא שיטה פשוטה ומהירה, אך אינה מבינה משמעות ולכן מתקשה עם ניסוחים חופשיים.

סיווג שאלות לפי נושאים מאפשר להפנות כל שאלה למסלול מתאים, אך תלוי באיכות הדאטה ועלול לטעות בסיווג.

Embedding ודמיון סמנטי מאפשרים למצוא מידע לפי משמעות ולא רק לפי מילים זהות, אך לא תמיד מספקים הסבר לבחירה.

מודלי Question Answering כמו DistilBERT מאפשרים לחלץ תשובה מדויקת מתוך טקסט, אך תלויים באיכות מקור המידע שנבחר.

מודלים ליצירת תשובה כמו FLAN-T5 משפרים את ניסוח התשובה והופכים אותה לטבעית, אך דורשים בקרה כדי שלא ייווצר מידע לא מדויק.

מערכות מבוססות כללים נותנות יציבות ושליטה, אך אינן גמישות מספיק כשפה חופשית.

לכן נבחר שילוב בין סיווג שאלה, חילוף נתונים למבנה JSON, חיפוש סמנטי, חילוף תשובה ויצירת תשובה טבעית לשיפור הדיוק והיציבות.

9. תיאור החלופה הנבחרת ונימוקים

הפתרון שנבחר בפרויקט משלב בין מספר גישות אלגוריתמיות, במטרה ליצור מערכת צ'אט חכמה, מדויקת ויציבה, המסוגלת להבין שאלות משתמשים, לחפש מידע רלוונטי ולהחזיר תשובה טבעית וברורה.

עקרונות הפתרון:

- שימוש במודל סיווג לזיהוי נושא השאלה.
- שימוש ב-Embedding למדידת דמיון סמנטי בין השאלה לבין מקורות המידע.
- שימוש ב-DistilBERT לחילוף תשובה מתוך טקסט רלוונטי.
- שימוש ב-FLAN-T5 לבניית תשובה סופית טבעית.
- שימוש בכללים לוגיים לשיפור יציבות המערכת וטיפול במקרי קצה.
- שימוש בהיסטוריית שיחה לצורך הבנת הקשר ומתן מענה מותאם יותר.

האלגוריתם בפועל:

קלט: שאלה מהמשתמש, בטקסט או בקול.

- שלב 1: במקרה של קלט קולי — המרת הדיבור לטקסט.
 - שלב 2: ניקוי ועיבוד ראשוני של הטקסט.
 - שלב 3: סיווג השאלה לאחד מנושאי שירות הלקוחות.
 - שלב 4: יצירת מבנה JSON מתאים לסוג השאלה.
 - שלב 5: חילוף הנתונים מהשאלה ומילוי שדות ה-JSON.
 - שלב 6: בניית שאילתת חיפוש קצרה ומדויקת מתוך ה-JSON.
 - שלב 7: הוספת נתונים מהשיחות הקודמות.
 - שלב 8: חיפוש מידע מתאים במאגרי הידע.
 - שלב 9: חישוב דמיון סמנטי באמצעות Embedding.
 - שלב 10: בחירת הקטעים הרלוונטיים ביותר.
 - שלב 11: חילוף תשובה ממוקדת באמצעות DistilBERT.
 - שלב 12: ניסוח תשובה טבעית וברורה באמצעות FLAN-T5.
 - שלב 13: שמירת השיחה, במבנה נתונים.
 - שלב 14: החזרת תשובה למשתמש בטקסט או בקול.
- בסיום:
- שלב 15: יצירת דו"ח שיחה ושמירה במסד הנתונים.

נימוקים לבחירה:

הפתרון נבחר משום שהוא משלב בין הבנת שפה חופשית לבין יציבות מערכתית. הסיווג מאפשר לנתב את השאלה לנושא המתאים, ה-Embedding מאפשר למצוא מידע לפי משמעות, DistilBERT מסייע בחילוף תשובה מדויקת, ו-FLAN-T5 משפר את ניסוח התשובה למשתמש.

בנוסף, שילוב כללים לוגיים מאפשר להתמודד עם מקרים שבהם לא נמצא מידע מתאים או כאשר השאלה אינה שייכת לנושאי שירות הלקוחות. לכן הפתרון מתאים למערכת שירות לקוחות מרכזית, גמישה ונוחה לשימוש.

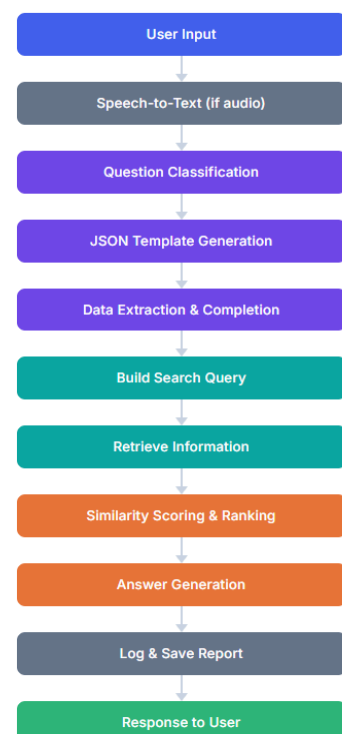
10. אפיון מערכת

המערכת **SmartService Chat** בנויה כמערכת צ'אט חכמה לשירות לקוחות, המשלבת בין ניהול שיחה עם המשתמש לבין ניתוח תוכן מתקדם של שאלות, חיפוש מידע רלוונטי ויצירת תשובה טבעית וברורה.

רכיבים עיקריים:

- ממשק משתמש ללקוח.
- ממשק מנהל מערכת.
- שרת עיבוד מרכזי.
- מודל המרת אודיו לטקסט.
- מודל ניקוי ועיבוד טקסט.
- מודל סיווג נושא השאלה.
- מודול יצירת JSON לפי סוג השאלה.
- מודול חילוץ נתונים מתוך השאלה.
- מודול בניית שאלת חיפוש מתוך ה-JSON.
- מודל Embedding וחישוב דמיון סמנטי.
- מודל חיפוש מידע במאגרי ידע ובמקורות חיצוניים.
- מודל חילוץ תשובה באמצעות DistilBERT.
- מודל יצירת תשובה טבעית באמצעות FLAN-T5.
- מנגנון הסתמכות על היסטוריית שיחה.
- מסד נתונים לשמירת משתמשים, חברות, שיחות, שאלות, תשובות ודוחות.

CUSTOMER SERVICE CHATBOT — DATA FLOW



10.1. ניתוח דרישות המערכת:

המערכת נדרשת:

- לנתח שאלות בשפה חופשית ולהבין את כוונת המשתמש.
- לזהות את נושא הפנייה, כגון מחיר, הזמנה, תשלום או תקלה.
- להתמודד עם ניסוחים שונים של אותה שאלה.
- לחפש מידע רלוונטי ולהחזיר תשובה מדויקת.
- להגיב בזמן סביר ולספק חוויית שימוש נוחה.
- לשמור היסטוריית שיחות ודוחות במסד הנתונים.
- לעבוד עם משתמשים מרובים ועם מגוון חברות ועסקים.

10.2. מודול המערכת:

המערכת בנויה במודל לקוח-שרת:

צד לקוח: ממשק משתמש לשליחת שאלות וקבלת תשובות.
צד שרת: עיבוד השאלה, ניתוח הטקסט, חיפוש מידע ובניית תשובה.
מסד נתונים: אחסון משתמשים, חברות, שיחות, שאלות, תשובות ודוחות.

בנוסף קיימים מודלים פנימיים:

- **מודל Embedding** - ייצוג טקסט וחישוב דמיון סמנטי.
- **מודל סיווג סוג שאלה** — סיווג השאלה לאחד מתוך 12 סוגים.
- **מודל JSON** — יצירת מבנה נתונים מתאים לפי סוג השאלה.
- **מודל חילוץ נתונים** — מילוי שדות ה-JSON מתוך שאלת המשתמש.
- **מודל בניית שאלתה** — יצירת שאלת חיפוש קצרה ומדויקת מתוך ה-JSON.
- **מודל חילוץ תשובה** - איתור תשובה מתאימה מתוך טקסט רלוונטי.
- **מודל יצירת תשובה** - ניסוח תשובה טבעית וברורה.
- **מודל חוקים** - טיפול במקרי קצה ושיפור יציבות המערכת.

10.3. אפיון פונקציונלי:

קלט:

- מסדי נתונים מחברות היוצרות מנוי.
- הרשמה — פרטי מנוי.
- שאלת משתמש.

עיבוד:

- ניתוח טקסט והבנת משמעות השאלה.
- סיווג השאלה לאחד מתוך 12 סוגי שאלות.
- יצירת מבנה JSON מתאים לסוג השאלה.
- חילוץ נתונים מהשאלה ומילוי שדות ה-JSON.

- בניית שאילתת חיפוש קצרה ומדויקת מתוך ה-JSON.
- הוספת נתונים קודמים מתוך היסטוריית השיחה.
- חיפוש מידע במסדי הנתונים של החברות המנויות, ולאחר מכן במידע כללי.
- חילוץ תשובה ממוקדת באמצעות DistilBERT.
- מתן ציון התאמה ובחירת הפסקאות המתאימות ביותר.
- בניית תשובה והחזרתה ללקוח.
- שמירת דו"ח סיכום.

פלט:

- תשובה למשתמש.

10.4. ביצועים עיקריים:

המערכת שפותחה מסוגלת לבצע מענה אוטומטי לשאלות משתמשים בתחום שירות הלקוחות, תוך הבנה סמנטית של השאלה, חיפוש מידע רלוונטי ובניית תשובה מתאימה.

פירוט:

- לזהות את נושא שאלת המשתמש, כגון מחיר, הזמנה, תשלום, החזרה או תקלה.
- להבין את משמעות השאלה ולא רק לזהות מילים זהות.
- לבצע ניתוח טקסט חופשי ולהתמודד עם מגוון ניסוחים שונים.
- לסווג שאלות משתמשים לאחד מתוך 12 סוגי פניות.
- להפיק מבנה JSON מתאים לכל סוג שאלה.
- לחלץ נתונים חשובים מתוך השאלה בלי לאבד מידע מרכזי.
- לבנות שאילתת חיפוש קצרה ומדויקת מתוך הנתונים שחולצו.
- לשלב מידע קודם מתוך היסטוריית השיחה לצורך הבנת ההקשר.
- לחפש מידע במסדי הנתונים של חברות מנויות, ולאחר מכן במידע כללי.
- לחשב ציון התאמה בין השאלה לבין מקורות המידע.
- לבחור את הפסקאות או המשפטים הרלוונטיים ביותר.
- לחלץ תשובה מתוך טקסט מתאים באמצעות DistilBERT.
- לבנות תשובה טבעית וברורה עבור המשתמש.
- לשמור דו"ח שיחה הכולל שאלות, תשובות, נושאי שיחה וניתוח בסיסי.

בנוסף, המערכת משלבת בין מודלי שפה, Embedding וכללים לוגיים, כדי לשפר את דיוק התשובה ולהפחית בחירה של מידע לא רלוונטי.

10.5. אילוצים:

אילוצים:

- המערכת תלויה באיכות השאלה שהמשתמש מזין.
- המערכת תלויה בכך שסוג השאלה יתאים לאחד מתוך 12 הסוגים שהוגדרו.
- שאלות שאינן מכילות מספיק פרטים עלולות לגרום ל-JSON חלקי.
- איכות החיפוש תלויה בדיוק הנתונים שחולצו מתוך השאלה.

- שאלות קצרות מדי או לא ברורות עלולות לפגוע בדיוק התשובה.
- המערכת תלויה באיכות מסדי הנתונים של החברות המנויות.
- קיימת תלות באיכות הדאטה ששימש לאימון מודלי הסיווג.
- מודלים כמו Embedding, DistilBERT ו-FLAN-T5 דורשים זמן עיבוד ומשאבי מחשוב.
- המערכת אינה מבטיחה תשובה נכונה בכל ניסוח חריג או מורכב.

הנחות יסוד:

- המשתמש שואל שאלה הקשורה לשירות לקוחות.
- קיימים מקורות מידע רלוונטיים לחיפוש.
- הטקסט שהתקבל מייצג את כוונת המשתמש.
- היסטוריית השיחה נשמרת בצורה תקינה.

גבולות הפרויקט:

- המערכת אינה מחליפה נציג שירות אנושי בכל מצב.
- המערכת אינה מטפלת בנושאים שאינם קשורים לשירות לקוחות.
- המערכת מתבססת על המידע שנמצא במאגרי החברות ובמקורות החיפוש, ולכן אינה יכולה להבטיח אימות מלא של כל פרט חיצוני.

11. תיאור הארכיטקטורה

המערכת **SmartService Chat** בנויה בארכיטקטורת לקוח-שרת, המפרידה בין ממשק המשתמש לבין תהליכי העיבוד והניתוח המתבצעים בצד השרת.

בצד הלקוח נמצא ממשק המשתמש, דרכו המשתמש נרשם למערכת, שולח שאלות ומקבל תשובות.

בצד השרת מתבצעים שלבי העיבוד המרכזיים: סיווג השאלה לאחד מתוך 12 סוגי שאלות, יצירת מבנה JSON מתאים, חילוץ הנתונים החשובים מתוך השאלה, בניית שאילתת חיפוש ממוקדת, חיפוש מידע, חישוב התאמה, חילוץ תשובה ובניית מענה סופי.

בנוסף, המערכת כוללת מסד נתונים המשמש לשמירת משתמשים, חברות מנויות, שאלות, תשובות, היסטוריית שיחות ודוחות סיכום. מבנה זה מאפשר הפרדה ברורה בין רכיבי המערכת, ניהול יעיל של המידע, והרחבה עתידית של המערכת לחברות ומשתמשים נוספים.

11.1. הארכיטקטורה של הפתרון המוצע:

המערכת בנויה בארכיטקטורת **Top-Down Layered Design**, כלומר חלוקה לשכבות, כאשר כל שכבה אחראית על חלק מסוים בתהליך.

שכבת הלקוח

שכבה זו כוללת את ממשק המשתמש. דרכה המשתמש נרשם, שולח שאלה בטקסט או בקול, ומקבל תשובה מהמערכת.

שכבת התקשורת

שכבה זו אחראית להעברת הנתונים בין הלקוח לשרת. היא מקבלת את שאלת המשתמש ומעבירה אותה לעיבוד בצד השרת.

שכבת העיבוד

בשכבה זו מתבצע עיבוד השאלה: ניקוי הטקסט, סיווג השאלה לאחד מתוך 12 סוגים, יצירת מבנה JSON מתאים, חילוף הנתונים החשובים מתוך השאלה ובניית שאילתת חיפוש קצרה ומדויקת.

שכבת הבינה המלאכותית

שכבה זו אחראית על פעולות NLP מתקדמות: חישוב Embedding, בדיקת דמיון סמנטי, בחירת פסקאות רלוונטיות, חילוף תשובה באמצעות DistilBERT ובניית תשובה טבעית באמצעות FLAN-T5.

שכבת הנתונים

שכבה זו כוללת את מסד הנתונים של המערכת. נשמרים בה פרטי משתמשים, חברות מנויות, שאלות, תשובות, היסטוריית שיחות ודוחות סיכום.

תקשורת בין הרכיבים:

המשתמש שולח שאלה דרך ממשק הלקוח. השאלה מועברת לשרת, שם היא עוברת בין שכבות העיבוד וה-NLP. לאחר יצירת התשובה, התוצאה נשמרת במסד הנתונים ומוחזרת למשתמש דרך ממשק הלקוח.

זרימה כללית:

לקוח -> שרת -> עיבוד שאלה -> JSON -> חיפוש מידע -> DistilBert - חילוף משפט תשובה -> embedding - אחוז התאמת משפט תשובה -> FLAN-T5 - שמירה במסד נתונים -> תשובה ללקוח.

11.2. תיאור הרכיבים בפתרון:**לקוח / ממשק משתמש**

מאפשר למשתמש להירשם למערכת, לשלוח שאלה בטקסט או בקול, ולקבל תשובה מהמערכת.

שרת תקשורת

אחראי על קבלת הבקשות מהלקוח, העברתן לשרת היישום, והחזרת התשובה למשתמש לאחר סיום העיבוד.

מודול למידת מכונה

אחראי על פעולות הבינה המלאכותית במערכת, כגון סיווג שאלות, יצירת Embedding, חישוב דמיון סמנטי, חילוף תשובה באמצעות DistilBERT ויצירת תשובה טבעית באמצעות FLAN-T5.

שרת בסיס נתונים — DB

אחראי על שמירת פרטי משתמשים, חברות מנויות, שאלות, תשובות, היסטוריית שיחות, דוחות סיכום ונתוני ניתוח.

מקורות מידע חיצוניים / מאגרי חברות

כוללים את מסדי הנתונים של החברות המנויות ומקורות מידע כלליים, שמהם נשלף המידע שעליו מתבססת התשובה.

מודל קול

אחראי על המרת שאלה קולית לטקסט, ובמקרה הצורך גם על החזרת תשובה קולית למשתמש.

11.3. תיאור תהליכים של מערכת ההפעלה:

בפרויקט נעשה שימוש במספר תהליכים הקשורים לניהול מערכת, עבודה עם משתמשים ואבטחת מידע. במערכת קיימים גם תהליכי קריאה וכתיבה למסד הנתונים, כגון שמירת משתמשים, שמירת שאלות ותשובות, ועדכון דו"ח שיחה. פעולות אלו דורשות סדר עבודה תקין כדי למנוע כפילויות או שמירה שגויה של מידע.

11.4. ארכיטקטורת רשת:

למערכת אין ארכיטקטורת רשת מורכבת מעבר למודל לקוח-שרת. התקשורת המרכזית מתבצעת בין ממשק הלקוח לבין שרת היישום באמצעות HTTP/HTTPS.

11.5. תיאור API בארכיטקטורה:

במערכת קיימים מספר ממשקי API המשמשים לתקשורת בין רכיבי המערכת ולחיבור עם מקורות חיצוניים. המערכת משתמשת ב-API פנימי בין צד הלקוח לשרת, דרכו נשלחות שאלות המשתמש ומוחזרות התשובות לאחר העיבוד. בנוסף, קיימת התממשקות עם מקורות מידע חיצוניים לצורך חיפוש מידע כללי, במקרה שבו לא נמצאה תשובה מתאימה במסדי הנתונים של החברות המנויות. כמו כן, קיימת אפשרות לשימוש ב-API להמרת דיבור לטקסט עבור שאלות קוליות, וב-API או מודל מתאים להמרת תשובה טקסטואלית לקול. ממשקי ה-API מאפשרים למערכת לקבל קלט, לעבד אותו, לשלוח מידע ממקורות שונים, ולהחזיר תשובה מתאימה למשתמש בצורה מסודרת.

11.6. תיאור פרוטוקולי תקשורת:

המערכת משתמשת בפרוטוקול HTTP/HTTPS לצורך תקשורת בין צד הלקוח לבין שרת היישום.

הבקשות המרכזיות במערכת מתבצעות באמצעות:

- **POST** - לשליחת נתונים לשרת, כגון הרשמה, התחברות, שליחת שאלה ושמירת דו"ח שיחה.
- **GET** - לקבלת מידע מהשרת, כגון שליפת תשובה, טעינת היסטוריית שיחות או קבלת נתוני משתמש.
- **HTTPS** - לשמירה על תקשורת מאובטחת בין הלקוח לשרת.

בנוסף, השרת מתקשר עם מסד הנתונים לצורך שמירה ושליפה של משתמשים, חברות, שאלות, תשובות ודוחות.

11.7. שרת-לקוח:

המערכת פועלת במודל שרת-לקוח. צד הלקוח הוא ממשק המשתמש, דרכו המשתמש נרשם, מתחבר, שולח שאלה ומקבל תשובה. צד השרת מקבל את הבקשות, מעבד את השאלה, מפעיל את מודלי למידת המכונה, מחפש מידע מתאים, בונה תשובה ושומר נתונים במסד הנתונים.

התקשורת בין הלקוח לשרת מתבצעת באמצעות HTTP/HTTPS. פעולות כמו הרשמה, התחברות ושליחת שאלה נשלחות לשרת, והשרת מחזיר תשובה מסודרת ללקוח. הלקוח אינו ניגש ישירות למסד הנתונים, אלא כל הפעולות מתבצעות דרך השרת, כדי לשמור על סדר, בקרה ואבטחה.

במערכת לא נעשה שימוש ב-Sockets או Pipes, משום שאין צורך בתקשורת רציפה. התקשורת מתבצעת במבנה רגיל של בקשה ותגובה: הלקוח שולח בקשה, השרת מעבד אותה ומחזיר תשובה.

12. תיאור תהליכי אבטחת מידע במערכת

המערכת כוללת מספר תהליכי אבטחת מידע שמטרתם להגן על פרטי המשתמשים, השיחות והמידע הנשמר במסד הנתונים.

בעת הרשמה והתחברות, פרטי המשתמש נשמרים בצורה מאובטחת. הסיסמאות אינן נשמרות כטקסט רגיל, אלא לאחר פעולת **Hash**, כך שלא ניתן לראות את הסיסמה המקורית ישירות מתוך מסד הנתונים.

בנוסף, התקשורת בין הלקוח לשרת מתבצעת באמצעות **HTTPS**, כדי להגן על המידע בזמן שליחתו ברשת.

המערכת שומרת במסד הנתונים רק מידע הדרוש לפעולתה, כגון פרטי משתמש, היסטוריית שיחות, שאלות, תשובות ודוחות. הגישה למידע זה מתבצעת דרך השרת בלבד, ולא ישירות מצד הלקוח.

כמו כן, קיימת הפרדה בין משתמשים רגילים, חברות מנויות ומנהל מערכת, כך שכל סוג משתמש מקבל הרשאות בהתאם לתפקידו במערכת.

12.1. תיאור התקיפה או ההגנה:

במערכת קיימים מנגנוני התחברות והרשמה המבוססים על **שם משתמש וסיסמה**. כדי להגן על פרטי המשתמש, הסיסמה אינה נשמרת במסד הנתונים כטקסט רגיל, אלא נשמרת לאחר פעולת **Hash**.

אמצעי ההגנה במערכת:

- שמירת סיסמאות באמצעות Hash.
- שימוש בשם משתמש וסיסמה לצורך זיהוי המשתמש.
- הפרדה בין משתמש רגיל, חברה מנויה ומנהל מערכת.
- הגבלת גישה למידע לפי סוג המשתמש.
- שמירת נתוני השיחות והדוחות במסד הנתונים ולא בצד הלקוח.

מקרי תקיפה שנלקחו בחשבון:

- ניסיון כניסה עם פרטי משתמש שגויים.
- ניסיון גישה למידע של משתמש אחר.

- ניסיון צפייה בסיסמאות מתוך מסד הנתונים.
- ניסיון לבצע פעולות של מנהל מערכת ללא הרשאה.

12.2. תיאור הצפנות:

במערכת קיימת הגנה על סיסמאות המשתמשים באמצעות פעולת **Hash**. כלומר, הסיסמה המקורית אינה נשמרת כטקסט רגיל, אלא נשמר ערך מוצפן/מעובד שמייצג אותה.

בנוסף, התקשורת בין הלקוח לשרת מתבצעת באמצעות **HTTPS**, המאפשר הצפנה של המידע בזמן מעברו ברשת.

לכן, ההצפנות/ההגנות הקיימות במערכת הן:

- **Hash לסיסמאות משתמשים.**
- **HTTPS לתקשורת מאובטחת בין הלקוח לשרת.**

13. למידת מכונה

מודל **SetFit** משמש לסיווג שאלת המשתמש לאחד מתוך סוגי השאלות שהוגדרו במערכת. תפקיד המודל הוא לקבל שאלה חופשית של לקוח, להבין את הנושא המרכזי שלה, ולהחזיר את סוג הפנייה המתאים ביותר.

לדוגמה, המודל מסווג שאלות לאחד מהנושאים הבאים:

- מחיר
- הזמנה
- מעקב
- תשלום
- ביטול או שינוי
- החזרה
- גישה לחשבון
- תקלה
- מידע על מוצר
- מדיניות
- תלונה
- זמן/תאריך

לאחר הסיווג, המערכת יודעת איזה מבנה תשובה או תהליך חיפוש מתאים לשאלה. שלב זה חשוב משום שהוא מאפשר למערכת לעבוד בצורה מסודרת ולא להתייחס לכל שאלה באותה צורה.

מודל **FLAN-T5-LARGE** משמש למילוי מבנה JSON מתאים לפי סוג השאלה שזוהה. לאחר ש-**SETFIT** מסווג את השאלה לנושא מסוים, המערכת בוחרת מבנה JSON ייעודי לאותו סוג שאלה.

תפקיד FLAN-T5-LARGE הוא לחלץ מתוך שאלת המשתמש את הנתונים החשובים ולמלא אותם בשדות המתאימים במבנה ה-JSON.

לדוגמה, בשאלת מחיר המודל עשוי לחלץ:

- סוג מוצר
- שם מוצר
- חברה
- כמות
- זמן
- מקום

לאחר מילוי ה-JSON, המערכת משתמשת בנתונים שנשלפו כדי לבנות שאלת חיפוש קצרה ומדויקת יותר. כך ניתן לשמור על המידע החשוב מתוך השאלה, לצמצם רעש, ולשפר את איכות החיפוש והתשובה הסופית.

13.1.1. תהליך איסוף הנתונים SetFit:

עבור אימון מודל **SetFit** נאספו שאלות המדמות פניות של לקוחות בתחום שירות הלקוחות. הנתונים כללו שאלות בניסוח חופשי, כפי שמשתמשים עשויים לשאול במערכת אמיתית.

הדאטה כלל שאלות מסוגים שונים, כגון:

- שאלות על מחירים.
- שאלות על מוצרים.
- שאלות על הזמנות.
- שאלות על תשלומים.
- שאלות על החזרות.
- שאלות על תקלות.
- שאלות כלליות נוספות בתחום שירות הלקוחות.

כל שאלה תויגה לפי סוג הפנייה המתאים לה. התיוגים שימשו את המודל כדי ללמוד לזהות לאיזה נושא שייכת כל שאלה.

בסיום התהליך התקבל דאטהסט מסווג, שבו כל רשומה כוללת שאלה ותווית נושא מתאימה.

13.1.2. תהליך איסוף הנתונים FLAN-T5-LARGE:

עבור אימון מודל **FLAN-T5-LARGE** נאספו שאלות לקוח יחד עם מבנה JSON מתאים לכל סוג שאלה. הנתונים כללו שאלות בניסוח חופשי, כאשר לכל שאלה הוגדר פלט רצוי במבנה JSON.

בתהליך זה נאספו נתונים כגון:

- טקסט השאלה המקורית.
- סוג השאלה שאליו היא שייכת.
- מבנה JSON המתאים לסוג השאלה.
- השדות שיש למלא מתוך השאלה.
- הערכים שחולצו מתוך השאלה.

לדוגמה, בשאלת מחיר נאספו שדות כמו סוג מוצר, שם מוצר, חברה, כמות, זמן ומיקום, בהתאם למידע שהופיע בשאלה.

כל דוגמה תויגה באמצעות JSON תקין, כך שהמודל למד לקבל שאלה וסוג פנייה, ולהחזיר מבנה נתונים מלא ככל האפשר.

בסיום התהליך התקבל דאטהסט שבו כל רשומה כוללת קלט טקסטואלי, JSON מתאים, ונושא פניה ופלט JSON מלא בנתוני השאלה.

13.2.1. טיוב ואיזון הנתונים SetFit:

עבור מודל SetFit בוצע טיוב של הנתונים כדי לשפר את יכולת הסיווג של שאלות המשתמשים.

התהליך כלל:

- בדיקה שכל שאלה מתאימה לתווית הנושא שלה.
- תיקון ניסוחים לא ברורים או לא תקינים.
- יצירת שאלות מגוונות עבור כל נושא, כדי שהמודל לא ילמד רק ניסוח אחד קבוע.
- איזון בין כמות הדוגמאות בכל סוג שאלה, כדי למנוע מצב שבו המודל מעדיף נושאים שיש להם יותר דוגמאות.
- שילוב שאלות קצרות, שאלות ארוכות ושאלות בניסוח חופשי.
- הסרת דוגמאות כפולות או דומות מדי.

מטרת הטיוב הייתה לגרום למודל לזהות את נושא השאלה לפי משמעותה, ולא רק לפי מילים שחוזרות בדאטהסט.

13.2.2. טיוב ואיזון הנתונים FLAN-T5-LARGE:

עבור מודל FLAN-T5-LARGE בוצע טיוב של הנתונים כדי לשפר את יכולת חילוף המידע ומילוי מבנה ה-JSON.

התהליך כלל:

- בדיקה שכל פלט JSON תקין מבחינה תחבירית.
- התאמת מבנה JSON קבוע לכל סוג שאלה.
- בדיקה שהשדות ב-JSON תואמים למידע שמופיע בשאלה.
- מילוי שדות חסרים בערך ריק כאשר המידע אינו מופיע בשאלה.
- יצירת דוגמאות מגוונות לכל סוג שאלה, כדי שהמודל ילמד לחלץ נתונים מניסוחים שונים.
- איזון בין סוגי השאלות, כדי שכל מבנה JSON יקבל מספיק דוגמאות.
- הסרת דוגמאות שבהן הפלט אינו מתאים לשאלה או כולל מידע שלא הופיע בה.

מטרת הטיוב הייתה לגרום למודל להחזיר JSON מדויק, מסודר ועקבי, בלי לאבד נתונים חשובים מתוך שאלת המשתמש.

13.3.1. סוג הלמידה הנבחר SetFit:

סוג הלמידה שנבחר הוא **למידה מונחית - Supervised Learning**. המודל אומן על שאלות מתויגות, כאשר לכל שאלה קיימת תווית נושא מתאימה.

בנוסף, SetFit מבוסס על גישת **Few-Shot Learning**, כלומר יכולת ללמוד גם מכמות יחסית קטנה של דוגמאות מתויגות.

13.3.2. סוג הלמידה הנבחר FLAN-T5-LARGE:

סוג הלמידה שנבחר הוא **למידה מונחית - Supervised Learning**. המודל אומן על זוגות של קלט ופלט: שאלת משתמש וסוג שאלה כקלט, ומבנה JSON מלא כפלט.

המשימה היא מסוג **Sequence-to-Sequence**, כלומר המודל מקבל טקסט כקלט ומייצר טקסט מובנה בפורמט JSON כפלט.

13.4.1. נוסחאות ורשתות למידה SetFit:

מודל **SetFit** משמש לסיווג שאלת המשתמש לאחד מתוך סוגי השאלות שהוגדרו במערכת.

התהליך מתחיל בקבלת שאלה חופשית מהמשתמש:

$$x = \text{שאלת המשתמש}$$

בשלב הראשון השאלה מומרת לייצוג מספרי באמצעות מודל Sentence Transformer. כלומר, הטקסט הופך לוקטור מספרי המייצג את משמעות המשפט:

$$v = \text{Encoder}(x)$$

כאשר:

- x - שאלת המשתמש.
- Encoder - מודל ההמרה לטקסט מספרי.
- v - וקטור Embedding של השאלה.

לאחר מכן המערכת משתמשת במסווג שלומד להפריד בין סוגי השאלות. המסווג מקבל את הווקטור ומחזיר הסתברות לכל נושא אפשרי:

$$P(y | x) = \text{Classifier}(v)$$

כאשר:

- y - סוג השאלה.
- $P(y | x)$ - ההסתברות שהשאלה שייכת לנושא מסוים.

לבסוף נבחר הנושא בעל ההסתברות הגבוהה ביותר:

$$\hat{y} = \operatorname{argmax} P(y | x)$$

כלומר, המערכת בוחרת את סוג השאלה שהמודל מעריך כמתאים ביותר.

במהלך האימון, המודל מקבל שאלות מתויגות ולומד לקרב שאלות מאותו נושא במרחב הווקטורי, ולהרחיק שאלות מנושאים שונים. כך המודל לומד לזהות את משמעות השאלה ולא רק מילים זהות.

13.4.2. נוסחאות ורשתות למידה FLAN-T5-LARGE:

מודל **FLAN-T5-LARGE** משמש להמרת שאלת המשתמש למבנה JSON מתאים לפי סוג השאלה שזוהה.

הקלט למודל כולל את שאלת המשתמש ואת סוג השאלה:

$$x = \text{שאלה} + \text{סוג שאלה} + \text{JSON מותאם לסוג השאלה}$$

המטרה היא להפיק JSON מלא בנתוני השאלה:

$$y = \text{JSON}$$

המודל פועל בשיטת **Sequence-to-Sequence**, כלומר הוא מקבל רצף טקסט כקלט ומייצר רצף טקסט חדש כפלט.

באופן מתמטי, המודל מחשב בכל שלב את ההסתברות למילה או לטוקן הבא בפלט:

$$p(y_t | y_1, y_2, \dots, y_{\{t-1\}}, x)$$

כאשר:

- x - הקלט למודל.
- y_t - הטוקן הבא שהמודל מייצר.
- $y_1 \dots y_{\{t-1\}}$ - הטוקנים שכבר נוצרו קודם.
- P - ההסתברות לבחירת הטוקן הבא.

הפלט הסופי הוא רצף הטוקנים בעל ההסתברות הגבוהה ביותר:

$$\hat{y} = \operatorname{argmax} P(y | x)$$

במקרה של הפרויקט, הפלט אינו תשובה חופשית רגילה, אלא JSON בעל שדות קבועים בהתאם לסוג השאלה. לדוגמה, בשאלת מחיר המודל ממלא שדות כמו מוצר, חברה, כמות וזמן.

במהלך האימון, המודל לומד לקשר בין ניסוח חופשי של שאלה לבין מבנה JSON תקין. כלומר, הוא לומד אילו פרטים חשובים יש לחלץ מהשאלה, באיזה שדה למקם אותם, ואילו שדות להשאיר ריקים כאשר המידע אינו מופיע בשאלה.

כך מתקבל ייצוג מובנה של השאלה, שממנו ניתן לבנות שאילתת חיפוש קצרה, מדויקת ושומרת על המידע החשוב.

13.5.1. כיוונון, ייעול וניטור מדדי ביצוע SetFit:

על מנת לשפר את מודל SetFit, בוצעו מספר פעולות כיוונון וייעול:

- חלוקת הנתונים לסט אימון וסט בדיקה.
- איזון מספר הדוגמאות בין סוגי השאלות.
- בדיקה ידנית של תיוגים שגויים ותיקונם.
- הוספת שאלות בניסוחים מגוונים לכל נושא.
- שינוי פרמטרים באימון, כגון מספר Epochs וגודל Batch.
- בדיקת תוצאות המודל על שאלות חדשות שלא הופיעו באימון.

תקינות המודל נבדקה באמצעות השוואה בין הסיווג שהמודל החזיר לבין הסיווג הנכון שהוגדר מראש. מדדי הביצוע שנבדקו:

- **Accuracy** - אחוז השאלות שסווגו נכון.
- **Precision** - עד כמה הסיווגים שהמודל בחר היו נכונים.
- **Recall** - עד כמה המודל הצליח לזהות את כל הדוגמאות מכל נושא.
- **Confusion Matrix** - בדיקה באילו נושאים המודל מתבלבל.

אחוז הדיוק הסופי של המודל: 94.87%

13.5.2. כיוונון, ייעול וניטור מדדי ביצוע FLAN-T5-LARGE:

על מנת לשפר את מודל FLAN-T5-LARGE, בוצעו תהליכי כיוונון שהתמקדו בדיוק מילויי שדות ה-JSON לפי שאלת המשתמש.

המודל מקבל כקלט את שאלת המשתמש, סוג השאלה שזוהה, ומבנה JSON מתאים לאותו סוג שאלה. תפקידו אינו לבחור את מבנה ה-JSON, אלא למלא את השדות הקיימים בו לפי המידע שמופיע בשאלה.

התהליך כלל:

- בדיקה שכל דוגמת אימון כוללת שאלה, סוג שאלה ו-JSON מתאים.
- שמירה על מבנה JSON קבוע לכל סוג שאלה.
- תיקון דוגמאות שבהן השדות מולאו בצורה לא מדויקת.
- הוספת דוגמאות עם ניסוחים שונים לאותה משמעות.
- בדיקה שהמודל ממלא רק מידע שהופיע בשאלה.
- בדיקה ששדות שאין עבורם מידע בשאלה נשארים ריקים.
- שינוי פרמטרים באימון, כגון מספר Epochs, אורך קלט ואורך פלט.

תקינות המודל נבדקה באמצעות השוואה בין ה-JSON שמולא על ידי המודל לבין ה-JSON המצופה.

מדדי הביצוע שנבדקו:

- **JSON Validity** - האם הפלט נשאר JSON תקין.
- **Field Accuracy** - האם השדות מולאו בערכים הנכונים.

- **Missing Field Rate** - האם המודל החסיר מידע שהופיע בשאלה.
- **Extra Information Rate** - האם המודל הוסיף מידע שלא הופיע בשאלה.
- **Exact Match** - האם ה-JSON המלא זהה ל-JSON המצופה.

14. תיאור התוכנה

התוכנה **SmartService Chat** היא מערכת צ'אט חכמה למתן שירות לקוחות עבור מגוון חברות ועסקים במקום אחד.

המערכת מאפשרת למשתמש להירשם, להתחבר, לשלוח שאלה בטקסט או בקול, ולקבל תשובה מתאימה. לאחר קבלת השאלה, המערכת מסווגת אותה לסוג שאלה, ממלאת מבנה JSON מתאים, מחפשת מידע רלוונטי, בוחרת את הטקסטים המתאימים, מחלצת תשובה ובונה מענה ברור למשתמש.

בנוסף, התוכנה שומרת היסטוריית שיחות, שאלות, תשובות ודוחות סיכום במסד הנתונים, ומשתמשת במודלים של למידת מכונה לצורך סיווג, חילוף מידע ובניית תשובה טבעית.

14.1. פירוט API:

במערכת נעשה שימוש במספר ממשקי API לצורך חיבור בין רכיבי המערכת ולצורך קבלת מידע חיצוני.

ה-API המרכזי הוא הממשק בין צד הלקוח לשרת. באמצעותו המשתמש שולח שאלות, מבצע הרשמה והתחברות, ומקבל תשובות מהמערכת.

בנוסף, המערכת מתממשקת עם שירות חיפוש חיצוני לצורך חיפוש מידע כללי, במקרה שבו לא נמצאה תשובה מתאימה במסדי הנתונים של החברות המנויות.

קיים גם שימוש בממשק להמרת קול לטקסט, כדי לאפשר למשתמש לשלוח שאלה קולית. לאחר ההמרה, השאלה ממשיכה לעבור עיבוד כמו שאלה טקסטואלית רגילה.

במידת הצורך, המערכת משתמשת גם בממשק להמרת טקסט לקול, כדי להחזיר למשתמש תשובה קולית.

לסיכום, ממשקי ה-API במערכת משמשים לשליחת שאלות, קבלת תשובות, חיפוש מידע חיצוני, המרת דיבור לטקסט והמרת תשובה טקסטואלית לקול.

14.2. סביבת עבודה:

הפרויקט פותח בסביבת עבודה המחולקת לצד לקוח, צד שרת ומסד נתונים.

צד לקוח פותח בשפת React, והוא אחראי על ממשק המשתמש, הרשמה והתחברות, שליחת שאלות וקבלת תשובות מהמערכת.

צד שרת פותח בשפת Python ו-C#, והוא אחראי על עיבוד השאלה, סיווג סוג הפנייה, מילוי JSON, חיפוש מידע, הפעלת מודלי למידת המכונה ובניית התשובה.

מודלי למידת המכונה נשמרים בתיקיית מודלים מקומית, ונטענים בצד השרת בזמן הרצת המערכת.

מסד הנתונים משמש לשמירת משתמשים, חברות מנויות, שאלות, תשובות, היסטוריית שיחות ודוחות סיכום.

בנוסף, נעשה שימוש בסביבת פיתוח מקומית, הכוללת עורך קוד, סביבת הרצה וירטואלית, ספריות NLP ולמידת מכונה, וקבצי נתונים לאימון ובדיקת המודלים.

14.3. שפות תכנות:

לקוח / ממשק משתמש
פותח בשפת **React**.

שרת תקשורת
פותח בשפת **Python / #C**.

מודל למידת מכונה
פותח בשפת **Python**.

שרת בסיס נתונים — DB
מבוסס על **SQL**.

מקורות מידע חיצוניים / מאגרי חברות
הגישה אליהם מתבצעת דרך **#C**, באמצעות שאילתות למסדי נתונים או קריאות למקורות מידע חיצוניים.

מודל קול
פותח בשפת **Python**.

15. ניתוח ותרשים Use cases / UML של המערכת המוצעת

במערכת קיימים מספר שחקנים עיקריים:

- **משתמש** - נרשם, מתחבר, שולח שאלה ומקבל תשובה.
- **חברה מנויה** - מספקת מידע ומסדי נתונים למערכת.
- **מנהל מערכת** - מנהל משתמשים, חברות, הרשאות ומידע במערכת.
- **שרת המערכת** - מעבד את השאלה, מפעיל מודלים ומחזיר תשובה.

תרשים ה-Use Case מציג את הקשרים בין המשתמשים לבין הפעולות המרכזיות במערכת, ומדגיש את תהליך קבלת השאלה, עיבודה והחזרת התשובה.

15.1. תיאור הCU העיקריים של המערכת:

הרשמת משתמש חדש

המשתמש מזין פרטים אישיים, שם משתמש וסיסמה. המערכת בודקת תקינות נתונים, שומרת את המשתמש במסד הנתונים, ומאפשרת לו להתחבר למערכת.

התחברות למערכת

המשתמש מזין שם משתמש וסיסמה. המערכת בודקת התאמה מול הנתונים השמורים, ואם הפרטים תקינים מאפשרת כניסה.

שליחת שאלה למערכת

המשתמש שולח שאלה בטקסט או בקול. אם הקלט קולי, המערכת ממירה אותו לטקסט.

האלגוריתם הראשי למענה על שאלה

המערכת מקבלת את השאלה, מסווגת אותה לאחד מתוך 12 סוגי שאלות, בוחרת מבנה JSON מתאים, ממלאת אותו לפי נתוני השאלה, ובונה ממנו שאלתת חיפוש קצרה ומדויקת. לאחר מכן המערכת מחפשת מידע במאגרי החברות ובמקורות כלליים, בוחרת טקסטים רלוונטיים, מחלצת תשובה, מנסחת מענה סופי על פי נושא הפניה ומחזירה אותו למשתמש.

שמירת דו"ח שיחה

בסיום התהליך נשמרים במסד הנתונים פרטי השיחה, השאלות, התשובות, נושאי הפנייה ונתוני הניתוח.

ניהול חברות מנויות

מנהל המערכת יכול להוסיף חברות מנויות ולעדכן את מקורות המידע שלהן, כדי שהמערכת תוכל לתת להן עדיפות בתהליך החיפוש.

15.2. הצגת מקרה (use case) עבור כל הפונקציות העיקריות בפרויקט:**1. הרשמת משתמש**

תנאים מוקדמים: המשתמש אינו רשום במערכת.

תהליך תקין: המשתמש מזין פרטים, המערכת בודקת תקינות, שומרת את המשתמש במסד הנתונים ומאפשרת התחברות.

במקרה שגיאה: אם חסרים פרטים או שהמשתמש כבר קיים, מוצגת הודעת שגיאה.

2. התחברות משתמש

תנאים מוקדמים: המשתמש רשום במערכת.

תהליך תקין: המשתמש מזין שם משתמש וסיסמה, המערכת בודקת התאמה ומכניסה אותו למערכת.

במקרה שגיאה: אם הפרטים שגויים, מוצגת הודעה מתאימה.

3. שליחת שאלה

תנאים מוקדמים: המשתמש מחובר למערכת.

תהליך תקין: המשתמש שולח שאלה בטקסט או בקול. אם הקלט קולי, הוא מומר לטקסט.

במקרה שגיאה: אם השאלה ריקה או לא תקינה, המערכת מבקשת להזין שאלה מחדש.

4. עיבוד שאלה ובניית JSON

תנאים מוקדמים: התקבלה שאלה תקינה.
תהליך תקין: המערכת מסווגת את השאלה לאחד מתוך 12 סוגים, בוחרת JSON מתאים וממלאת אותו לפי הנתונים שבשאלה.
במקרה שגיאה: אם חסרים נתונים, השדות החסרים נשארים ריקים או שהמערכת מחזירה הודעה מתאימה.

5. חיפוש מידע ובחירת תשובה

תנאים מוקדמים: קיים JSON מלא או חלקי לחיפוש.
תהליך תקין: המערכת בונה שאילתת חיפוש, מחפשת במאגרי החברות ולאחר מכן במידע כללי, ובוחרת את הטקסטים המתאימים ביותר.
במקרה שגיאה: אם לא נמצא מידע מתאים, המערכת מחזירה הודעה שלא נמצאה תשובה מספקת.

6. יצירת תשובה למשתמש

תנאים מוקדמים: נמצא מידע רלוונטי.
תהליך תקין: המערכת מחלצת תשובה, מנסחת אותה בצורה טבעית וברורה, ומחזירה אותה למשתמש.
במקרה שגיאה: אם התשובה אינה מספיק מדויקת, המערכת מחזירה תשובה כללית או הודעה מתאימה.

7. שמירת דו"ח שיחה

תנאים מוקדמים: הסתיימה אינטראקציה עם המשתמש.
תהליך תקין: המערכת שומרת במסד הנתונים את השאלה, התשובה, נושא הפנייה ונתוני הסיכום.
במקרה שגיאה: אם השמירה נכשלה, המערכת ממשיכה לפעול אך מציגה/שומרת הודעת שגיאה פנימית.

8. ניהול חברות מנויות

תנאים מוקדמים: מנהל מערכת מחובר.
תהליך תקין: המנהל מוסיף או מעדכן חברה מנויה ואת מקורות המידע שלה.
במקרה שגיאה: אם חסרים פרטי חברה או שיש כפילות, המערכת מציגה הודעת שגיאה.

15.3 מבני נתונים בהם השתמשתי:

במהלך הפרויקט נעשה שימוש במספר מבני נתונים, בהתאם לצורך של כל שלב במערכת.

List — רשימה

נעשה שימוש ברשימות לשמירת שאלות, תשובות אפשריות, פסקאות שנשלפו ותוצאות ביניים. מבנה זה נבחר משום שהוא פשוט ונוח למעבר בלולאה.

Dictionary / Hash Map — מילון

נעשה שימוש במילונים לשמירת מידע בצורה של מפתח וערך, לדוגמה שדות ה־JSON והערכים שחולצו מהשאלה. מבנה זה מאפשר גישה מהירה לנתונים לפי שם השדה.

JSON

נעשה שימוש ב־JSON כדי לייצג את שאלת המשתמש בצורה מסודרת לפי סוג הפנייה. לכל סוג שאלה קיים מבנה מתאים, והמערכת ממלאת בו את הפרטים שנמצאו בשאלה.

Embedding Vectors — וקטורים מספריים

השאלה והתשובות האפשריות מומרות לווקטורים מספריים, כדי לחשב דמיון סמנטי ביניהן. מבנה זה מאפשר לבדוק התאמה לפי משמעות ולא רק לפי מילים זהות.

Binary Search Tree — עץ חיפוש בינארי

נעשה שימוש בעץ חיפוש בינארי לצורך שמירת תשובות לפי ציון ההתאמה שלהן. כך ניתן למיין את התשובות ולבחור את הקטעים המתאימים ביותר. וכן שמירת ה"דברים שיש לו" וכן נושאי השיחה בעץ חיפוש שכל צומת מכיל מבנה.

Hash לסיסמאות

בנוסף, נעשה שימוש בפעולת Hash לשמירת סיסמאות בצורה בטוחה יותר. הסיסמה המקורית אינה נשמרת במסד הנתונים, אלא רק ערך Hash חד-כיווני.

לסיכום, השימוש במבני הנתונים השונים מאפשר למערכת לשמור מידע, לעבד שאלות, לדרג תשובות ולנהל את היסטוריית השיחות בצורה מסודרת ויעילה.

15.4. חישוב יעילות האלגוריתם:

בפרויקט נבחנו סיבוכיות הזמן והמקום של השלבים המרכזיים במערכת.

נסמן:

n - אורך שאלת המשתמש.

m - מספר הטקסטים או הפסקאות שנמצאו.

t - אורך ממוצע של טקסט.

a - מספר התשובות האפשריות שחולצו.

d - גודל וקטור ה-Embedding.

סיווג השאלה

המערכת מסווגת את שאלת המשתמש לאחד מסוגי הפניות. זמן הפעולה תלוי באורך השאלה:

$O(n)$

מילוי JSON

המערכת עוברת על השאלה ומחלצת ממנה נתונים למבנה JSON מתאים:

$O(n)$

סיבוכיות המקום היא:

$O(1)$

משום שמבנה ה-JSON קבוע יחסית לכל סוג פנייה.

חיפוש מידע

אם קיימים **m** טקסטים לבדיקה, זמן החיפוש הוא:

$O(n)$

וסיבוכיות המקום היא:

 $O(m)$

חילוץ תשובות באמצעות DistilBERT

DistilBERT עובר על הטקסטים שנמצאו ומחלץ תשובות אפשריות:

 $O(m*t)$

כאשר t הוא אורך ממוצע של טקסט.

דירוג באמצעות Embedding

לאחר חילוץ התשובות, המערכת מחשבת דמיון סמנטי בין השאלה לבין כל תשובה אפשרית:

 $O(a*d)$

אם גודל הווקטור קבוע, ניתן להתייחס לכך כ-:

 $O(a)$

שמירה בעץ חיפוש בינארי

הכנסת התשובות לעץ לפי ציון ההתאמה דורשת במוצע:

 $O(a \log a)$

לאחר מכן נבחרים 10 הקטעים הטובים ביותר, וזה נחשב פעולה קבועה:

 $O(1)$

יצירת תשובה באמצעות FLAN-T5-LARGE

המודל מקבל את 10 הקטעים שנבחרו, את שאלת המשתמש ואת סוג הפנייה, ומנסח תשובה סופית. מכיוון שמספר הקטעים קבוע, זמן הפעולה תלוי בעיקר באורך הקלט למודל.

סיכום סיבוכיות הזמן

השלבים המרכזיים הם חילוץ התשובות, דירוגן ושמירתן לפי ציון. לכן סיבוכיות הזמן הכוללת היא בקירוב:

 $O(m*t + a \log a)$

סיכום סיבוכיות המקום

המערכת שומרת טקסטים, תשובות אפשריות, ציוני התאמה ווקטורים מספריים. לכן סיבוכיות המקום היא בקירוב:

 $O(m+a)$

15.5. הקשרים בין היחידות השונות:

המערכת בנויה כרצף של יחידות, כאשר כל יחידה מקבלת מידע מהיחידה הקודמת ומעבירה תוצאה ליחידה הבאה.

תחילה המשתמש שולח שאלה דרך ממשק הלקוח. אם השאלה קולית, מודול הקול ממיר אותה לטקסט. לאחר מכן השאלה נשלחת לשרת, שם מתבצע עיבוד ראשוני וניקוי של הטקסט.

בשלב הבא מודל הסיווג מזהה את סוג הפנייה, ולפי הסוג שנבחר המערכת יוצרת מבנה JSON מתאים. מודול חילוף הנתונים ממלא את שדות ה-JSON לפי הפרטים שהופיעו בשאלה.

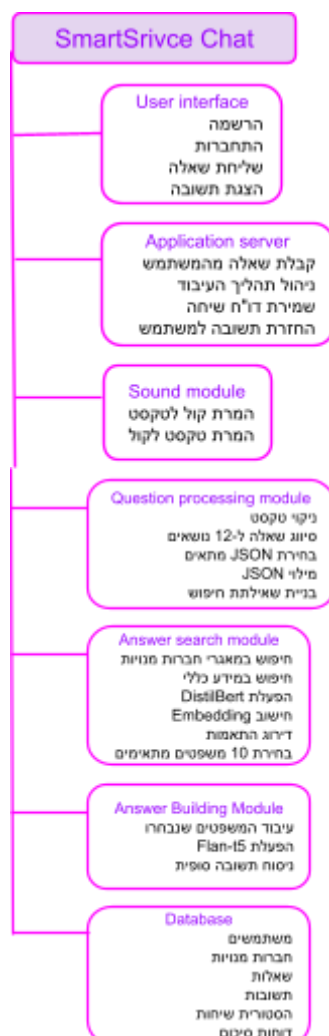
לאחר מכן נבנית שאילתת חיפוש קצרה ומדויקת מתוך ה-JSON. השאילתה נשלחת למודול החיפוש, שמחפש מידע במאגרי החברות המנויות ובמקורות מידע כלליים.

הטקסטים שנמצאו מועברים אל DistilBERT, שמחלץ מהם תשובות אפשריות. לאחר מכן התשובות עוברות למודול ה-Embedding, שמחשב את רמת ההתאמה ביניהן לבין שאלת המשתמש.

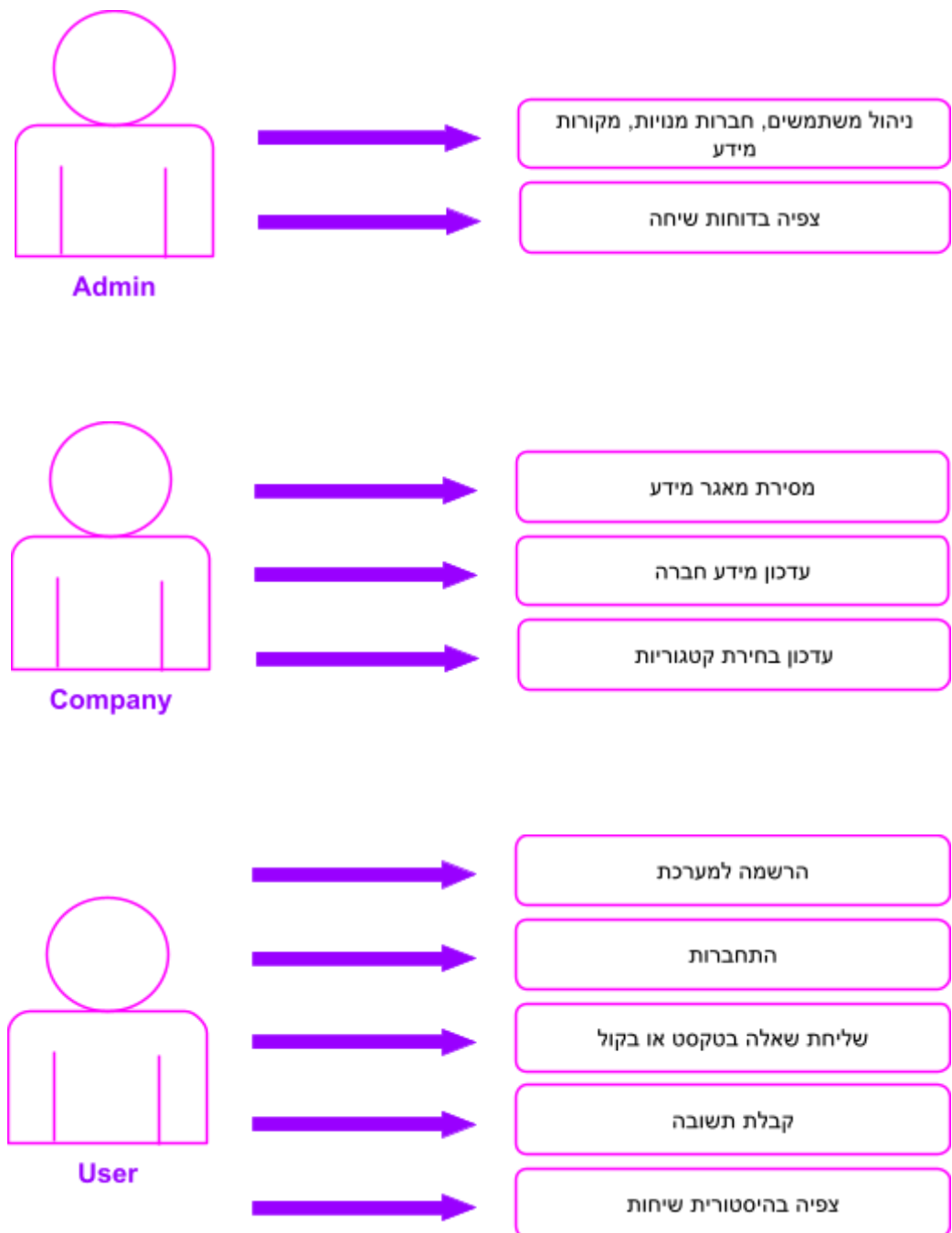
התשובות מדורגות לפי ציון ההתאמה, ומתוכן נבחרים 10 הקטעים המתאימים ביותר. קטעים אלו מועברים אל FLAN-T5-LARGE, שמנסח מהם תשובה סופית בהתאם לסוג הפנייה.

בסיום, התשובה מוחזרת למשתמש דרך ממשק הלקוח, ובמקביל נשמרים במסד הנתונים פרטי השיחה, השאלה, התשובה, סוג הפנייה והנתונים הדרושים לדו"ח השיחה.

15.6. עץ המודלים:



15.7 Use Case Diagram



15.8 רשימת Use Cases

משתמש:

1. הרשמה למערכת.
2. התחברות למערכת.
3. שליחת שאלה בטקסט.
4. שליחת שאלה בקול.

5. קבלת תשובה מהמערכת.
6. צפייה בהיסטוריית שיחות.

חברה מנויה:

1. מסירת מאגר מידע למערכת.
2. עדכון מידע החברה.
3. עדכון בחירת קטגוריות.

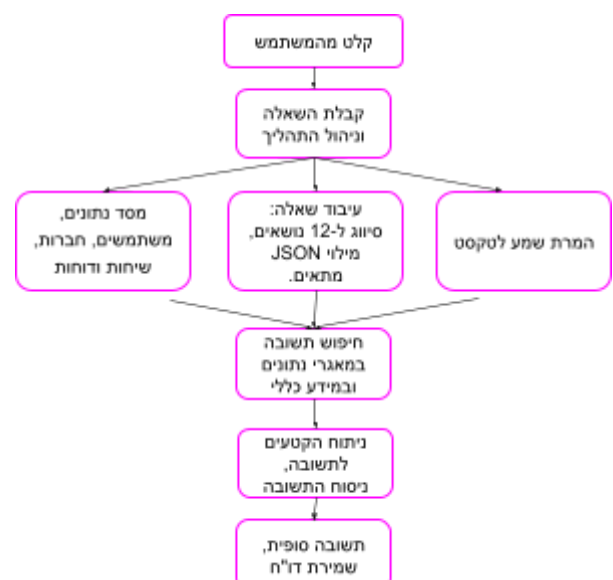
מנהל מערכת:

1. ניהול משתמשים.
2. ניהול חברות מנויות.
3. ניהול סוגי מנויים.
4. ניהול מקורות מידע.
5. צפייה בדוחות שיחה.

מערכת / תהליכים פנימיים:

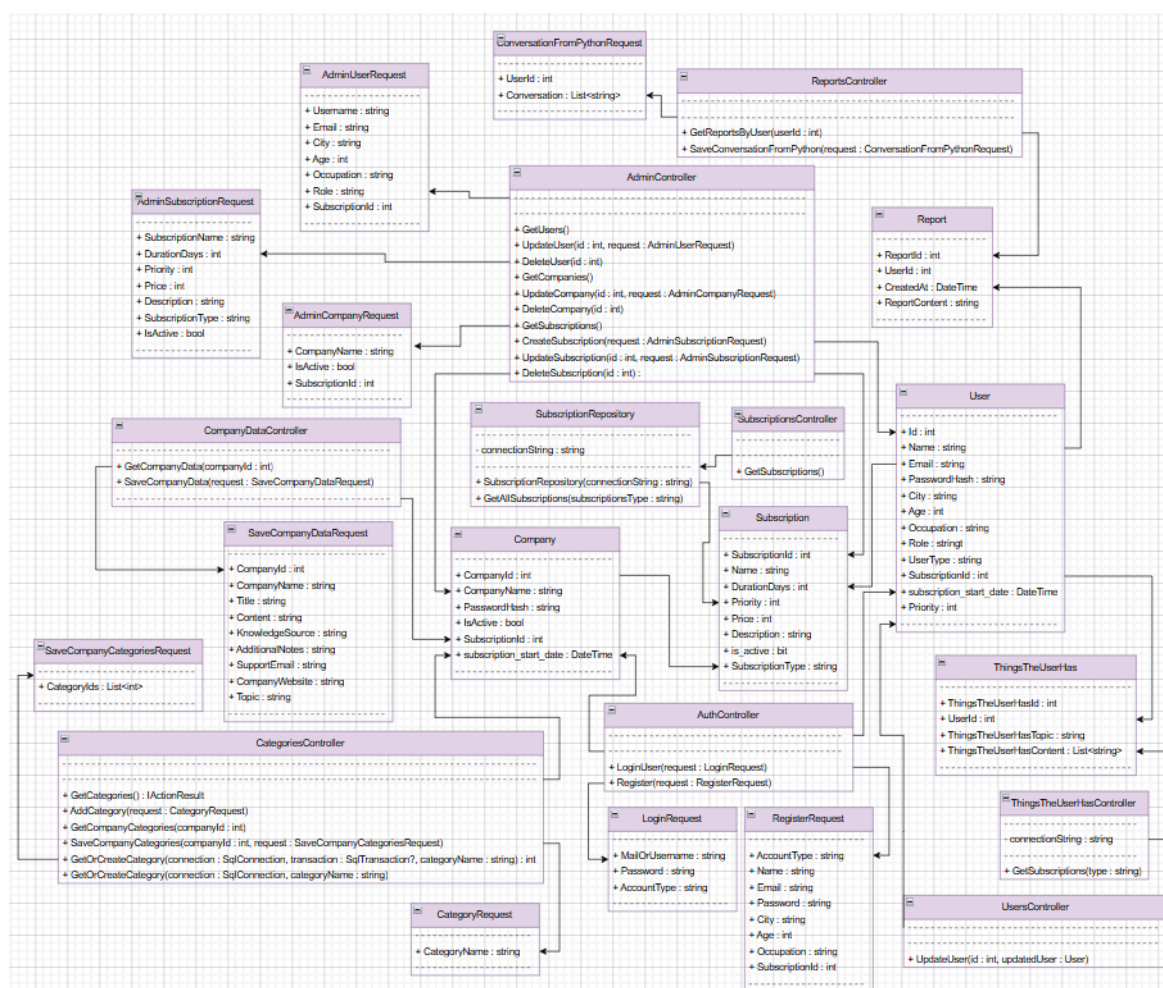
1. המרת קול לטקסט.
2. סיווג השאלה לאחד מתוך 12 סוגים.
3. מילוי JSON לפי סוג השאלה.
4. בניית שאלתת חיפוש.
5. חיפוש מידע במאגרי חברות ובמידע כללי.
6. חילוץ תשובה.
7. דירוג התאמה.
8. בחירת 10 הקטעים המתאימים ביותר.
9. בניית תשובה סופית.
10. שמירת דו"ח שיחה.

15.9. תרשים UML:



התרשים מציג את רכיבי המערכת המרכזיים ואת הקשרים ביניהם. המשתמש מזין שאלה בטקסט או בקול, והבקשה מועברת לשרת היישום. השרת מפעיל את מודלי העיבוד: המרת קול לטקסט במידת הצורך, סיווג השאלה, מילוי JSON, חיפוש מידע, הפעלת מודל למידת מכונה ובניית תשובה. בסיום התהליך התשובה מוחזרת למשתמש, ובמקביל נשמר דו"ח השיחה במסד הנתונים.

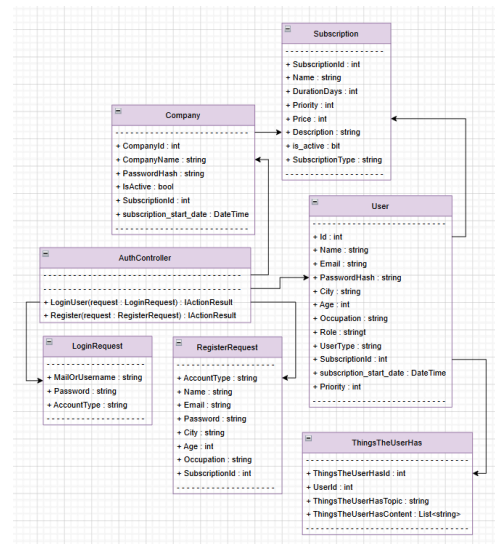
15.10. Design class diagram



15.11. תרשים מחלקות:

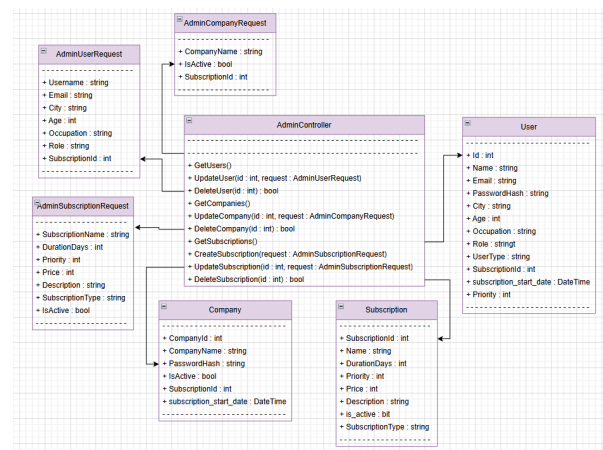
חיבורי התחברות והרשמה

ה-AuthController מקבל בקשות התחברות והרשמה, ובודק או יוצר משתמשים וחברות.



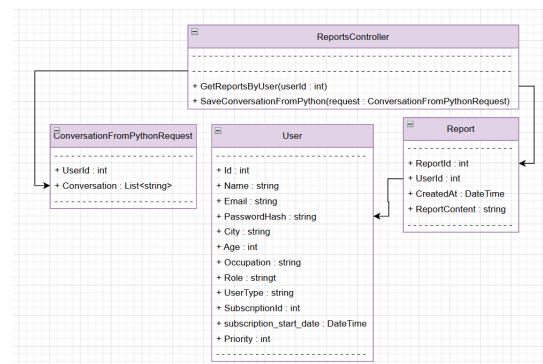
חיבורי ניהולי מערכת

ה-AdminController מנהל משתמשים, חברות וסוגי מנויים במערכת.



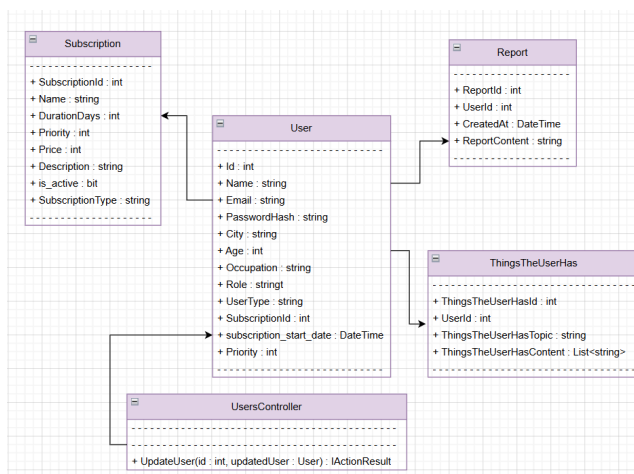
חיבורי דוחות שיחה

ה-ReportsController שומר ושולף דוחות שיחה. כל דו"ח שייך למשתמש אחד, ולמשתמש יכולים להיות כמה דוחות.



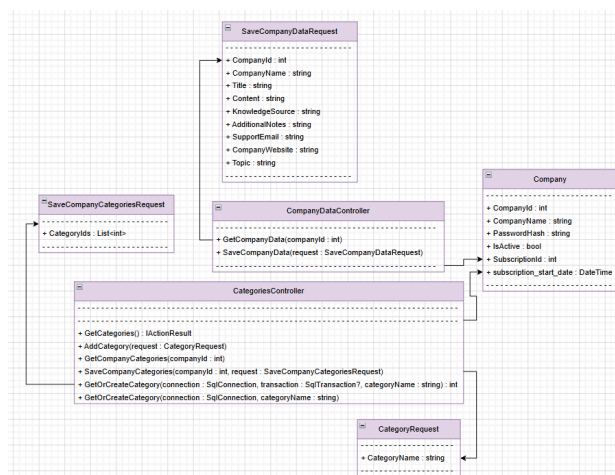
חיבורי משתמשים

ה-UsersController מטפל בפרטי המשתמש. המשתמש קשור לסוג מנוי ויכול להכיל מידע נוסף שנשמר עליו.



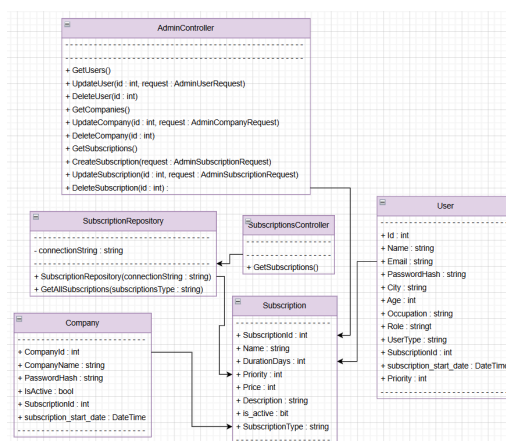
חיבורי חברות

ה-CompanyDataController מטפל במידע של חברה מנויה. כל חברה קשורה לסוג מנוי.



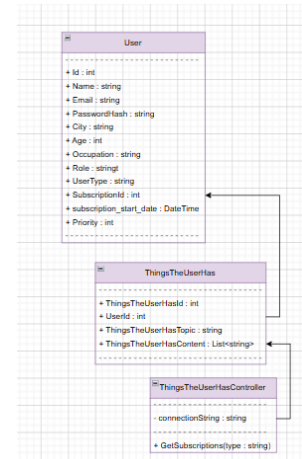
חיבורי מנויים

ה-SubscriptionsController שולף מנויים, וה-AdminController מאפשר למנהל להוסיף ולעדכן מנויים.



חיבורי מידע נוסף על המשתמש

ה-ThingsTheUserController מנהל מידע נוסף שנשמר על המשתמש לצורך התאמה טובה יותר של התשובות.



15.12. תיאור המחלקות המוצעות:

המערכת בנויה ממספר מחלקות מרכזיות, כאשר כל מחלקה אחראית על חלק אחר בתהליך: התחברות והרשמה, ניהול משתמשים, ניהול חברות, ניהול מנויים, שמירת שיחות, שמירת מידע נוסף על המשתמש ושליפת נתונים ממסד הנתונים.

User

תפקידה לייצג משתמש במערכת.

הקלט שלה הוא פרטי משתמש כמו שם, מייל, סיסמה, עיר, גיל, עיסוק, תפקיד וסוג מנוי. הפלט שלה הוא פרטי משתמש שנשמרים או נשלפים מהמסד.

Company

תפקידה לייצג חברה מנויה במערכת.

הקלט שלה הוא שם חברה, סיסמה, סטטוס פעילות וסוג מנוי. הפלט שלה הוא פרטי חברה שנשמרים או נשלפים מהמסד.

Subscription

תפקידה לייצג מנוי במערכת.

הקלט שלה הוא שם מנוי, מחיר, משך מנוי, עדיפות, תיאור וסוג מנוי. הפלט שלה הוא פרטי מנוי עבור משתמש או חברה.

Report

תפקידה לייצג דו"ח שיחה.

הקלט שלה הוא מזהה משתמש, תוכן הדו"ח ותאריך יצירה. הפלט שלה הוא דו"ח שיחה שנשמר או נשלף ממסד הנתונים.

ThingsTheUserHas

תפקידה לשמור מידע נוסף שנאסף על המשתמש.
הקלט שלה הוא מזהה משתמש, נושא ותוכן מידע.
הפלט שלה הוא מידע קודם על המשתמש שניתן להשתמש בו בהמשך השיחה.

AuthController

תפקידו לטפל בהתחברות ובהרשמה למערכת.
הקלט שלו הוא בקשת התחברות או בקשת הרשמה.
הפלט שלו הוא משתמש מחובר, חברה מחוברת, או הודעת שגיאה.

LoginRequest

תפקידה להעביר נתוני התחברות מהלקוח אל השרת.
הקלט שלה הוא שם משתמש או מייל, סיסמה וסוג חשבון.
הפלט שלה הוא בקשת התחברות שנשלחת ל-AuthController.

RegisterRequest

תפקידה להעביר נתוני הרשמה מהלקוח אל השרת.
הקלט שלה הוא פרטי משתמש או חברה חדשים.
הפלט שלה הוא בקשת הרשמה שנשלחת ל-AuthController.

AdminController

תפקידו לנהל את נתוני המערכת מצד המנהל.
הקלט שלו הוא בקשות לניהול משתמשים, חברות ומנויים.
הפלט שלו הוא נתונים מעודכנים, נתונים שנשלפו מהמסד, או הודעת הצלחה/שגיאה.

AdminUserRequest

תפקידה להעביר נתוני משתמש לעדכון על ידי מנהל.
הקלט שלה הוא שם משתמש, מייל, עיר, גיל, עיסוק, תפקיד ומנוי.
הפלט שלה הוא בקשה לעדכון משתמש.

AdminCompanyRequest

תפקידה להעביר נתוני חברה לעדכון על ידי מנהל.
הקלט שלה הוא שם חברה, סטטוס פעילות ומנוי.
הפלט שלה הוא בקשה לעדכון חברה.

AdminSubscriptionRequest

תפקידה להעביר נתוני מנוי לניהול על ידי מנהל.
הקלט שלה הוא שם מנוי, מחיר, משך מנוי, עדיפות, תיאור, סוג מנוי וסטטוס פעילות.
הפלט שלה הוא בקשה להוספה או עדכון מנוי.

ReportsController

תפקידו לטפל בדוחות שיחה.
הקלט שלו הוא מזהה משתמש או בקשה לשמירת שיחה.
הפלט שלו הוא דוחות שיחה שנשלפו או דו"ח חדש שנשמר במסד.

ConversationFromPythonRequest

תפקידה להעביר שיחה משרת Python אל שרת #C לצורך שמירה.
הקלט שלה הוא מזהה משתמש ורשימת הודעות השיחה.
הפלט שלה הוא בקשה ליצירת דו"ח שיחה.

UsersController

תפקידו לטפל בעדכון פרטי משתמש.
הקלט שלו הוא מזהה משתמש ופרטים מעודכנים.
הפלט שלו הוא משתמש מעודכן או הודעת שגיאה.

CompanyDataController

תפקידו לטפל בבסיס הידע של החברה.
הקלט שלו הוא מזהה חברה ונתוני מידע של החברה.
הפלט שלו הוא מידע חברה שנשמר או נשלף ממסד הנתונים.

SaveCompanyDataRequest

תפקידה להעביר נתוני מידע של חברה מהלקוח אל השרת.
הקלט שלה הוא מזהה חברה, שם חברה, כותרת, תוכן, מקור ידע, הערות, אתר, מייל תמיכה ונושא.
הפלט שלה הוא בקשה לשמירת בסיס ידע של חברה.

CategoriesController

תפקידו לטפל בקטגוריות של חברות.
הקלט שלו הוא שם קטגוריה או רשימת קטגוריות של חברה.
הפלט שלו הוא קטגוריות שנשמרות, נשלפות או מקושרות לחברה.

CategoryRequest

תפקידה להעביר שם קטגוריה להוספה.
הקלט שלה הוא שם קטגוריה.
הפלט שלה הוא בקשה ליצירת קטגוריה חדשה.

SaveCompanyCategoriesRequest

תפקידה להעביר את הקטגוריות שנבחרו עבור חברה.
הקלט שלה הוא רשימת מזהי קטגוריות.
הפלט שלה הוא בקשה לשיוך קטגוריות לחברה.

SubscriptionsController

תפקידו לשלוף מנויים לפי סוג מנוי.
הקלט שלו הוא סוג מנוי, למשל משתמש או חברה.
הפלט שלו הוא רשימת מנויים מתאימים.

SubscriptionRepository

תפקידו לבצע את שליפת המנויים ממסד הנתונים.
הקלט שלו הוא סוג מנוי ומחרוזת חיבור למסד הנתונים.
הפלט שלו הוא רשימת אובייקטים מסוג Subscription.

ThingsTheUserHasController

תפקידו לשלוף מידע נוסף שנשמר על משתמש.
הקלט שלו הוא מזהה משתמש.

הפלט שלו הוא מידע נוסף על המשתמש, ולעיתים גם שליחה של המידע לשרת Python.

זרימת מידע כללית

זרימת המידע מתחילה מהמשתמש או מהמנהל בצד הלקוח.

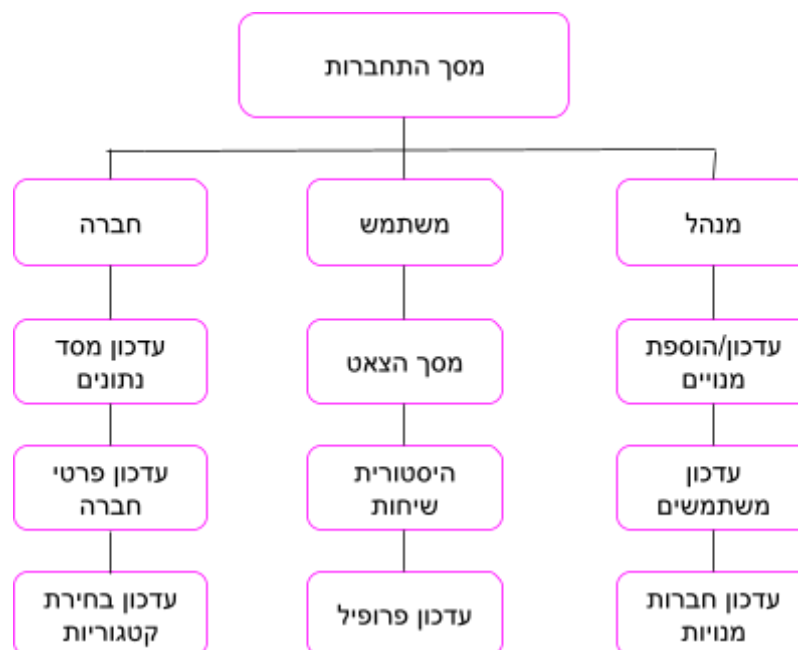
הבקשה נשלחת אל ה-Controller המתאים בשרת #C.

ה-Controller משתמש במחלקות הבקשה ובמחלקות המודל כדי לשמור או לשלוף נתונים ממסד הנתונים.

כאשר מדובר בשיחת משתמש, המידע עובר גם לשרת Python לצורך עיבוד השאלה ובניית תשובה.

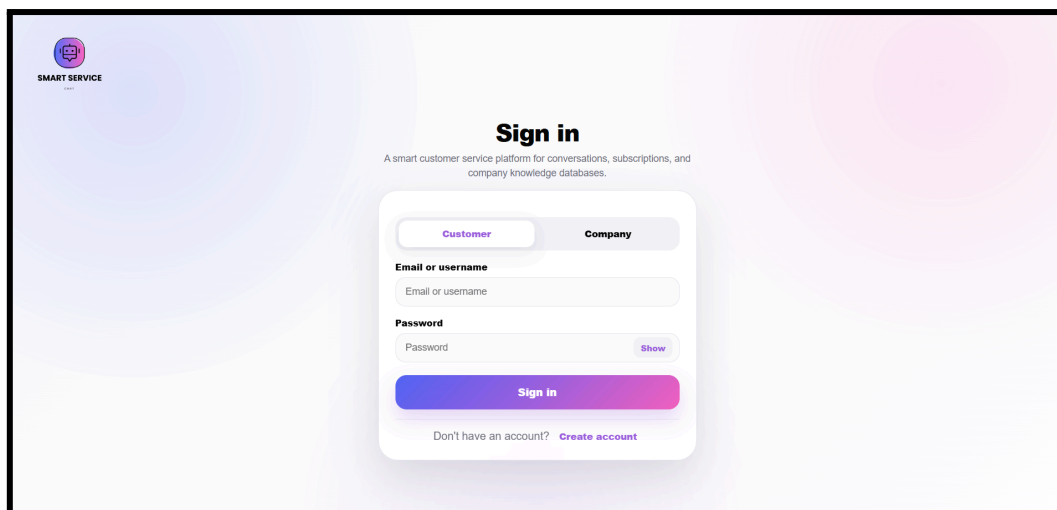
בסיום התהליך התשובה מוחזרת למשתמש, ובמקרה הצורך נשמר דו"ח שיחה במסד הנתונים.

16. תיאור מסכים המתאר את היררכיית המסכים והמעברים ביניהם



17. עבור כל מסך באפליקציה

מסך התחברות



Sign in

A smart customer service platform for conversations, subscriptions, and company knowledge databases.

Customer **Company**

Email or username

Email or username

Password

Password [Show](#)

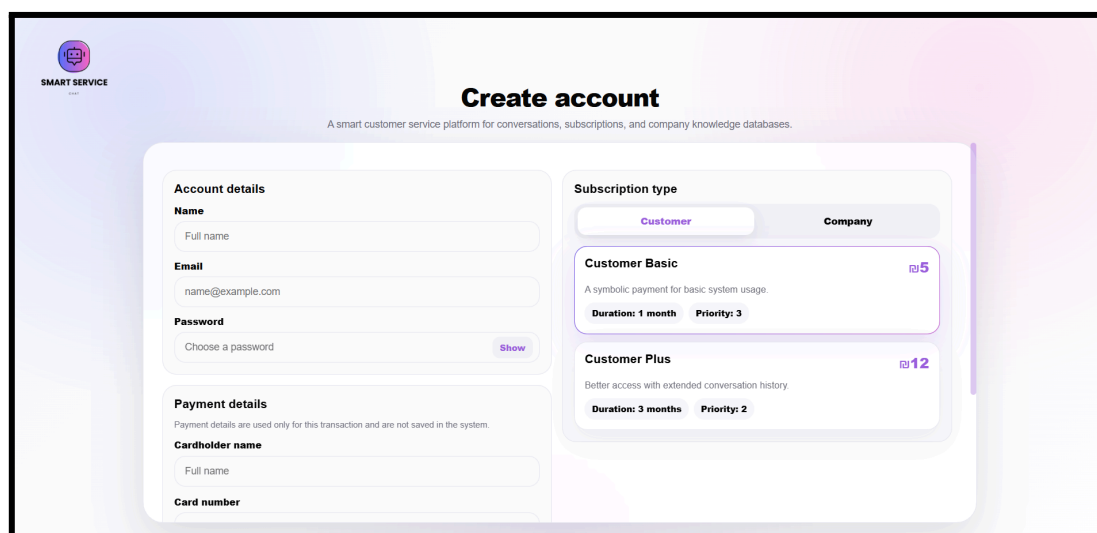
Sign in

Don't have an account? [Create account](#)

מסך זה מאפשר למשתמש או לחברה להתחבר למערכת באמצעות שם משתמש או מייל וסיסמה.

במסך מופיע טופס התחברות הכולל בחירה בין **Customer** לבין **Company**, שדה להזנת מייל או שם משתמש, שדה להזנת סיסמה וכפתור התחברות. בנוסף קיימת אפשרות להצגת הסיסמה וכפתור מעבר למסך יצירת חשבון חדש.

הרשמת משתמש



Create account

A smart customer service platform for conversations, subscriptions, and company knowledge databases.

Account details

Name

Full name

Email

name@example.com

Password

Choose a password [Show](#)

Payment details

Payment details are used only for this transaction and are not saved in the system.

Cardholder name

Full name

Card number

Subscription type

Customer **Company**

Customer Basic ₪5

A symbolic payment for basic system usage.

Duration: 1 month **Priority: 3**

Customer Plus ₪12

Better access with extended conversation history.

Duration: 3 months **Priority: 2**

מסך הרשמת משתמש משמש ליצירת חשבון חדש עבור משתמש רגיל במערכת. במסך זה הלקוח מזין פרטים אישיים כמו שם, אימייל וסיסמה, ממלא פרטי תשלום לצורך רכישת מנוי, ובוחר את סוג המנוי המתאים לו מתוך האפשרויות המוצגות. לאחר מילוי הנתונים ואישור ההרשמה, המערכת שומרת את פרטי המשתמש והמנוי במסד הנתונים ומאפשרת לו להשתמש במערכת הצ'אט.

הרשמת חברה

SMART SERVICE

Create account

A smart customer service platform for conversations, subscriptions, and company knowledge databases.

Account details

Company name

Email

Password

 [Show](#)

Subscription type

Customer **Company**

Company Basic ₪99

For small companies with a low number of support requests.

Duration: 1 month **Priority:** 3

Company Pro ₪499

For companies with a medium number of support requests.

Duration: 6 months **Priority:** 2

Company Premium ₪899

High priority, extended service, and advanced management.

Duration: 1 year **Priority:** 1

Payment details

Payment details are used only for this transaction and are not saved in the system.

Cardholder name

Card number

מסך הרשמת חברה משמש ליצירת חשבון חדש עבור חברה המעוניינת להצטרף למערכת כמנויה. במסך זה החברה מזינה פרטי חשבון כמו שם חברה, אימייל וסיסמה, ממלאת פרטי תשלום, ובוחרת את סוג המנוי המתאים לה מתוך מנויי החברות המוצגים. לאחר אישור ההרשמה, פרטי החברה והמנוי נשמרים במסד הנתונים, והחברה יכולה להוסיף מידע שישמש את המערכת במענה ללקוחות.

צאט משתמש

nechami moriel user [Profile](#) [Logout](#)

[+ New chat](#)

Conversation history

- New conversation** Now
- Previous conversation** 24.5.2026
- I'm looking for comfortable head** 19.5.2026
- Hi, I want a smartwatch that wor** 19.5.2026
- Hello, I need some clarification** 19.5.2026
- Hi, I'm thinking about buying yo** 19.5.2026

I'm looking for comfortable head Smart customer service in real time [Online](#)

I'm looking for comfortable headphones for working from home.

Over-ear, comfort is key.

Can I connect to multiple devices?

Over-ear or in-ear?

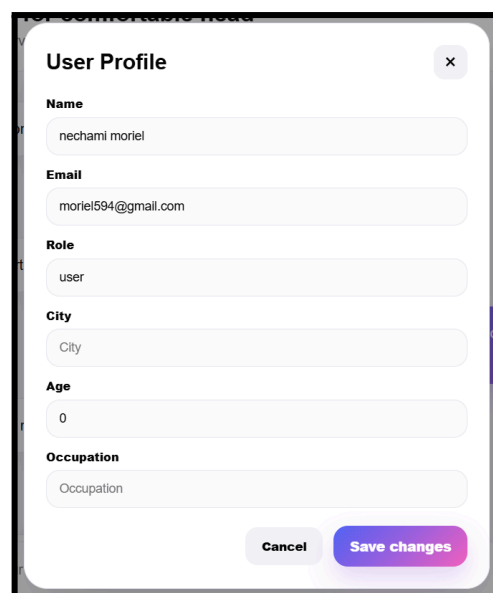
The NoiseCancel Pro series offers active noise cancellation, memory foam padding, and 30 hours battery life.

Yes, switch between laptop and phone easily.

Type your message... [Send](#)

מסך צ'אט משתמש משמש לניהול שיחה בין הלקוח לבין מערכת Smart Service Chat. במסך זה המשתמש יכול לפתוח שיחה חדשה, לשלוח הודעות טקסט או קול, לקבל תשובות מהמערכת בזמן אמת, ולצפות בהיסטוריית שיחות קודמות. בנוסף מוצגים פרטי המשתמש, מצב החיבור, כפתור יציאה ואזור כתיבת הודעה לשליחת שאלה חדשה.

פרופיל משתמש



User Profile [X]

Name
nechami moriel

Email
moriel594@gmail.com

Role
user

City
City

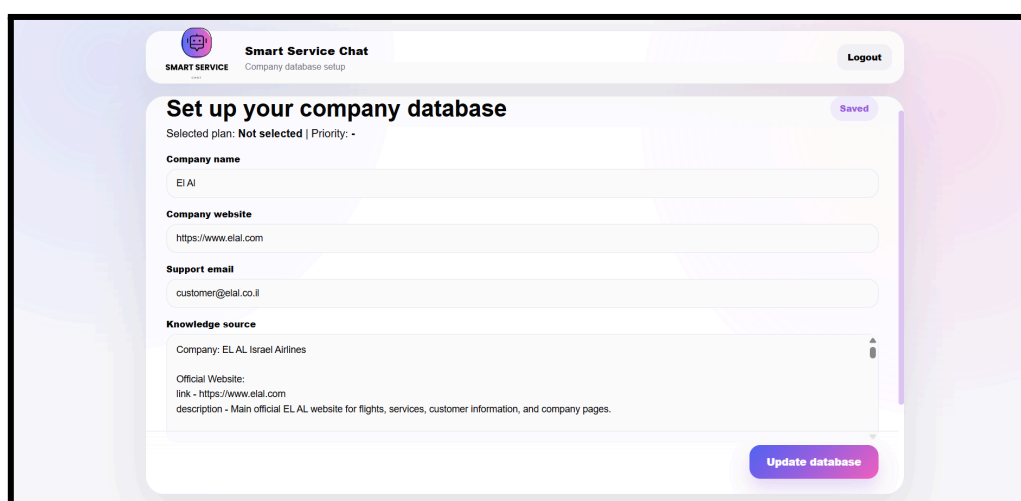
Age
0

Occupation
Occupation

Cancel Save changes

מסך פרופיל משתמש משמש להצגת ועדכון פרטי המשתמש המחובר למערכת. במסך זה מוצגים פרטים כמו שם, אימייל, תפקיד, עיר, גיל ועיסוק, והמשתמש יכול לעדכן חלק מהנתונים ולשמור את השינויים באמצעות כפתור השמירה, או לבטל את הפעולה ולחזור למסך הקודם.

עדכון מסד נתוני חברה מנויה



Smart Service Chat Logout

SMART SERVICE Company database setup

Set up your company database Saved

Selected plan: Not selected | Priority: -

Company name
El Al

Company website
https://www.elal.com

Support email
customer@elal.co.il

Knowledge source
Company: EL AL Israel Airlines
Official Website:
link - https://www.elal.com
description - Main official EL AL website for flights, services, customer information, and company pages.

Update database

מסך עדכון מסד נתוני חברה מנויה משמש לחברה הרשומה במערכת להזין ולעדכן את פרטי מאגר הידע שלה. במסך זה החברה ממלאת פרטים כמו שם החברה, אתר החברה, אימייל תמיכה ומקור הידע שעליו המערכת תתבסס בעת מענה ללקוחות. לאחר לחיצה על עדכון, המידע נשמר במסד הנתונים ומשמש את מערכת הצ'אט לחיפוש תשובות הקשורות לאותה חברה.

בחירת קטגוריות

Choose company categories

The priority is automatic according to the company subscription.

Add new category Add category

☐ Candy
 ☐ Electrical products
 ☐ Flights

☐ food
 ☒ Jewelry
 ☐ Shoes

☐ Transportation services

Cancel Done

מסך בחירת קטגוריות משמש לחברה הרשומה/הנרשמת במערכת להזין ולעדכן את הקטגוריות. המידע נשמר במסד הנתונים ומשמש את מערכת הצ'אט לחיפוש תשובות על פי קטגוריות ועל פי סדר עדיפויות.

ניהול מערכת: הצגת/עדכון/מחיקת משתמשים

System Management

View, edit, activate, deactivate and delete system data.

Users Companies Subscriptions Refresh

ID	Name	Email	City	Age	Occupation	Role	Subscription ID	Actions
1	mosh chen	mc123456@gmail.com		54		user	4	Save Delete
2	tamar moriel	moriel9847594@gmail.co	Tel Aviv	0		admin	5	Save Delete
3	nechami moriel	moriel594@gmail.com		0		user	4	Save Delete

מסך ניהול משתמשים במנהל מערכת משמש להצגת משתמשי המערכת ולעדכון הפרטים שלהם. במסך זה המנהל יכול לצפות ברשימת המשתמשים, לערוך נתונים כמו שם, אימייל, עיר, גיל, עיסוק, תפקיד ומזהה מנוי, לשמור שינויים עבור כל משתמש.

ניהול מערכת: הצגת/עדכון/מחיקת חברות מנויות

System Management

View, edit, activate, deactivate and delete system data.

Users

Companies

Subscriptions

Refresh

ID	Company name	Status	Subscription ID	Subscription start	Actions
1	EI AI	Active	1	2026-05-27	Save Delete
2	JBL	Active	2	2026-05-27	Save Delete
3	osem	Active	3	2026-05-27	Save Delete
4	Eged	Active	1	2026-05-27	Save Delete
5	magnolia	Active	2	2026-05-27	Save Delete

מסך ניהול חברות מנויות משמש את מנהל המערכת לצפייה ועדכון של החברות הרשומות במערכת. במסך זה המנהל יכול לראות את רשימת החברות, לערוך את שם החברה, לשנות את סטטוס הפעילות שלה, לעדכן את מזהה המנוי שלה, לשמור שינויים ולרענן את הנתונים המוצגים.

ניהול מערכת: הצגת/עדכון סוגי מנויים

System Management

View, edit, activate, deactivate and delete system data.

Users

Companies

Subscriptions

Refresh

+ Add subscription

ID	Name	Days	Priority	Price	Description	Type	Status	Actions
3	Company Premium	365	1	899	High priority, extended serv	Company	Inactive	Save Delete
2	Company Pro	180	2	499	For companies with a medi	Company	Active	Save Delete
1	Company Basic	30	3	99	For small companies with a	Company	Active	Save Delete
5	Customer Plus	90	2	12	Better access with extende	Customer	Active	Save Delete
4	Customer Basic	30	3	5	A symbolic payment for bas	Customer	Active	Save Delete

מסך ניהול סוגי מנויים משמש את מנהל המערכת לצפייה, עריכה והוספה של מנויים במערכת. במסך זה המנהל יכול לעדכן את שם המנוי, מספר הימים, רמת העדיפות, המחיר, התיאור וסוג המנוי - לקוח או חברה - לשמור שינויים, להוסיף סוג מנוי חדש ולרענן את הנתונים המוצגים.

מסך נשמר בהצלחה

✓

נשמר בהצלחה

סוג המנויים והקטגוריות של החברה נשמרו בהצלחה.

Back to company setup

מסך זה מוצג כאשר נשמרים/מעודכנים דברים בהצלחה במסד נתונים.

18. הסבר על תפקידי אלמנטים בתצוגת הפרויקט

מסך התחברות

מסך זה מכיל אלמנטים המשמשים לכניסה למערכת.

1. כפתורי הבחירה **Customer / Company** מאפשרים לבחור האם להתחברות היא של לקוח רגיל או של חברה.
2. תיבת הטקסט **Email or username** משמשת להזנת שם משתמש או אימייל.
3. תיבת הטקסט **Password** משמשת להזנת סיסמה.
4. כפתור **Show** מאפשר להציג או להסתיר את הסיסמה.
5. כפתור **Sign in** שולח את פרטי ההתחברות לבדיקה מול השרת.
6. הקישור **Create account** מעביר את המשתמש למסך הרשמה.

מסך הרשמת משתמש

מסך זה מכיל אלמנטים ליצירת חשבון לקוח חדש.

1. תיבות הטקסט **Name, Email** ו-**Password** משמשות להזנת פרטי החשבון.
2. כפתור **Show** מאפשר להציג או להסתיר את הסיסמה.
3. אזור **Payment details** כולל שדות להזנת פרטי תשלום עבור רכישת המנוי.
4. כפתורי הבחירה **Customer / Company** מאפשרים לבחור את סוג הנרשם.
5. כרטיסיות המנויים מציגות את סוגי המנויים, המחיר, משך המנוי ורמת העדיפות, ולחיצה עליהן בוחרת את המנוי הרצוי.

מסך הרשמת חברה

מסך זה משמש לרישום חברה חדשה למערכת.

1. תיבת הטקסט **Company name** משמשת להזנת שם החברה.
2. תיבת **Email** להזנת כתובת מייל.
3. תיבת **Password** להזנת סיסמה.
4. אזור **Payment details** משמש להזנת פרטי תשלום.
5. כפתור **Choose categories** משמש להצגת קטגוריות.
6. כרטיסיות מנויי החברה מציגות את האפשרויות הקיימות, כגון Company Basic, Company Pro ו-Company Premium, כולל מחיר, משך מנוי ועדיפות. בחירת מנוי קובעת את סוג השירות שהחברה תקבל במערכת.

מסך בחירת קטגוריות

מסך זה משמש לבחירת קטגוריות לחברה חדשה למערכת.

1. תיבת הטקסט **Add new category** משמשת להזנת שם קטגוריה חדשה.
2. כפתור **Add category** משמש להוספת הקטגוריה החדשה למסד הנתונים והוספתו להצגה.
3. כפתורי הקטגוריה משמשים לבחירת קטגוריה המתאימה לחברה.
4. כפתור **Done** משמש לבחירה ואישור הקטגוריות הנבחרות.
5. כפתור **Cancel** משמש לביטול העדכון.

מסך צ'אט משתמש

מסך זה מכיל אלמנטים לניהול שיחה עם המערכת.

1. כפתור **New chat** פותח שיחה חדשה.
2. רשימת **Conversation history** מציגה שיחות קודמות ומאפשרת לחזור אליהן.
3. תיבת ההקלדה בתחתית המסך משמשת להזנת הודעה חדשה.
4. כפתור **Send** שולח את ההודעה למערכת.
5. כפתור המיקרופון מאפשר שליחת שאלה קולית.
6. כפתור **Profile** פותח את פרטי המשתמש.
7. כפתור **Logout** מנתק את המשתמש מהמערכת.

מסך פרופיל משתמש

מסך זה מכיל שדות להצגת ועדכון פרטי המשתמש.

1. תיבות הטקסט **Name, Email, Role, City, Age** ו-**Occupation** מציגות את פרטי המשתמש.
2. כפתור **Save changes** שומר את השינויים במסד הנתונים.
3. כפתור **Cancel** מבטל את הפעולה.
4. כפתור הסגירה סוגר את חלון הפרופיל.

מסך עדכון מסד נתוני חברה

מסך זה מאפשר לחברה לעדכן את מאגר הידע שלה.

1. תיבת **Company name** משמשת להזנת שם החברה.
2. תיבת **Company website** משמשת להזנת אתר החברה.
3. תיבת **Support email** משמשת להזנת כתובת שירות.
4. אזור **Knowledge source** משמש להזנת המידע שעליו המערכת תתבסס במענה ללקוחות.
5. כפתור **Update database** שומר את המידע המעודכן במסד הנתונים.
6. כפתור **Logout** מנתק את החברה מהמערכת.

מסך ניהול משתמשים

מסך זה משמש את מנהל המערכת לניהול משתמשים.

1. הכפתורים **Users, Companies** ו-**Subscriptions** מאפשרים מעבר בין סוגי הנתונים.
2. כפתור **Refresh** מרענן את המידע.
3. בטבלת המשתמשים מופיעים שדות עריכה כמו שם, אימייל, עיר, גיל, עיסוק, תפקיד ומזהה מנוי.
4. כפתור **Save** בכל שורה שומר את השינויים עבור אותו משתמש.
5. כפתור **Delete** משמש למחיקת משתמש.

מסך ניהול חברות

מסך זה מאפשר למנהל לערוך חברות הרשומות במערכת.

1. הטבלה כוללת שדות כמו מזהה חברה, שם חברה, סטטוס פעילות ומזהה מנוי.
2. רשימת הבחירה **Status** מאפשרת לשנות אם החברה פעילה או לא.
3. כפתור **Save** שומר את השינויים בשורה המתאימה.
4. כפתור **Refresh** טוען מחדש את הנתונים מהשרת.
5. כפתור **Delete** משמש למחיקת חברה.

מסך ניהול סוגי מנויים

מסך זה משמש לניהול מנויי המערכת.

1. הטבלה כוללת שדות לעריכת שם המנוי, מספר הימים, עדיפות, מחיר, תיאור וסוג המנוי — לקוח או חברה.
2. כפתור **Save** שומר את השינויים במנוי.
3. כפתור **Add subscription** מוסיף מנוי חדש למערכת.
4. כפתור **Refresh** מרענן את רשימת המנויים.
5. כפתור **Delete** משמש למחיקת סוג מנוי.

19. הודעות למשתמש למיניהם

- שגיאת התחברות: כאשר המשתמש מזין שם משתמש או סיסמה לא נכונים.
- שדות חסרים בהרשמה: כאשר המשתמש מנסה להירשם בלי למלא את כל הפרטים הנדרשים.
- שגיאת שמירת משתמש: כאשר פרטי המשתמש לא נשמרו במסד הנתונים.
- אישור הרשמה: כאשר משתמש או חברה נרשמים בהצלחה למערכת.
- שגיאת שליחת הודעה: כאשר הודעת הצ'אט לא נשלחת או לא מתקבלת תשובה מהשרת.
- שגיאת מיקרופון: כאשר המשתמש מנסה לשלוח שאלה קולית ואין הרשאה לשימוש במיקרופון.
- אישור שמירת פרופיל: כאשר פרטי המשתמש עודכנו בהצלחה.
- שגיאת עדכון פרופיל: כאשר שמירת השינויים בפרופיל נכשלה.

- אישור עדכון מסד חברה: כאשר חברה עדכנה בהצלחה את מאגר הידע שלה.
- שגיאת עדכון מסד חברה: כאשר חסרים פרטים או שהשמירה נכשלה.
- אישור שמירה במסך מנהל: כאשר מנהל שומר בהצלחה משתמש, חברה או מנוי.
- שגיאת טעינת נתונים: כאשר מסך המנהל לא מצליח לטעון נתונים מהשרת.

20. ממשק משתמש

עקרונות עיצוב וניווט

במערכת נבחר עיצוב נקי, מודרני ואחיד. פלטת הצבעים מבוססת על רקע בהיר, צבעי סגול-ורוד לכפתורים מרכזיים, וטקסט שחור או אפור כהה לקריאות גבוהה. בכל המסכים נשמרת אחידות בעיצוב: כרטיסיות לבנות, פינות מעוגלות, כפתורים בצבעי מעבר, שדות קלט מעוגלים וכפתור יציאה קבוע במסכים הרלוונטיים.

הניווט במערכת פשוט וברור. משתמש יכול לעבור בין התחברות, הרשמה, צ'אט ופרופיל. חברה מנויה עוברת למסך עדכון מאגר הידע שלה, ומנהל מערכת עובר למסכי ניהול משתמשים, חברות ומנויים באמצעות כפתורי בחירה פנימיים.

דרישות חומרה ותוכנה

המערכת פועלת דרך דפדפן ולכן דרישות המשתמש אינן גבוהות. נדרש מחשב שולחני או נייד עם לפחות 4GB RAM, חיבור אינטרנט תקין ודפדפן מודרני כגון Chrome, Edge או Firefox. מומלץ להשתמש במסך מחשב רגיל כדי שהטבלאות, הצ'אט וטפסי הניהול יוצגו בצורה נוחה.

טיפול בשגיאות ומשוב למשתמש

המערכת מציגה הודעות למשתמש כאשר פעולה נכשלת או כאשר חסרים נתונים. לדוגמה, אם המשתמש מזין פרטי התחברות שגויים, חסרים שדות בהרשמה, אין הרשאת מיקרופון, או שמירה במסד הנתונים נכשלה — מוצגת הודעה מתאימה. לאחר פעולות תקינות, כמו שמירת פרופיל, עדכון מאגר חברה או שמירת נתונים במסך מנהל, המשתמש מקבל משוב שהפעולה הצליחה.

הנגשה

הממשק בנוי כך שיהיה ברור ונוח לשימוש. קיימת ניגודיות טובה בין הטקסט לרקע, הכפתורים גדולים וברורים, השדות מסומנים בכותרות מתאימות, והעיצוב שומר על מרווח נוח בין רכיבים. בנוסף, במסכי הסיסמה קיים כפתור Show המאפשר להציג או להסתיר את הסיסמה, כדי להקל על המשתמש בזמן ההקלדה.

21. קוד התוכנית

receive_chat_message - קבלת הודעה מצד לקוח

```
#קבלת הודעה מצד לקוח
def receive_chat_message(data: ChatMessageRequest) :
    message = data.message.strip()
    #אם הודעה ריקה
    if message == "":
        return {
            "success": False,
            "answer": "",
            "chatId": data.chatId,
            "messages": get_chat_messages(data.chatId)
        }

    user_data = {
        "userId": data.userId,
        "username": data.nameUser,
        "email": data.email,
        "city": data.city,
        "age": data.age,
        "occupation": data.occupation,
        "role": data.role
    }

    chat_id = str(data.chatId)
    #אם צאט זה לא קיים במילון שאלה-תשובה
    if chat_id not in dic_question_answer:
        #הוספת השיחה למילון
        dic_question_answer[chat_id] = create_chat_structure(
            user_id=data.userId,
            chat_id=data.chatId,
            report_id=data.reportId,
            title="New conversation",
            conversation=[],
            is_changed=False
        )

    #עדכון פרטי משתמש
    dic_question_answer[chat_id]["user_id"] = data.userId
    dic_question_answer[chat_id]["report_id"] = data.reportId
    #אם לא קיים מצביע לעץ חיפוש בינארי - לנושאי השיחה
    if ("topics_tree" not in dic_question_answer[chat_id] or
        dic_question_answer[chat_id]["topics_tree"] is None):
```

```

dic_question_answer[chat_id]["topics_tree"] = BinarySearchTree()

# הוספת הודעה למילון שיחות נוכחיות
add_question_answer_to_dic(chat_id, "User: " + message)

print("User ID:", data.userId)
print("Chat ID:", data.chatId)
print("Report ID:", data.reportId)
print("Message:", message)
print("User data:", user_data)

try:
    # פרטי שיחה נוכחית
    chat_history = get_chat(chat_id)
    # זימון פונקציה טיפול בהודעה
    answer = message_handling(message, user_data, chat_history)
    # מודפס - None אם ההודעה הינה
    if answer is None:
        answer = "I could not build an answer for this message."
    else:
        print()
        print(answer)

    # הוספת התשובה למילון על פי קוד צאט
    add_question_answer_to_dic(chat_id, "Bot: " + answer)
    # החזרת התשובה למשתמש
    return {
        "success": True,
        "answer": answer,
        "chatId": chat_id,
        "reportId": dic_question_answer[chat_id].get("report_id"),
        "messages": get_chat_messages(chat_id)
    }

# במקרה אירעה שגיאה
except Exception as error:
    print("Chat message handling error:", error)
    traceback.print_exc()

    return {
        "success": False,
        "answer": "An error occurred while processing your message.",
        "chatId": chat_id,
        "reportId": dic_question_answer[chat_id].get("report_id"),
        "messages": get_chat_messages(chat_id)
    }

```

קלט: פרטי משתמש והודעת המשתמש.

פלט: תשובה למשתמש או הודעת שגיאה.

הפונקציה מקבלת הודעה מהלקוח, בודקת שהיא לא ריקה, בונה אובייקט נתוני משתמש, שולחת את ההודעה לעיבוד, ומחזירה תשובה לממשק.

message_handling - האלגוריתם הראשי של מענה לשאלה

```
#טיפול בהודעה
def message_handling(text, user_data, chat_history):
    #מציאת סוג הפניה
    category = CategorizedByTopic.categories(text)

    #המתאים לסוג הפניה JSON שאיבת ה-
    json_question = JsonToTopic.get_json(category)
    #בערכים משאלת הלקוח JSON מילוי ה-
    full_json = JsonFilling.json_filling(category, text, json_question)
    #מציאת הנתונים החסרים
    json_question = finding_missing_details(json_question, user_data,
chat_history)
    #מלאים JSON בדיקה שכל הערכים הנחוצים ב-
    missing_item = if_everything_full(full_json)
    if len(missing_item) > 0:
        return "The following items are missing: " + ", ".join(missing_item)
    #בניית השאלה המקוצרת
    the_question = build_question(full_json)

    #סוג הפניה לא תלונה
    if category != "complaint" and category != "other":
        item_in_question = json_question["required"]
        #חיפוש תשובה במסד נתונים
        lst_par_to_answer =
SearchInDB.handling_company_database(the_question[0], json_question[item_in_questio
n[1]])
        if len(lst_par_to_answer) < settings["NUM_TO_ANSWER"]:
            #במקרה הצורך - חיפוש כללי
            contents = SearchInDB.handling_Internet_database(the_question)
            #טיפול באתרים שנפתחו
            SearchInDB.select_most_suitable_simplifiers(contents, the_question,
category, user_data)

            #הוצאת מספר התשובות המקוצרות המתאימות ביותר
            text_to_answer =
dic_BinarySearchTree[user_data["userId"]].bst_to_list(len(lst_par_to_answer))
            lst_par_to_answer = lst_par_to_answer + text_to_answer

            if(len(text_to_answer) == 0):
```

```

        return "Error, The URL not opening AND There is no suitable answer."

    # בניית התשובה
    answer_final =
    AnswerBuilding.create_answer(the_question, "\n".join(text_to_answer))
    return answer_final
else:
    if category == "other":
        return "I am not allowed to answer this question because it does not
deal with customer service."
    # טיפול בתלונה
    else:
        return complaint_handling(text, user_data, chat_history)

```

קלט: טקסט השאלה ונתוני המשתמש.

פלט: תשובה סופית למשתמש.

הפונקציה מסווגת את השאלה, שואבת JSON מתאים, משלימה פרטים חסרים, בונה שאלתת חיפוש, מחפשת מידע במסדי נתוני החברות המנויות ורק לאחר מכן - במקרה הצורך - חיפוש כללי, מחלצת תשובות, מדרגת אותן, בוחרת קטעים מתאימים ומעבירה אותם לבניית תשובה סופית. וכן מטפלת בתלונה.

finding_missing_details - מציאת פרטים חסרים

```

# מציאת פרטים חסרים
def finding_missing_details(json_question, user_data, chat_history):
    # JSON שליופת רשימת ההנחיות למציאת הפרטים מתוך ה-
    search_missing_details = json_question["search_missing_details"]
    details_topics = None
    details = None
    # מעבר על הרשימה
    for item in search_missing_details:
        # אם הפריט שצריך להתקיים מלא
        if json_question[item["if_field_exists"]] != "":
            where_search = ""
            if details_topics == None:
                # חיפוש הערך החסר בעץ נושאי השיחה
                details_topics =
            chat_history["topics_tree"].search_node_in_tree(chat_history["topics_tree"].Root,
            FindSimilar.embeddings_encode(json_question[item["if_field_exists"]]), json_questio
            n[item["if_field_exists"]], "topics")
            # תעבור על הפריטים שאולי ריקים
            for detail in item["if_fields_missing"]:
                # אם הפריט ריק
                if user_data[detail["field"]] == "":
                    # שליופת עבור פריט זה איפה לחפש - מה שהוא/מה שיש לו

```

```

        where_search = item["search_missing_value_in"]
        # אם חיפוש הפריט החסר הוא בדברים שיש לו
        if where_search == "ThingsTathUserHas":
            # מילוי הערך החסר מתוך נושאי השיחה
            json_question =
JsonFilling.json_filling_form_missing_details(json_question,details_topics.value.t
hingsTheUserHasContent,detail)
            # אם לא נמצא מתאים
            if details_topics == None or
            json_question[detail["field"]] == "":
                # מציאת הצומת בעץ הדברים שיש לו המתאימה לערך המלא
                details =
dic_tree_things[user_data["userId"].search_node_in_tree(dic_tree_things[user_data
["userId"]].Root,FindSimilar.embeddings_encode(json_question[item["if_field_exists
"]]),json_question[item["if_field_exists"]],"things")
                if details != None:
                    # מילוי הפריט החסר מתוך עץ הדברים שיש לו
                    json_question =
JsonFilling.json_filling_form_missing_details(json_question,details.value.thingsTh
eUserHasContent,detail)

            else:
                json_question[detail["field"]] =
user_data.get(detail["search_value"],"")
                # הכנסת הפריט לנושאי השיחה
                details_topics =
chat_history["topics_tree"].insert_the_item_into_node(details_topics,
FindSimilar.embeddings_encode(json_question[item["if_field_exists"]]) ,
json_question[item["if_field_exists"]], json_question[detail["field"]],"topics")

        return json_question

```

קלט: JSON מלא או חלקי של השאלה, ופרטי משתמש.

פלט: JSON מלא בנתונים החסרים מתוך - נתונים ישנים, פרטי משתמש.

הפונקציה שולפת את פרטי טיפול בפרטים חסרים מ-JSON, במקרה והערך ריק - חיפוש בעץ דברים שיש לו/פרטי משתמש - על פי ההנחיה, ומילוי הערך ב-JSON.

search_node_in_tree - חיפוש הצומת המתאימה לנושא

```

#חיפוש הצומת המתאימה לנושא
def search_node_in_tree(self,node,text_emb,text,issue):
    if node == None:
        #אם מתבצע חיפוש בנושאי השיחה
        if issue == "topics":

```

```

        #יצירת צומת חדשה עם נושא
        return self.insert_the_item_into_node(node,text_emb,text,"",issue)
    else:
        return None
    if FindSimilar.matching_two_vectors(text_emb,
node.key)>settings["60_percent"]:
        if FindSimilar.matching_two_vectors(text_emb,
node.key)<settings["80_percent"]:
            return self.search_node_in_tree(node.right, text_emb,text,issue)
        if issue == "topics":
            #אם נמצא צומת מתאימה - הוספת הנתון לצומת
            node =
self.insert_the_item_into_node(node,text_emb,node.value.thingsTheUserHasTopic
,text,issue)
            return node
        return self.search_node_in_tree(node.left, text_emb,text,issue)

```

קלט: עץ, צומת ספציפית לעץ, וקטור מספרי לטקסט, טקסט.

פלט: צומת המתאימה לנושא.

הפונקציה עוברת על העץ על פי חיפוש בינארי - התאמה בין הוקטור ל-Key של צומת. לאחר מכן מזמנת את הפונקציה של הכנסת הנושא לצומת.

insert_the_item_into_node - בניית שאילתת חיפוש מתוך ג'סון

```

#הכנסת הערך לצומת העץ
def insert_the_item_into_node(self,node,text_emb,title,text,issue):
    if node == None:
        thing = ThingsTheUserHas(thingsTheUserHasId= 0, userId = 0 ,
thingsTheUserHasTopic = title , thingsTheUserHasContent=[text] if title != text
else [])
        if issue == "topics":
            self.insert_to_topic(text_emb,thing)
        else:
            self.insert_to_things(text_emb,thing)
        return Node(text_emb, thing)
    #אם פרט זה קיים כבר במאגר - לא להכניס ולהחזיר את הצומת
    if title.lower() == node.value.thingsTheUserHasTopic.lower() and text.lower()
in node.value.thingsTheUserHasContent or text.lower() ==
node.value.thingsTheUserHasTopic.lower() :
        return node
    node.value.thingsTheUserHasContent.append(text)
    return node

```

קלט: עץ, צומת ספציפית לעץ, וקטור מספרי לטקסט, טקסט.

פלט: צומת לאחר עדכון.

הפונקציה עוברת על הנושא והפרטים שבצומת - אם הנושא לא קיים שם, מכניסים אותו לרשימה. במקרה והצומת None - לא נמצא צומת מתאימה לנושא, יצירת צומת חדשה והוספת לעץ.

if_everything_full - בדיקת שדות חובה

```
#האם כל הפרטים החשובים מלאים? מחזיר מערך של הפרטים החסרים
#במקרה ולא חסר פרטים מחזיר מערך ריק
def if_everything_full(json):
    missing_item = []
    #הוצאת רשימת הערכים החשובים
    required = json["required"]
    for item in required:
        if json[item] == "":
            missing_item.append(item)
    return missing_item
```

קלט: JSON של השאלה.

פלט: רשימת שדות חסרים.

הפונקציה בודקת האם כל השדות שהוגדרו כחובה קיימים, ואם חסר מידע היא מחזירה את שמות השדות החסרים.

handling_Internet_database - אלגוריתם חיפוש כללי

```
#main - במקרה הצורך - חיפוש כללי
def handling_Internet_database(the_question):
    response = tavilApi.search_in_DB(the_question[0])
    results = response["results"]
    #הוצאת הכותרות
    titles = [r["title"].rsplit("-", 1)[0].strip() for r in response["results"]]
    #מציאת התאמה
    similarity_scores = matching_percentages(the_question[0], titles)
    #יצירת זוגות: אחוז התאמה עם תשובת האינטרנט
    pairs = list(zip(results, similarity_scores))
    #סינון לפי סף התאמה
    filtered = [p for p in pairs if p[1] >= settings["THRESHOLD"]]
    #מיון לפי אחוז התאמה (מהגבוה לנמוך)
    filtered.sort(key=lambda x: x[1], reverse=True)

    #בכדי למצוא תשובה URL איחוד
    urls = []
    for url in filtered:
```

```

urls.append(url[0]["url"])

# זימון הפעולה המחלצת טקסט מתוך URL
content = []
# המתאימים ביותר URL פתיחת ה-
contents = information_from_URL(urls)
return contents

```

קלט: טקסט השאלה.

פלט: טקסט 3 אתרים המתאימים ביותר לשאלה - שקישורים נפתח בהצלחה.

הפונקציה מבצעת חיפוש כללי לפי השאלה, מחשבת התאמה בין השאלה לכותרות התוצאות, מסננת וממיינת את התוצאות הרלוונטיות ביותר, מחלצת מהן קישורים ומחזירה את התוכן שנשלף מהם.

select_most_suitable_simplifiers - מציאת המשפטים המתאימים ביותר

```

# מציאת המשפטים המתאימים ביותר
def select_most_suitable_simplifiers(contents, the_question, category, user_data):
    dic_BinarySearchTree[user_data["userId"]] = BinarySearchTree()
    # מעבר על האתרים שפתחנו מה- URL
    for content in contents:
        # חלוקת התוכן לפסקאות
        paragraphs =
SplitTextToParagraphs.split_titles_and_large_sections(content)
        # עבור כל פסקה
        for par in paragraphs:
            # מציאת התשובה המתאימה ביותר באותה פסקה
            short_sen = distilBert.pipeline_DistilBert(par, the_question[0])
            # בדיקת אחוז ההתאמה - כמה התשובה הקצרה מתאימה לשאלת הלקוח בתור תשובה
            score = predict_match("topic: " + category + " , question: " +
the_question[0] + " , answer: " + short_sen , the_question[1])
            # הכנס התשובה הקצרה לעץ כשהמפתח הוא אחוז ההתאמה

dic_BinarySearchTree[user_data["userId"]].insert_to_string(score, short_sen)

```

קלט: טקסט האתרים שנפתחו, השאלה המקוצרת, קטגוריה, פרטי משתמש.

פלט: אין.

הפונקציה עוברת על האתרים, מחלקת לפסקאות, מוציאה משפט תשובה - DistilBert, הכנסה לעץ חיפוש בינארי על פי רמת ההתאמה.

build_question - בניית שאילתת חיפוש מתוך ג'סון

```
#JSON בניית שאלה מתוך ה-
def build_question(json):
    #רשימת השאלה
    the_question = []
    #רשימת משקלי מילות השאלה
    token_weights = [0.2, 3.0, 1.0, 1.0]
    #הוצאת רשימת הערכים לחיפוש מה-JSON
    search = json["search"]
    #מעבר על הערכים
    for item in search:
        bool = False
        #במידה ויש מילה להוסיף לפני הערך במידה וערך זה מלא
        if "text_if_field_exists" in item:
            bool = True
        field = item["field"]
        #אם הוא ריק לא להוסיף לשאלה - לא אותו ולא את המילת הקישור שלו
        if json[field] == '':
            bool = False
            continue
        #אם הוא לא ריק ויש מילת קישור לפני
        if bool == True:
            #מוסיפים לשאלה את המילת קישור
            the_question.append(item["text_if_field_exists"])
            #הוספת מישקל כללי למילת הקישור
            token_weights.append(1.0)
            bool = False
        #הוספת הערך לשאלה
        the_question.append(json[field])
        #הוספת משקלים לרשימת המשקלים על פי הערכים המסומנים
        for word in json[field].split(" "):
            token_weights.append(item["token_weight"])
    #החזרת רשימה, במיקום הראשון - השאלה המקוצרת כמחרוזת, במיקום השני - רשימת המשקלים
    return [" ".join(the_question), token_weights]
```

קלט: JSON מלא או חלקי של השאלה.

פלט: שאילתת חיפוש ומשקלים למילים.

הפונקציה עוברת על שדות החיפוש ב-JSON, מוסיפה רק שדות שיש בהם מידע, ובונה מהם טקסט חיפוש ממוקד יחד עם משקלים.

json_filling - מילוי הג'סון

```
def json_filling(topic, text, json_data):
```

```

if topic == "pricing":
    return pricing_json_filling(text, json_data)
if topic == "product_specs":
    return product_specs_json_filling(text, json_data)
if topic == "complaint":
    return complaint_json_filling(text, json_data)
if topic == "date_and_time":
    return date_and_time_json_filling(text, json_data)
#אחרת זימון המודל המאומן על שאר הנושאים
return filling_all_the_remaining_json(text, json_data)

```

קלט: סוג השאלה, טקסט השאלה ותבנית JSON.

פלט: JSON מלא יותר עם נתונים שחולצו מהשאלה.

הפונקציה מפנה את השאלה לפונקציית מילוי מתאימה לפי סוג הפנייה, כמו מחיר, מידע על מוצר, תלונה או תאריך ושעה.

pipeline_DistilBert - חילוץ תשובה מתוך טקסט

```

#מציאת התשובה
def pipeline_DistilBert(context, question):
    inputs = tokenizer(question, context, return_tensors="pt" ,
return_offsets_mapping=True)
    offset_mapping = inputs.pop("offset_mapping")[0].tolist()
    with torch.no_grad():
        outputs = model(**inputs)

    #מבודד את הלוגים של המסמך.
    start_logits = outputs["start_logits"][0].tolist()
    end_logits = outputs["end_logits"][0].tolist()

    #חוזרת 20 הערכים להתחלה הגבוהים ביותר, מהגדול - לקטן
    top_start_indices = indices_of_largest(start_logits)
    #חוזרת 20 הערכים לסיום הגבוהים ביותר, מהגדול - לקטן
    top_end_indices = indices_of_largest(end_logits)
    sequence_ids = inputs.sequence_ids(0)
    #מציאת הזוג הראשון המתאים ביותר להיות תשובה לשאלה
    start_idx, end_idx = find_best_logit_pair(start_logits, top_start_indices,
end_logits, top_end_indices, sequence_ids, offset_mapping)
    #אם מחוץ לתחום התשובה - אין תשובה מתאימה
    if start_idx == -1 or end_idx == -1:
        return ""

    #תו ההתחלה/סיום - על פי הזוג שמצאנו
    start_char = offset_mapping[start_idx][0]
    end_char = offset_mapping[end_idx][1]

```

```
# אם הסיום לפני ההתחלה - אין תשובה
if end_char <= start_char:
    return ""

# המשפט כולו שבו נמצאת התשובה המתאימה ביותר לשאלה
answer = the_sentence_of_the_answer(start_char, end_char, context)

# חזרת המשפט
return answer
```

קלט: טקסט מקור ושאלת המשתמש.

פלט: משפט תשובה אפשרי מתוך הטקסט.

הפונקציה מפעילה את DistilBERT על שאלה וטקסט, מוצאת את מיקום התשובה המתאים ביותר, ומחזירה את המשפט המלא שבו נמצאה התשובה.

find_best_logit_pair - בחירת טווח תשובה

```
# מציאת הזוג המתאים ביותר
def find_best_logit_pair(start_logits, best_start_indices, end_logits,
                        best_end_indices, sequence_ids, offset_mapping):
    best_score = float("-inf")
    best_pair = (-1, -1)
    # מעבר על לוגיטי ההתחלה
    for start_idx in best_start_indices:
        # מעבר על לוגיטי הסיום
        for end_idx in best_end_indices:
            # בדיקה שההתחלה לפני הסיום שהם בתוך תחום התוכן
            if sequence_ids[start_idx] != 1 or sequence_ids[end_idx] != 1 or
            end_idx < start_idx or end_idx - start_idx > settings["MAX_ANSWER_LENGTH"] or
            offset_mapping[start_idx][0] == offset_mapping[start_idx][1] or
            offset_mapping[end_idx][0] == offset_mapping[end_idx][1]:
                continue
            # מספר התאמה
            score = start_logits[start_idx] + end_logits[end_idx]

            if score > best_score:
                best_score = score
                best_pair = (start_idx, end_idx)
    # חזרת הזוג המתאים ביותר
    return best_pair
```

קלט: לוגיטים של התחלה וסוף תשובה, אינדקסים מובילים ומיפוי מיקומים.

פלט: זוג אינדקסים של תחילת וסוף התשובה.

הפונקציה בוחרת את זוג האינדקסים בעל הציון הגבוה ביותר, תוך בדיקה שהתשובה נמצאת בתוך טקסט המקור ולא בתוך השאלה.

predict_match - בדיקת התאמה בין תשובה לשאלה

```
#פונקציה לחיזוי אחוז התאמה בין שאלה לתשובה
def predict_match(text: str, token_weights=None) -> float:
    cleaned_text = my_split(text)
    #המרת הטקסט לקלט מספרי עבור המודל
    encoded = encode_pair(tokenizer, cleaned_text, max_len=settings["max_len"],
token_weights=token_weights)
    #העברת הנתונים למכשיר הרצה
    input_ids = encoded["input_ids"].to(device)
    attention_mask = encoded["attention_mask"].to(device)
    token_weights = encoded["token_weights"].to(device)
    #הרצת המודל ללא אימון וחישוב ציון התאמה
    with torch.no_grad():
        score = model(
            input_ids,
            attention_mask,
            token_weights=token_weights,
            return_score=True
        )
    #החזרת ציון ההתאמה כמספר
    return score.item()
```

קלט: טקסט הכולל נושא, שאלה ותשובה אפשרית.

פלט: ציון התאמה בין 0 ל-1.

הפונקציה מפעילה את מודל ה-Transformer המותאם ובודקת עד כמה התשובה מתאימה לשאלה.

insert_to_string - הכנסת מחרוזת לעץ חיפוש בינארי

```
#הכנסת מחרוזת לעץ
def insert_to_string(self, key, value):
    self = self
    #יצירת צומת חדשה
    new_node = Node(key, value)
    #אם הוא עץ חדש
    if self.Root == None:
        self.Root = new_node
        return
    #מצביע לראש העץ
    current = self.Root
    #None עובר על עץ - כל עוד המצביע לא שווה ל-
```

```

while current!=None:
    # אם הערך של הצומת גדולה הערך הצומת החדשה
    if current.key > new_node.key:
        if current.left == None:
            # הגענו לסוף העץ - מכניסים את הצומת החדשה
            current.left = new_node
            return
        # פונים לשמאל
        current = current.left
    # אחרת - הגענו לסוף העץ - מכניסים את הצומת החדשה
    elif current.right == None:
        current.right = new_node
        return
    else:
        # פונים ימין
        current = current.right

```

קלט: ציון התאמה וטקסט תשובה.

פלט: אין פלט.

העץ שומר את התשובות לפי ציון ההתאמה, ולאחר מכן מאפשר לשלוף את הקטעים הטובים ביותר להמשך בניית התשובה.

bst_to_list - הוצאת מספר הקטעים המתאימים ביותר

```

# פונקציה מחזירה את מספר הקטעים שהמפתח שלהם הגבוה ביותר - מהגבוה לנמוך
def bst_to_list_from_high_to_low(self,node,lst_text, len_par_to_answer):
    # אם הגעת למספר או שהחוליה שאליה הגעת - תחזיר את הרשימה
    if node is None or len(lst_text)+len_par_to_answer == settings["NUM_TO_ANSWER"]:
        return lst_text
    # דימון לימין
    lst_text = self.bst_to_list_from_high_to_low(node.right,lst_text)
    # טיפול באמצעי
    # תוסיף את הקטע לרשימה אם NUM_TO_ANSWER אורך הרשימה קטן מ
    if len(lst_text)+len_par_to_answer<settings["NUM_TO_ANSWER"]:
        lst_text.append(node.value)
    # דימון לשמאל - אם עדין אורך הרשימה קטן מ
    if len(lst_text)+len_par_to_answer<settings["NUM_TO_ANSWER"]:
        lst_text = self.bst_to_list_from_high_to_low(node.left,lst_text)
    # החזרת הרשימה
    return lst_text

# פונקציה תווך למציאת מספר קטעים המתאימים ביותר
def bst_to_list(self, len_par_to_answer):

```

```

#רשימת הקטעים
lst_text = []
#זימון הפונקציה למציאת מספר הקטעים המתאימים ביותר
lst_text =
self.bst_to_list_from_high_to_low(self.Root,lst_text,len_par_to_answer)
#החזרת רשימת הקטעים
return lst_text

```

קלט: מצביע לעץ - self.

פלט: רשימת 10 הקטעים בעלי הציון הגבוה ביותר.

הפונקציה עוברת על העץ בסריקה: ימין - אמצע - שמאל. ושולפת את 10 הקטעים הטובים ביותר להמשך בניית התשובה.

create_answer - יצירת תשובה סופית

```

# פונקציה שמקבלת שאלה והקשר, ומחזירה תשובה שנוצרה על ידי המודל
def create_answer(question, context):

    # בניית ההוראות למודל: איך לענות, על סמך מה לענות, ובאיזה פורמט להחזיר
    prompt = f"""
You are a customer support assistant.
Answer the question using only the provided context.
Use only information that appears in the text.
Do not ask a new question.
Do not invent, assume, or add missing details.
If the text does not contain enough information, say that clearly.
Return only valid JSON.
Do not add explanations before or after the JSON.

Now answer this:

Question:
{question}

Text:
{context}

JSON:
"""

    # המרת הטקסט למבנה מספרי שהמודל יכול לקרוא
    inputs = tokenizer(
        prompt,
        return_tensors = settings["return_tensors"],

```

```

        truncation = settings["true"],
        max_length = settings["max_length_to_answer"]
    )

    # הפעלת המודל
    with torch.no_grad():

        # יצירת תשובה חדשה על ידי המודל
        outputs = model.generate(
            **inputs,
            max_new_tokens = settings["max_new_tokens"],
            do_sample = settings["false"],
            num_beams = settings["num_beams"],
            early_stopping = settings["true"],
            no_repeat_ngram_size = settings["no_repeat_ngram_size"],
            repetition_penalty = settings["repetition_penalty"]
        )

        # המרת הפלט המספרי של המודל בחזרה לטקסט רגיל
        answer = tokenizer.decode(outputs[0], skip_special_tokens=True)

    return answer

```

קלט: שאלת המשתמש והקטעים שנבחרו.

פלט: תשובה סופית.

הפונקציה בונה Prompt עבור FLAN-T5-LARGE, דורשת ממנו לענות רק לפי ההקשר שניתן, ומחזירה תשובה טבעית וברורה למשתמש.

save_chat_to_csharp - שמירת שיחה בסיום ההתכתבות

```

# SQL שמירת השיחה+נושאי שיחה ב
def save_chat_to_csharp(user_id: int, chat_id: str):
    chat_id = str(chat_id)

    if chat_id not in dic_question_answer:
        return {
            "success": False,
            "message": "Chat was not found in Python."
        }

    # הצאת הנוכחי
    chat = dic_question_answer[chat_id]

```

```

#לא נעשה שינוי בשיחה - לא שומרים מחדש
if not chat.get("is_changed", False):
    return {
        "success": True,
        "message": "Chat was not changed. No save needed.",
        "reportId": chat.get("report_id")
    }

#רשימת השאלה-תשובה
messages = chat.get("messages", [])

#בדיקה שהלקוח שאל שאלה וזה לא צאט ריק - רק עם הודעת פתיחת הצאט
has_user_message = any(
    isinstance(message, str) and message.strip().startswith("User:")
    for message in messages
)

if not has_user_message:
    return {
        "success": True,
        "message": "Chat has no real user message. No save needed.",
        "reportId": chat.get("report_id")
    }

#סיכום השיחה
conversation_for_save = list(messages)

topics_tree = chat.get("topics_tree")
topics_text = ""
#הפיכת עץ נושאי השיחה לטקסט
if topics_tree is not None and topics_tree.Root is not None:
    topics_text = topics_tree.tree_to_str()

#הוספת כותרת
conversation_for_save.append("")
conversation_for_save.append("Topics in report:")

if topics_text.strip() != "":
    conversation_for_save.extend(topics_text.splitlines())

#הוספת רגשות
conversation_for_save.append("Sentiment and Emotions:")
conversation_for_save.append("Sentiment: Neutral")
conversation_for_save.append("Emotions detected: Neutral")

#API גוף שליחת ה-
body = {
    "userId": user_id,

```

```

    "reportId": chat.get("report_id"),
    "conversation": conversation_for_save
}

#שליחת השיחה
saved_data = make_csharp_post_request(
    f"{CSHARP_API_BASE_URL}/api/reports/save-chat",
    body
)

if "reportId" in saved_data:
    chat["report_id"] = saved_data["reportId"]

chat["is_changed"] = False

return saved_data

```

קלט: מזהה משתמש וקוד שיחת הצ'אט.

פלט: הצלחה או כישלון בשליחת השיחה לשמירה.

הפונקציה שולחת את רשימת השאלות והתשובות + טקסט נושאי השיחה לשרת C# לצורך שמירת ועדכון דו"ח שיחה במסד הנתונים.

22. תיאור מסד הנתונים

מסד הנתונים של המערכת שומר את המידע הדרוש להפעלת SmartService Chat: פרטי משתמשים, חברות מניות, סוגי מנויים, מאגרי מידע של חברות, נתונים נוספים על המשתמשים ודוחות סיכום שיחה.

במהלך השיחה עצמה השאלות והתשובות נשמרות במבנה נתונים זמני בזיכרון, ורק בסיום השיחה נשמר דו"ח מסכם במסד הנתונים. בנוסף, נשמרים במסד הנתונים פרטים המשמשים להתחברות, הרשמה, ניהול מנויים, ניהול חברות ועדכון מאגרי הידע של החברות.

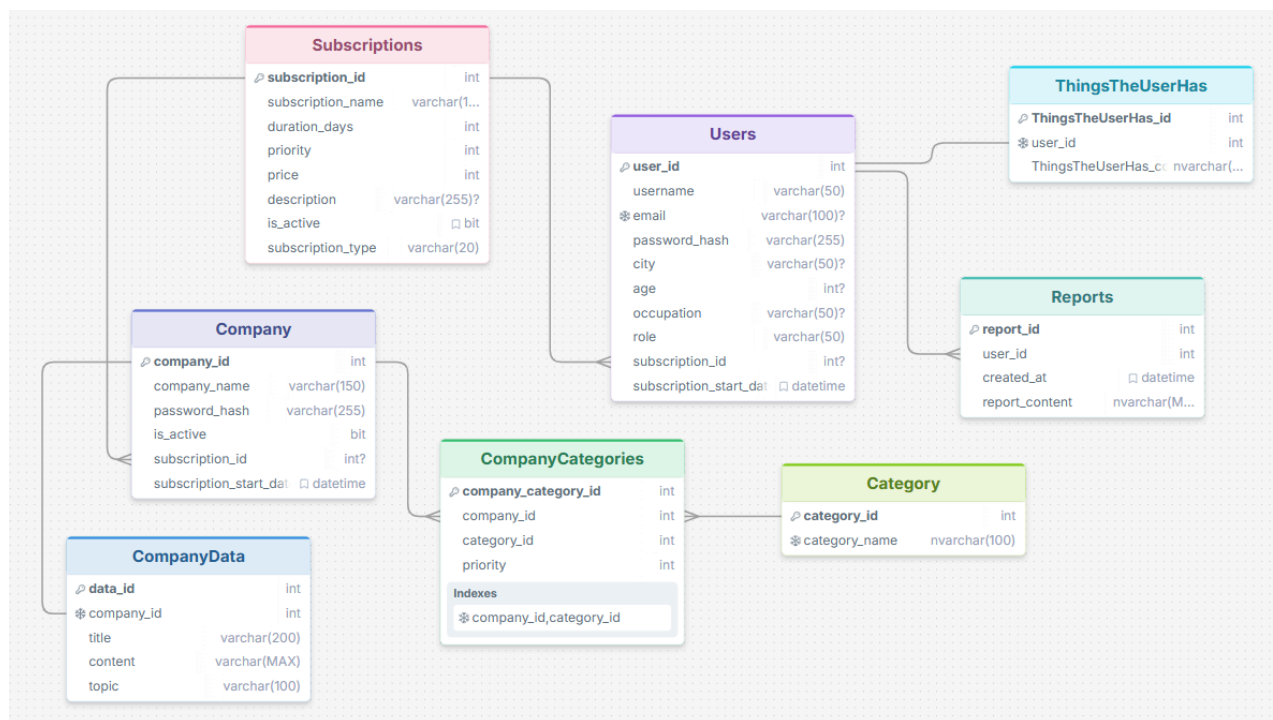
סכמה כללית של הישויות

הישויות המרכזיות במסד הנתונים הן:

- **Users** - משתמשי המערכת ופרטי החשבון שלהם.
- **Companies** - חברות מניות הרשומות במערכת.
- **Subscriptions** - סוגי המנויים הקיימים ללקוחות ולחברות.
- **CompanyData** - מאגרי המידע והתוכן של החברות.
- **ThingsTheUserHas** - מידע נוסף שנשמר על המשתמש לצורך התאמה טובה יותר.
- **Reports** - דוחות סיכום שיחה שנשמרים בסיום שיחה.
- **Category** - סוגי קטגוריות.
- **CompanyCategories** - קטגוריה לחברה - במה עוסקת החברה.

תרשים DSD:

תרשים ה-DSD מציג את מבנה הטבלאות בפועל במסד הנתונים, כולל העמודות, המפתחות הראשיים והמפתחות הזרים.



תרשים ERD כללי

- ממשמש לדוחות שיחה - בקשר של יחיד לרבים.
- ממשמש לדברים שיש למשתמש - בקשר של יחיד ליחיד.
- מחברה למסדי נתוני חברה - בקשר של יחיד ליחיד.
- משתמש לסוג מנוי - בקשר של רבים ליחיד.
- מחברה לסוג מנוי - בקשר של רבים ליחיד.
- מחברה לקטגוריות חברה - בקשר של יחיד לרבים.
- מקטגוריה לקטגוריות חברה - בקשר של יחיד לרבים.

תיאור הקשרים:

- משתמש אחד יכול להחזיק מספר דוחות שיחה.
- כל דו"ח שיחה שייך למשתמש אחד.
- משתמש אחד יכול להחזיק פריטים/דברים אישיים אחד במערכת.
- כל הדברים של המשתמש שייך למשתמש אחד בלבד.
- חברה אחת מחזיקה מאגר מידע אחד.
- כל מאגר מידע שייך לחברה אחת בלבד.
- סוג מנוי אחד יכול להיות משויך למספר משתמשים.
- כל משתמש שייך לסוג מנוי אחד.

- סוג מנוי אחד יכול להיות משויך למספר חברות.
- כל חברה שייך לסוג מנוי אחד.
- חברה אחת יכולה להיות משויכת למספר קטגוריות.
- כל שיוך קטגוריה שייך לחברה אחת בלבד.
- קטגוריה אחת יכולה להיות משויכת למספר חברות.
- כל שיוך קטגוריה שייך לקטגוריה אחת בלבד.

בתרשים ה-ERD הכללי מוצגות הישויות המרכזיות במסד הנתונים והקשרים ביניהן.

טבלת Users - משתמשים

שם שדה	תפקיד	סוג נתונים	חובה
user_id	מפתח ראשי	INT	כן
username	שם משתמש	VARCHAR	כן
email	מייל	VARCHAR	לא
password_hash	סיסמה לאחר Hash	VARCHAR	כן
city	עיר	VARCHAR	לא
age	גיל	INT	לא
occupation	עיסוק	VARCHAR	לא
role	תפקיד משתמש	VARCHAR	כן
subscription_id	קוד סוג מנוי	INT	כן
subscription_start_date	תאריך תחילת המנוי	DateTime	כן

טבלת Company - חברות מנויות

שם שדה	תפקיד	סוג נתונים	חובה
company_id	מפתח ראשי	INT	כן
company_name	שם החברה	VARCHAR	כן
password_hash	סיסמה	VARCHAR	כן
is_active	האם החברה פעילה	BIT	כן
subscription_id	קוד סוג מנוי	INT	כן

subscription_start_date	תאריך תחילת המנוי	DateTime	כן
-------------------------	-------------------	----------	----

טבלת CompanyData - מאגרי מידע של חברות

שם שדה	תפקיד	סוג נתונים	חובה
data_id	מפתח ראשי	INT	כן
company_id	מפתח זר לטבלת Companies	INT	כן
title	כותרת המידע	VARCHAR	כן
content	תוכן המידע	TEXT	כן
topic	נושא המידע	VARCHAR	כן

טבלת Reports - דוחות שיחה

שם שדה	תפקיד	סוג נתונים	חובה
report_id	מפתח ראשי	INT	כן
user_id	מפתח זר לטבלת Users	INT	כן
created_at	זמן יצירת השיחה	DATETIME	כן
report_content	תוכן דו"ח השיחה	TEXT	כן

טבלת ThingsTheUserHas - דברים שיש לו

שם שדה	תפקיד	סוג נתונים	חובה
ThingsTheUserHas_id	מפתח ראשי	INT	כן
user_id	מפתח זר לטבלת Users	INT	כן
ThingsTheUserHas_content	תוכן הדברים שיש לו	NVARCHAR	כן

טבלת Subscriptions - דוחות שיחה

שם שדה	תפקיד	סוג נתונים	חובה
subscription_id	מפתח ראשי	INT	כן
subscription_name	שם המנוי	VARCHAR	כן
duration_days	מספר ימים	INT	כן
priority	עדיפות	INT	כן
price	מחיר	INT	כן
description	תיאור המנוי	VARCHAR	כן
is_active	האם פעיל	BIT	כן
subscription_type	למי מיועד המנוי: חברה/לקוח	VARCHAR	כן

טבלת Category - קטגוריה

שם שדה	תפקיד	סוג נתונים	חובה
category_id	מפתח ראשי	INT	כן
category_name	שם קטגוריה במילה אחת	NVARCHAR	כן

טבלת CompanyCategories - קטגוריה לחברה

שם שדה	תפקיד	סוג נתונים	חובה
company_category_id	מפתח ראשי	INT	כן
company_id	מפתח זר לטבלת Company	INT	כן
category_id	מפתח זר לטבלת Category	INT	כן
priority	עדיפות חברה	INT	כן

Stored Procedures / Views

בשלב זה לא נעשה שימוש הכרחי ב־Stored Procedures או Views. הפעולות המרכזיות מול מסד הנתונים מתבצעות באמצעות שאילתות רגילות: שמירה, שליפה ועדכון נתונים.

23. מדריך למשתמש

המערכת מיועדת לשלושה סוגי משתמשים: משתמש רגיל, חברה מנויה ומנהל מערכת.

התחברות למערכת

בכניסה למערכת יש להזין אימייל או שם משתמש וסיסמה. לאחר מכן יש ללחוץ על כפתור **Sign in**. אם הפרטים תקינים, המשתמש מועבר למסך המתאים לפי סוג המשתמש שלו: לקוח, חברה או מנהל מערכת. אם למשתמש אין חשבון, ניתן לעבור למסך הרשמה באמצעות הקישור **Create account**.

משתמש רגיל - לקוח

לקוח חדש צריך להירשם למערכת, להזין שם, אימייל, סיסמה, פרטי תשלום ולבחור סוג מנוי. לאחר ההרשמה וההתחברות, הלקוח נכנס למסך הצ'אט.

במסך הצ'אט הלקוח יכול לשלוח שאלות למערכת בטקסט או בקול. אם נעשה שימוש בקלט קולי, יש לאפשר לדפדפן גישה למיקרופון. לאחר שליחת השאלה, המערכת מנתחת אותה, מחפשת מידע מתאים ומחזירה תשובה. בנוסף, הלקוח יכול לפתוח שיחה חדשה, לצפות בהיסטוריית שיחות ולעדכן את פרטי הפרופיל שלו.

הדרישות מהלקוח: הרשמה למערכת, בחירת מנוי, כתיבת שאלות ברורות ככל האפשר, ובמקרה של קלט קולי – אישור שימוש במיקרופון.

חברה מנויה

חברה המעוניינת להצטרף למערכת נרשמת דרך מסך הרשמת חברה. עליה להזין שם חברה, אימייל, סיסמה, פרטי תשלום ולבחור סוג מנוי חברה.

לאחר ההתחברות, החברה נכנסת למסך עדכון מסד הנתונים שלה. במסך זה החברה מזינה את פרטי החברה, אתר החברה, אימייל תמיכה ומקור ידע. מקור הידע הוא המידע שעליו המערכת תתבסס כאשר לקוחות ישאלו שאלות הקשורות לחברה.

לאחר הכנסת פרטים אלו, על החברה להיכנס לבחירת קטגוריות ולבחור את הקטגוריות הקשורות לחברה שלו.

הדרישות מהחברה: הרשמה למערכת, בחירת מנוי, הזנת פרטי חברה תקינים, והכנסת מידע ברור ומעודכן למאגר הידע של החברה, וכן בחירת קטגוריות.

מנהל מערכת

מנהל המערכת נכנס למערכת באמצעות פרטי התחברות של מנהל. לאחר ההתחברות הוא מועבר למסך הניהול.

במסך הניהול המנהל יכול לצפות במשתמשים, לעדכן פרטי משתמשים, לנהל חברות מנויות, לשנות סטטוס פעילות של חברות, לעדכן סוגי מנויים, להוסיף מנויים חדשים ולצפות בנתונים הנשמרים במערכת. בנוסף, המנהל אחראי לוודא שהמידע במערכת תקין ומעודכן.

הדרישות ממנהל המערכת: התחברות עם הרשאת מנהל, ניהול תקין של משתמשים, חברות וסוגי מנויים, ובדיקה שהנתונים במערכת נשמרים בצורה נכונה.

24. מדריך למשתמש

במהלך פיתוח המערכת בוצעו בדיקות שונות כדי לוודא שהמערכת פועלת בצורה תקינה ומתמודדת עם סוגי קלט שונים.

בוצעו בדיקות התחברות והרשמה עבור לקוח, חברה ומנהל מערכת, כדי לוודא שכל משתמש מועבר למסך המתאים לו. בנוסף, נבדקו מסכי ניהול המשתמשים, החברות והמנויים, כולל שמירה, עדכון ורענון נתונים מול מסד הנתונים.

בצד הצ'אט בוצעו בדיקות על שאלות פשוטות ומורכבות, כדי לוודא שהמערכת מזהה נכון את סוג הפנייה, ממלאת JSON מתאים, מחפשת מידע רלוונטי ומחזירה תשובה ברורה. נבדקו גם שאלות בנושאים שונים כמו מחיר, מוצר, הזמנה, תקלה, מדיניות ותלונה.

בנוסף, בוצעו בדיקות של קלטים חסרים או שגויים, כגון שדות הרשמה ריקים, סיסמה לא נכונה, שאלה ריקה או חוסר הרשאה למיקרופון. מטרת הבדיקות הייתה לוודא שהמערכת מציגה הודעות שגיאה מתאימות ואינה קורסת.

25. המסקנות

במהלך פיתוח הפרויקט למדתי הרבה גם מבחינה מקצועית וגם מבחינה אישית. מבחינה מקצועית, הפרויקט חיזק אצלי את ההבנה בעבודה עם מערכות לקוח-שרת, מסדי נתונים, עיבוד שפה טבעית, מודלי למידת מכונה ובניית מערכת מלאה שמחברת בין כמה רכיבים שונים.

בנוסף, למדתי להתמודד עם בעיות שלא תמיד יש להן פתרון מיידי, כמו שגיאות בקוד, קושי בחיבור בין המודלים, טיפול בקלטים שונים של משתמשים ושיפור איכות התשובות של המערכת. העבודה דרשה ממני לחשוב בצורה מסודרת ומחוץ לקופסא, לבדוק כל שלב בנפרד, ולשפר את המערכת בהדרגה.

מבחינה אישית, הפרויקט לימד אותי התמדה, סבלנות וניהול זמן. הבנתי עד כמה חשוב לתכנן מראש, אבל גם לדעת לשנות כיוון כשמשווא לא עובד כמו שציפיתי. לא להיבהל מנפילות, מטעויות ומשגיאות, אלא לצמוח מהם. בסופו של דבר, הפרויקט נתן לי ניסיון משמעותי בבניית מערכת אמיתית, וחיזק אצלי את הביטחון ביכולת ללמוד נושאים מורכבים וליישם אותם בפועל.

26. שיפורים עתידיים

בעתיד ניתן לשדרג את המערכת במספר כיוונים:

1. הוספת אפשרות לדירוג תשובות על ידי המשתמש, כדי שהמערכת תוכל לקבל משוב על איכות המענה ולשפר את התשובות בהמשך.
2. שיפור דיוק המודלים באמצעות הרחבת מאגר הנתונים, הוספת דוגמאות נוספות ואימון מדויק יותר של מודלי הסיווג ומילוי ה-JSON.
3. הרחבת סוגי הפניות במערכת מעבר ל-12 הסוגים הקיימים, כדי לאפשר מענה למגוון רחב יותר של שאלות משתמשים.
4. הוספת אפשרות לצ'אט טלפוני, כך שהמשתמש יוכל להתקשר למערכת כמו לשיחת טלפון רגילה, לשאול את השאלה בקול, והמערכת תמיר את הדיבור לטקסט, תעבד את השאלה ותחזיר תשובה קולית למשתמש.
5. שיפור ביצועי המערכת באמצעות הרצה על שרת חזק יותר או GPU, כדי לקצר את זמן התגובה של הצ'אט בזמן שימוש אמיתי.

27. ביבליוגרפיה

במהלך העבודה על הפרויקט נעזרתי במספר אתרים לצורך מחקר, למידה, תכנון ופיתוח המערכת:

IBM
 Hugging Face
 PyTorch
 Scikit-learn
 FastAPI
 Microsoft Learn
 React
 arXiv
 Sentence Transformers
 GitHub
 spacy
 GeeksforGeeks

בנוסף, נעזרתי בהסברים, בדוגמאות קוד ובכלי פיתוח שונים לצורך פתרון שגיאות, הבנת מודלים של עיבוד שפה טבעית, עבודה עם מסדי נתונים, בניית ממשק המשתמש וחיבור בין צד הלקוח לצד השרת.